

One defect class in the MACH-Aero adjoint: a branch recorded on a differentiated path the slope limiter in parallel, the slope limiter in serial, the IDWarp rotation guard, the ILU conditioning wall — and three configurations the solver accepted and did not run

Certonomous CFD Lab

2026-08-10

Abstract

On a 2,777-cell OpenFOAM case run under `dafoam/opt-packages:latest` (DAFoam 5.0.0, OpenFOAM v2506), varying *only* the mesh decomposition moves the analytic $dC_D/d(\text{shape})$ gradient from 0.00054% agreement with its own finite difference (`simple` 4x1x1) to 8.95% disagreement (`scotch`, the shipped default), with the converged primal invariant to five significant figures and the adjoint right-hand side invariant to 4.8×10^{-6} . The linear solve is not at fault: it converges cleanly (`PetscConvergedReason: 2`, true residual 1.7×10^{-7} in the unpreconditioned norm) and is unmoved by a tolerance tightened four decades. The discriminating measurement is a cross-residual: the converged `scotch` adjoint vector, permuted to serial ordering by integer `decomposePar` addressing and evaluated under the serial reverse-AD operator, leaves a residual of $329 \times \|b\|$ against that operator's own 1.14×10^{-4} floor — concentrated in the x -momentum rows of partition-interface cells. The parallel matrix-free transposed-Jacobian product is not the transpose Jacobian of the residual it claims to differentiate.

What carries it is now named to one word. Changing `div(phi,U)` from `bounded Gauss linearUpwind limited` to `bounded Gauss linearUpwind default` — same scheme family, same order, the same halo-exchanged gradient-correction term still evaluated, with only the limiter's min/max selection over the stencil removed — takes the gradient error from 8.95% to 0.849%, the adjoint Krylov count from 719 to 41, and the cross-residual from $329 \times \|b\|$ to $1.35 \times 10^{-2} \times \|b\|$: a 24,270-fold collapse of the operator error, recomputed from the raw dumped vectors by an independent sweep. The other lever that cleans this gradient does not clean the operator. Swapping the farfield `U` boundary condition leaves the transposed-Jacobian product wrong at $1.047 \times \|b\|$ — 163,548 times its own serial floor — underneath a `check_totals` reading 0.019%. An earlier revision of this document called that configuration clean. It is not safe; it is quiet.

The same recorded branch then broke a case with no processor boundary in it at all. Installed on a 4,032-cell structured NACA0012, the limiter drives the *serial* adjoint to 92.8% against its own step-stable finite difference, with one of eight gradient components sign-flipped — and the identical one-word edit returns 0.121%. Both decompositions of that case agree to 3.9×10^{-4} while both are 92.8% wrong: invariance certifies consistency, not correctness.

We report the phenomenon, the discriminators that convicted the operator and exonerated every other candidate, the localization, the campaign that named the branch and separated the fix from the disguise, the serial variant and its single-process reproducer, why no published verification could have caught any of it, and two adjacent findings from the same forensic arc: an IDWarp degenerate-branch guard that differentiates to a hard zero at exactly the state every gradient evaluation uses (root-caused, patched in four lines, and answering a five-year-old open upstream issue), and an ASM sub-block ILU factorization whose failure is broken by the PETSc developers' own sanctioned escalation to complete LU.

A fourth section, added 2026-08-08 and extended 2026-08-10, reports a failure of a different kind, found while carrying that last remedy to a transonic case — and corrects a claim of our own. This lab's record held that the ONERA M6 discrete adjoint was blocked by conditioning at every mesh size tested, `PetscConvergedReason: -5` from 21,840 to 99,840 cells. Every

one of those runs set `transonicPCOption: 2`, which is dead code for the solver that read it: `DAResidualRhoSimpleCFoam.C:173` implements only `== 1`, and `== 2` lives in a different solver's source. The option parses, warns nothing, and takes no branch. Setting it to 1 — one token, on the identical configuration, same mesh, same partition, same coloring cache, stock ILU — converts both diverged solves into **reason 2** at 368 and 383 iterations, and the converged C_L solve begins at $1.839195903440 \times 10^{-1}$, the number the record's own run stalled on, and descends through it. A pre-registered negative control at the record value reproduces the recorded failure bit for bit — iteration-0 residual, the whole printed stall sequence, the terminal denormal $3.945602898014 \times 10^{-308}$ — and a pre-registered finite-difference table verifies two of four probed gradient components at 0.18% and 0.93%, with the other two reported unverdicted rather than forced. The next rung up, 42,120 cells, converges too — 987 and 1171 iterations, with all three of its probed components evaluable, step-consistent and inside 0.28% — so the correction rests on two rungs rather than one. The third rung, 79,560 cells, does *not* converge: it stagnates flat at a cap 1.6 times the predicted requirement with memory comfortable, which places a ceiling on the reopened ladder somewhere between 42,120 and 79,560 cells and breaks the two-rung iteration-scaling law by kind rather than by degree. The wall was real; at the rungs below that ceiling its cause was misattributed. A second silent configuration is reported alongside: the wrapper writes the primal end state back into time 0, so the cold start you assumed is not the start that ran. And a third, with the widest audience of the three: the adjoint solver calls `KSPSetFromOptions` and then overrides it at twelve later call sites, so `-ksp_type`, `-pc_type` and `-sub_pc_type` are accepted without error and discarded — while `-ksp_monitor_true_residual` and `-ksp_view_pmat` work perfectly. The same channel that hands a user a correct diagnosis silently drops the remedy it implies, which this lab established from both ends: ten candidate causes of the ceiling are eliminated by measurement through that channel — including diagonal spread, by the decisive observation that 14.40 decades converges and 14.47 does not, and non-normality, by the stalling rung measuring $1.62\times$ *less* non-normal than the converging one — and every remedy those eliminations point to next is one the channel discards.

That last defect is no longer a proposal. The fix this report recommends has since been implemented on a locally patched image and regression-controlled: with the option channel honoured and no options set, the reference case returns the same 368 and 383 Krylov iterations, **reason 2** on both, and the same cold-start signature to sixteen digits — so the override is measured *not* load-bearing, which is the maintainer's first objection answered by measurement rather than by argument. Honouring the channel then made two of the discarded remedies runnable for the first time: `-ksp_type lgmres` and `-pc_type gamg`, both off-the-shelf at documented defaults, both reporting the control's own iteration-0 residual, and both failing — the eleventh and twelfth eliminations. Nothing run on a patched image revises the shipped-toolchain ceiling, and nothing here claims more than that two off-the-shelf members of that class fail on this operator. **Nothing in this document has been filed upstream.**

Status. No issue has been opened, no maintainer contacted, nothing posted. This document is the extended, citable companion to two prepared-but-unfiled reports, `demo-output/website/dafoam/UPSTREAM_BUG_REPORT_decomposition_adjoint.md` and `demo-output/website/dafoam/UPSTREAM_BUG_REPORT_mesh_warpDeriv.md`. It does not replace them. Both remain **PREPARED**: complete, carrying the 2026-08-07 addenda these sections summarise, and *held*. A third prepared document has since joined them — `demo-output/website/dafoam/DEFECT_CANDIDATE_ksp_options_override.md`, the option-channel defect of Section 7.12, **filing-ready and not filed**. That one now carries an implemented, regression-controlled patch of its own recommended fix (Section 7.12); the patched image is local, nothing is pushed anywhere, and the document's status is unchanged by it. The other two silent-configuration findings of Section 7 have no prepared report at all — only the measured record cited there. Submissions are parked at the owner's discretion; whether any of this is sent upstream is the lab principal's call, not the authors'. Where our own record was found defective by our own verification, the defect is printed here rather than repaired silently — including, in Section 7, a causal claim of ours that this lab's own arms overturned.

Contents

1	Summary of findings	4
2	The decomposition-dependent adjoint	8
2.1	The phenomenon	8
2.2	The discriminators	9
2.3	The objection this measurement must survive	11
2.4	Sign-convention provenance, stated against ourselves	11
2.5	Localization	12
2.6	The mechanism, named at operator level	12
2.7	A usability finding recorded on the way	13
2.8	How far the defect reaches	14
2.9	The branch, named to one word	15
2.10	Operator or contraction: the two levers are not the same kind of fix	17
2.11	What the campaign eliminated, and the confound it did not	18
3	The same branch, in serial	19
3.1	The arm	19
3.2	Convicting the analytic, not the finite difference	20
3.3	The attack that could have killed it	21
3.4	What the serial variant is, and what it changes	21
4	Why it was never caught	22
4.1	The papers' own protocols	22
4.2	The community record	24
5	The IDWarp rotation guard	25
5.1	The line	26
5.2	Narrowing the claim, against ourselves	27
5.3	Why upstream's CI cannot see it	27
5.4	The upstream issue this answers	28
5.5	The patch, and what it proves	28
5.6	The second regime, and the fix that covers both	29
5.7	Workaround, with its price named	30
6	The conditioning wall, and the sub-LU remedy	30
6.1	The wall	30
6.2	The offline reproducer, which made every candidate a ten-second experiment	30
6.3	Runtime options: reachable, measured, and empty	31
6.4	The one-word change, offline and then in-solver	31
6.5	The gradient, and its verification	32
6.6	Where the remedy comes from, and why it is not exotic	32
6.7	What the change does <i>not</i> fix	33
7	The option that parsed and did nothing	33
7.1	The record this section corrects	33
7.2	The lever that was never pulled	34
7.3	Two not-evaluables, printed at their cost	35
7.4	The epilogue: one token, on the identical configuration	35
7.5	The control that could have taken it back	36
7.6	The gradient, and the two components it does not certify	37
7.7	The second rung, and a negative reason code refused	37

7.8	The anomaly that was over-generalised, not invented	38
7.9	The third rung, and the ceiling it found	40
7.10	What the ceiling is not: twelve eliminations and the number that survived	41
7.11	The second silent configuration: the time-0 rewrite	45
7.12	The third silent configuration: an option channel that answers and does not obey .	45
7.13	The class the build forbade, tested at last: the eleventh and twelfth eliminations .	48
7.14	A diagnostic that does not diagnose: three arms on the condition estimate	49
7.15	What is reportable here, and what is not	50
7.16	Scope, stated against ourselves	50
8	Methods: adversarial verification as practice	51
8.1	What a supervisor sweep is	52
8.2	What the sweeps actually found	53
8.3	Two audits larger than a sweep — one of which was itself wrong	53
8.4	Pre-registration	55
8.5	Cost, stated including the overruns	56
8.6	The lessons this arc produced	58
A	Reproduction appendix	59
A.1	Environments	59
A.2	Switches and stamps	59
A.3	Varying the decomposition without falling into the trap	60
A.4	The cross-residual protocol	60
A.5	The two one-word reproducers	61
A.6	The third one-token reproducer, and how to prove it ran	62
A.7	Reproducing the IDWarp defect in isolation	62
A.8	Rebuilding the sub-LU image	63
A.9	Artifact index	63
A.10	What is still open	67

1 Summary of findings

Seven findings, from four weeks of forensic work on the MACH-Aero/DAFoam toolchain. Three of them are one defect class — a *branch* recorded on a differentiated path, whose selection the reverse sweep gets wrong — in two repositories, in parallel and in serial. The fourth is a wall in a shipped default with a remedy that is one option away. The last three share a shape with each other and not with the first four: configuration the tool accepts and does not run — an option that is dead code for the solver that read it, a state rewrite that no log line announces, and an entire PETSc option channel that answers diagnostics faithfully and discards the remedies typed into it.

Finding	Where	Headline measurement	Status
<i>The class: a branch recorded on a differentiated path.</i>			

Finding	Where	Headline measurement	Status
D1. The limiter, in parallel: a decomposition-dependent adjoint	DAFoam v5, matrix-free (Jacobian-free) adjoint; cellLimited Gauss linear 1 feeding linearUpwind , recorded on the reverse tape across processor boundaries	Gradient error 8.95% at np=4 scotch vs 0.00054% at np=4 simple 4x1x1 on the same mesh, same script, same container. Cross-residual of the converged scotch adjoint under the serial operator: 3.288×10^2 of $\ b\ $, against a 1.14×10^{-4} serial floor. One word of fvSchemes (limited → default) takes those to 0.849% and 1.35×10^{-2} , and the Krylov count from 719 to 41	Mechanism named to a recorded construct, not to a source line. Report drafted with two addenda, not filed . No prior report of the decomposition phenomenon anywhere (63 searches, 10 venues); the limiter result post-dates that sweep and has had none
D1s. The same limiter, in serial	Same cellLimited feeding linearUpwind , on A1's 4,032-cell structured NACA0012 at np=1 — no processor boundary anywhere in the case	$dC_D/d(\text{shape})$ 92.8% against its own finite difference, which moves 2.10% between $h = 10^{-3}$ and 3×10^{-3} — 44x too little to explain the gap; C_L/shape 7.9%; one of eight components sign-flipped, with both one-sided FD slopes on the wrong side of zero from the analytic. np=1 and np=4 agree to 3.9×10^{-4} while both are 92.8% wrong. The same one-word edit reads 0.121%	Decomposition-independent — the first in this class. Single-process reproducer, one-word on/off. Carried into the D1 report as its 2026-08-07 addendum, not filed
D2. Degenerate-branch rotation guard	IDWarp 2.6.2, src/utils/vectorUtils.f90:58 , reverse mode vectorUtils_b.f90:123–128	warpDeriv discards the surface-normal-rotation term exactly whenever current normals equal reference normals — the state of every gradient evaluation. 210%/213% (upstream's own test), 634% sign flip, 207%/122% sign flips; all collapse to FD-truncation level under a four-line analytic-limit patch, primal md5-identical	Root cause confirmed by repair. Matches open upstream issue mdolab/idwarp#57 (2021-07-14, zero comments). Comment-on-#57 prepared, not filed
<i>Adjacent: a shipped default whose own remedy ladder does not contain its remedy.</i>			
D3. ILU conditioning wall	DAFoam v5, DALinearEqn.C: 266–267 : ASM sub-block PC hard-coded to PCILU	PetscConvergedReason: -9 (DIVERGED_NANORINF) at iteration 0 on a 21,000-cell closure case becomes reason 2 in 667 iterations under complete LU in the sub-blocks; the resulting 21,000-component field-inversion gradient verifies against central differences at 0.085%, 0.059%, 0.199%	Fixed locally by a one-hunk, default-off source patch. PETSc-sanctioned remedy; PR-ready in principle, not filed

A second class: configuration the solver accepts and does not run.

Finding	Where	Headline measurement	Status
D4. An option that is dead code for the solver reading it	DAFoam v5, DAResidualRhoSimpleCFoam.C:173 (only == 1 implemented); == 2 exists at DAResidualTurboFoam.C:176, a different solver. Read by an unvalidated <code>getOption</code> lookup and echoed back live in the settings dump	Every archived ONERA M6 adjoint in this lab set <code>transonicPCOption: 2</code> and none of them ran it, so the recorded “conditioning wall at every mesh size” (–5 at 21,840–99,840 cells) was measured with the transonic preconditioner silently off. 2 → 1 alone — identical mesh, partition, coloring cache, stock ILU — takes both solves from <code>DIVERGED_BREAKDOWN</code> to reason 2 at two rungs (21,840 cells: 368/383 iterations; 42,120 cells: 987/1171). Pre-registered negative control at the record value reproduces the record bit for bit; FD tables PASS at both rungs (rung 1: 2 of 4 probed components, 0.18%/0.93%; rung 2: all three, 0.0077%/0.2740%/0.0172%). Rung 3 (79,560 cells) stagnates flat at a 4,000 cap with memory comfortable (11.65 of 22 GiB): a ceiling, bracketed and not located. A second recorded conclusion is rescoped rather than reversed: R5’s “strengthening the PC reintroduces collapse” is real and <i>mesh-dependent</i> — with the PC active, Richardson is the largest iteration cut at 21,840 cells (–44%) and collapses at the first restart boundary at 42,120	One claim of ours corrected and one rescoped, at two verified rungs under a measured ceiling. Upstream behaviour — silent acceptance of an inapplicable value — described, not filed ; no prepared report exists
D5. A state rewrite with no log line	DAFoam v5 pyDAFoam: the primal end state is written back into the time-0 directory (0/U bit-identical to 1000/U); pyDAFoam.py:1543 then hard-raises on the leftover time directory it created	Every rerun of a case directory silently warm-starts. Discriminator: first Time step continuity errors , cold-from-uniform 0.597 vs warm 0.0107 (55×), bit-reproducible across six cold starts. Audited backwards over nine conclusion-bearing records at zero solver cost: seven cold-clean (staged per-arm copies, bit-identical start states), two flagged, none load-bearing	Instrument hazard, measured and audited. Not filed

D6. An option channel that answers and does not obey	DAFoam v5, DALinearEqn.C:138 calls KSPSetFromOptions; twelve later call sites override it — GMRES at 144, ASM at 212, ILU at 286, plus restart, PC side, tolerances, overlap and fill	-ksp_type lgmres, -pc_type gamg, -pc_type fieldsplit, -sub_pc_type lu and the rest are accepted without error and have no effect; -ksp_view prints the hardcoded stack. Monitors and viewers are NOT overridden — -ksp_monitor_true_residual and -ksp_view_pmat work perfectly — so the diagnostic path demonstrates the channel is live while the remedy path discards the instruction. Measured instance: this lab diagnosed a preconditioned $\kappa = 9.57 \times 10^{10}$ through that channel and could reach none of the remedies it implies. The recommended reorder is now implemented and regression-controlled: with it in force and no options set, the reference case reproduces 368/383 iterations and reason 2 bit-identically, and -ksp_view then reports type: fgmres where the shipped build reports type: gmres	Diagnosability defect; no wrong number produced. Twelve call sites tabulated, three graded fixes drafted, reproducer 5 minutes. Fix (a) implemented on a local image, measured behaviour-neutral by default and measured to restore the channel; recommendation revised to (a) plus an effective-value echo , because the build's own echo goes stale under an override. FILING-READY, not filed
--	---	--	--

Table 1: The seven findings. Sources: Sections 2, 3, 5, 6, and 7 for the last three.

What links the first three. A guard, a limiter, and a switching boundary condition are the same kind of object: a `min`, a `max`, an `abs`, or an `if` evaluated on data and then recorded on a tape. Reverse-mode AD differentiates the branch that was taken, correctly and by design. What it cannot do is know that the branch it recorded is the wrong function of the inputs — a constant where the true map has a first-order term (D2), or a selection whose stencil reaches across a processor face and whose reverse does not (D1), or one that is simply mis-swept in a single process (D1s). The class is cheap to hunt once it is named: every member here is removable by a one-line dictionary edit or a four-line derivative, which makes it the cheapest discriminator available, and it was one such edit that closed D1's mechanism hunt.

What links all seven. They are invisible to the checks that ship with the tools. D2's discarded term sums to *exactly* zero over the surface, and the sum is the only quantity upstream's test asserts on. D1 produces a gradient that is self-consistent under the decomposition it was measured on, and no published verification varies the decomposition. D1s is self-consistent across decompositions too — it passes the invariance gate this lab adopted after D1, to 3.9×10^{-4} , while being 92.8% wrong. D3 announces itself loudly, but the documented remedy ladder does not contain the remedy. D4, D5 and D6 are the degenerate case of the same problem: the instrument reports the dictionary, and the dictionary is not the run — an option echoed back verbatim while no branch takes it, a start state replaced between runs with nothing printed, and a command line that is parsed, delivered, and overwritten before it reaches the solve. D6 is the sharpest of the three, because the same channel that drops the user's remedy hands back their diagnosis correctly — and because, when the remedy is finally forced through on a build patched to obey, the solver's own setup echo still prints the switch that did *not* run (Section 7.12). That is this document's own L-40 lesson occurring inside the code the document is about. Each was found by measuring the thing the instrument was not looking at.

What D4 and D5 cost, and why they are here. Neither is a wrong number. D4 produced nine days of correct measurements assembled into one incorrect sentence and one over-general one — the expensive kind of defect, because every individual figure survives the audit. The first was causal: a wall attributed to conditioning, and it dissolves when the option is activated.

The second outlived it by weeks and is subtler: the recorded finding that strengthening the preconditioner reintroduces collapse on this family, which made every documented escalation look counterproductive. That observation turns out to be *true and mesh-dependent* — it reproduces on a live preconditioner one rung up — so what the dead lever cost there was not the finding but its width (Section 7.8). A wrong wall stops work; a correct observation recorded at the wrong scope redirects it. D5 produced no wrong conclusion at all, on this evidence, and is reported because the audit that established that is the only reason anyone knows. A defect report that only lists other people’s wrong numbers is not the same document as one that lists the ways a toolchain can quietly make you wrong.

2 The decomposition-dependent adjoint

2.1 The phenomenon

The case is A4, a coarse Ahmed body: 2,777 cells, snappyHexMesh-class grid with refinement interfaces, DASSimpleFoam, kOmegaSST, one FFD shape design variable. The instrument is OpenMDAO’s `check_totals`, whose finite-difference column is computed *per run, inside that run’s own check_totals* — not computed once and reused. Only the decomposition varies across the rows of Table 2; verified by `diff` of every run directory against the source case (the np runs carry a byte-identical `runScript.py`; the `simple` runs differ by exactly one inserted `decomposeParDict` daOption line; the tolerance run by exactly the `gmresRelTol` value), and by reading the `Decomposition method` line back out of each log.

configuration	log	analytic	FD (same run)	rel. err	
np=1, stock	a4_np1_stock.log	2.3965e-01	2.4232e-01	1.10%	
np=1, patched IDWarp	a4_np1_patched.log	2.4150e-01	2.4232e-01	0.34%	
np=2 scotch	a4_np2_patched.log	2.4118e-01	2.4182e-01	0.26%	
np=3 scotch	a4_np3_patched.log	2.5641e-01	2.4178e-01	6.05%	analytic high
np=4 scotch	a4_np4_patched.log	2.2086e-01	2.4258e-01	8.95%	analytic low
np=4 scotch, gmresRelTol 1e-10	a4_np4_ tol1.0e-10.log	2.2086e-01	2.4258e-01	8.95%	811 iters, reason 2
np=4 simple 4x1x1	a4_np4_ simple4x1x1.log	2.4220e-01	2.4220e-01	0.00054%	
np=4 simple 1x4x1	a4_np4_ simple1x4x1.log	2.4379e-01	2.4265e-01	0.47%	
np=4 simple 4x1x1, stock	a4_np4_simple411_ stock.log	2.4037e-01	2.4220e-01	0.76%	
np=4 scotch, stock	a4_h1e-3_stock.log	2.1821e-01	2.4258e-01	10.04%	= published

Table 2: The verified phenomenon: $dC_D/d(\text{shape})$, analytic against each run’s own finite difference. Every cell re-extracted from the run logs by an adversarial supervisor sweep, `VERIFICATION_A4_decomposition_supervisor_sweep.md` (27D25762), Axis 1. Logs under `/home/ubuntu/certonomous-runs/W4-a4-stepsweep/`. “patched” denotes a locally patched IDWarp removing the independent defect of Section 5; the decomposition effect exists stock too, as the last two rows show.

Four properties of this table decide everything that follows.

1. **The primal does not move.** Converged baseline C_D spans 0.15296979–0.15297237 across *all* configurations — invariant to five significant figures. np=1 and np=4 `scotch` differ by 5.5×10^{-7} absolute, 3.6×10^{-6} relative: separately reconverged SIMPLE solves at `primalMinResTol` 1e-4, i.e. reconvergence noise, four orders of magnitude below the 8.95% gradient shift. The `simple` arm — the arm with the *correct* gradient — differs from np=1 by *more* than `scotch` does.

2. **The finite difference does not move meaningfully.** The FD column spans (2.4178–2.4265)e-01, a 0.36% spread, against an 8.95% analytic excursion.
3. **Rank count is not the trigger.** np=2 **scotch** sits at the np=1 floor (0.26%); np=3 is catastrophic in one direction (6.05%, analytic high) and np=4 in the other (8.95%, analytic low); np=4 **simple** 4x1x1 is the cleanest configuration in the table. What changes between the dirty and clean rows is the *shape and orientation of the cut*.
4. **The solve is not the problem.** Tightening gmresRelTol from 1e-6 to 1e-10 costs 811 GMRES iterations instead of 719 and changes the answer by nothing. The np=4 **scotch** solve reports PetscConvergedReason: 2 at final true residual 1.749572×10^{-7} , and DAfoam’s own solver configuration makes that literal: DALinearEqn.C sets KSPSetPCSide(PC_RIGHT) and KSPSetNormType(KSP_NORM_UNPRECONDITIONED), so “converged in the unpreconditioned norm” means what it says.

This is the signature of a *converged wrong answer*. It is also the signature that makes the defect survivable in production: nothing in the solver’s own telemetry is anomalous.

2.2 The discriminators

Four candidate carriers were named in advance and executed cheapest-first (DISCRIMINATORS_A4_decomposition_mechanism.md; run artifacts under /home/ubuntu/certonomous-runs/W4-a4-discriminators/, with per-arm directories, ledger.txt, runScript_w4.py, build_maps.py, analyze_mats.py).

M3(a) — the primal, exonerated at zero cost. Point 1 above, plus this session’s own arms: baseline C_D 0.1529738 / 0.1529749 / 0.1529732 (np1 / scotch / simple), and mapped primal *state vectors* agreeing across decompositions at 2.0×10^{-3} / 2.8×10^{-3} relative — with the duplicated processor-face ϕ copies agreeing to 2.7×10^{-15} , machine precision. That last number does double duty: it validates the permutation used by every measurement below.

M3(b) — the mesh-derivative chain, exonerated by measurement. The dCD/dXv capture hook (a real reverse seed on DAfoamWarper) was run at all three decompositions with the warp finite-difference directional derivative dXv/dshape computed identically in each arm:

arm	$\langle dC_D/dX_v, dX_v/ds \rangle$	framework compute_totals	agreement
np=1	2.414994844096177e-01	2.414994857859927e-01	5.7e-09
np=4 scotch	2.208588587555006e-01	2.208588593817390e-01	2.8e-09
np=4 simple 4x1x1	2.422030559126777e-01	2.422030573188107e-01	5.8e-09

The hand composition equals the framework total to 6×10^{-9} in *every* arm: the warp/DVGeo leg composes exactly at np=4 too. The warp direction itself is decomposition-invariant — dXv/dshape mapped point-by-point via pointProcAddressing matches np=1 at **0.0e+00** in both np=4 arms. What differs is the captured dCD/dXv seed: mapped to np=1 point ordering, **scotch** differs from np=1 by **38.4%** in norm, **simple** by 3.1%. The IDWarp/DVGeo leg is exonerated; dCD/dXv is wrong because the ψ it contracts is wrong.

M2 — coloring and preconditioner, exonerated by architecture and by measurement. Both dRdWTPC and the full assembled dRdWT were dumped in all three arms and permuted to np=1 ordering. The diffs do not correlate with the defect: full-dRdWT Frobenius diff 4.8×10^{-2} (**scotch**) against 5.4×10^{-2} for **simple** — *the FD-exact arm*; PC diff 3.7×10^{-4} against 3.7×10^{-3} . This is a recorded negative, and the reason it is uninformative is itself measured: at np=1 the assembled dRdWT applied to ψ already disagrees with the AD product by $230 \times \|b\|$. The assembled matrix is

a crude finite-difference object; only the preconditioner is built from it, which is precisely why the KSP operator is matrix-free. Diff operator *actions*, not operator *entries*.

Architecturally the exoneration is stronger still: in the Jacobian-free path the coloring machinery builds only `dRdWTPC`, and a wrong preconditioner cannot move a right-preconditioned GMRES solution that converged in the unpreconditioned norm (`DALinearEqn.C:170,313`) — which this solve did.

M1 — the cross-residual, which convicts the operator. Each arm’s converged adjoint ψ was dumped and permuted to `np=1` global state ordering by an *exact integer map* built from `cellProcAddressing` / `faceProcAddressing` (sign carried for flipped faces) plus DA Foam’s own `AdjointIndexing` dumps (`writeJacobians: ["adjointIndexing"]`) — no coordinate matching anywhere in the construction. The true residual $\|A^T\psi + b\|$ was then evaluated under the *serial* AD operator via `calcJacTVecProduct`.

ψ from	$\ A^T\psi + b\ $ under <code>np=1</code> operator	ratio to $\ b\ = 0.1826$	$\ \psi\ $
<code>np=1</code> (control)	2.083e-05	1.14e-04	3.277e-02
<code>np=4</code> scotch	6.004e+01	3.288e+02	5.411e-01
<code>np=4</code> simple 4x1x1	1.711e-02	9.37e-02	1.814e-02

Table 3: The cross-residual. See Section 2.4 for the provenance of these values, which is not what the original record’s caption claimed.

Three decisive facts follow.

1. **The `scotch` ψ does not remotely satisfy the serial system** — $329 \times \|b\|$, 3.3 million times the `np=1` floor — while it satisfies its *own* `scotch`-evaluated operator, same instrument, same API, at 1.2×10^{-3} relative, and the KSP that produced it reports final true residual 1.7×10^{-7} (719 iterations, reason 2). Same vector, two operators, answers five orders apart: **the operators differ**.
2. **The right-hand side is not the carrier.** $b = dC_D/dW$ mapped across decompositions is invariant to 4.8×10^{-6} (**scotch**) and 6.9×10^{-6} (**simple**) relative.
3. **$\|\psi\|$ is inflated 16.5× under `scotch`** (0.541 against 0.033 `np=1` and 0.018 `simple`) at identical $\|b\|$. The extra content sits in the pressure (0.52 against 0.031) and z -momentum (0.117 against 0.005) adjoint blocks.

Confounds closed. (a) *Linearization state.* The cross-residual was re-evaluated with the `np=1` operator linearized at the `scotch` arm’s own mapped state: **scotch** reads 6.004094×10^1 , unchanged to six digits, while the `np=1` control degrades only from 1.14×10^{-4} to 1.0×10^{-3} . The state confound therefore contributes at the 10^{-3} level, five orders below the signal — and the **simple** arm’s state differs *more* from `np=1` than `scotch`’s does (2.8×10^{-3} against 2.0×10^{-3}) while its residual is 3,500 times smaller. (b) *Duplicated- ϕ averaging in the ψ map.* Replacing the average by either copy moves the residual by at most 9.1×10^{-3} , four orders below the signal. This bound was computed with the assembled matrix — the instrument M2 discredits — which the original record did not disclose; the supervisor sweep found it and reproduced the number exactly as $\|A_{\text{asm}}(\psi_{\text{copy}} - \psi_{\text{avg}})\|$. It is immaterial by margin: even inflated by the full 230× assembled-vs-AD discrepancy the bound stays roughly 30× below the 60.04 signal, and the duplicate copies disagree by at most 3.6×10^{-4} to begin with. The confound stays closed; the instrument should have been flagged.

2.3 The objection this measurement must survive

A knowledgeable reader will ask: is the `scotch` operator perhaps the *exact* adjoint of a different-but-equivalent parallel discretization? If decomposition changed the discrete residual itself at processor boundaries, the `scotch` adjoint could be consistent with *its* residual while differing from serial, and its 1.2×10^{-3} self-consistency would even fit that story.

It does not survive the finite-difference column. The FD in Table 2 is computed per run, inside each arm’s own `check_totals`. An adjoint that were the exact adjoint of an equivalent parallel discretization would match the finite difference of *that* discretization. The `scotch` analytic 2.2086e-01 disagrees by 8.95% with its own decomposition’s FD of 2.4258e-01. Combined with the fact that OpenFOAM processor patches evaluate the same interior face fluxes — no legitimate $O(1)$ residual-function difference is available — and that b maps invariantly at 4.8×10^{-6} , the supported conclusion is the strong one: **the parallel operator is not the transpose Jacobian of the residual it claims to differentiate.**

This gap was not acknowledged in the original mechanism record; the supervisor sweep found it and closed it from the prior record (`VERIFICATION_A4_mechanism_supervisor_sweep.md`, 8E0A08BC, Axis 4). It is stated here in the body, not in a footnote, because it is the first objection a maintainer will raise.

2.4 Sign-convention provenance, stated against ourselves

The values in Table 3 appear in no log, and the original record’s caption cited a log that contradicts them on its face. The instrument (`runScript_w4.py`, tasks `w4_dump` / `w4_crossres`) computed `res = Atpsi - b`, whereas the system OpenMDAO seeds is $A^T\psi = -b$. (*Past tense as of 2026-08-07: the instrument has since been repaired — see the note closing this subsection. Every number in this document was measured under the convention described here.*) Every arm’s log therefore prints the sign-degenerate `ratio=2.000000e+00` for its own-operator check, and none of the record’s own-operator numbers — the 1.14×10^{-4} floor, the 1.2×10^{-3} self-consistency, the 2.083×10^{-5} — appears in any log.

The tabulated values are exact offline sign corrections, `res + 2b`, recomputed from the dumped vectors (`w4x_res*.numpy`, `w4_Atpsi`, `w4_b`). The correction is exact, and the adversarial sweep reproduces every digit of it three independent ways:

- **Recomputation from the dumps** with the sweep’s own scripts: `np=1` 1.140697×10^{-4} , `scotch` 3.288078×10^2 (absolute 6.004094×10^1 , max entry 42.25), `simple` 9.367397×10^{-2} .
- **An independently rebuilt permutation**, constructed from `cellProcAddressing` / `faceProcAddressing` and the `AdjointIndexing` dumps by the sweep’s own parsers, bit-for-bit identical to the session’s map in `tgt` and `sgn` — and validated on a channel the original session never used: the `AdjointIndexing` dumps carry cell and face centre coordinates, and mapped `np=4` coordinates match `np=1` coordinates at **max** $|\Delta xyz| = 0.0$ over all 26,149 (`scotch`) and 26,227 (`simple`) entries. Every entry of the map is thereby validated exactly, independent of any solved field.
- **A fresh serial container run**: a new case directory staged from the same source, fresh serial primal, `w4_crossres` applied to ψ vectors mapped by the sweep’s own permutation. Raw `scotch` ratio 3.288139×10^2 ; sign-corrected 3.288078×10^2 ; `np=1` control 1.140697×10^{-4} ; `simple` 9.367397×10^{-2} (`w4-a4-verify-audit/audit_crossres.log`, 9 s, 0.30 core-min).

The defensible citation for Table 3 is therefore the dumped vectors plus the stated sign correction — not `d_crossres.log`. Anyone reproducing this protocol should fix the convention in the instrument rather than offline; `w4_crossres2`, written later, has the sign right and its log matches its numbers.

Note added 2026-08-07: the instrument has been repaired, and the repair confirms the corrections. This document’s own instruction — fix the convention in the instrument rather than offline — had not been executed, and a family supervision pass filed that as a standing high-severity finding: the two reused tasks still carried the wrong sign, and `w4x_res_*.npz` (needing $+2b$) sat beside `w4x2_res_*.npz` (already the true residual) under same-family filenames with nothing declaring which. Both sites now compute `res = Atpsi + b`; every W4D/crossres log line prints an explicit convention banner; and the sign-fixed task writes a distinct `w4xf_res_*.npz` prefix so pre-fix and post-fix dumps cannot be conflated (SUPERVISOR_FAMILY_REVIEW_2026-08-07.md, item A-1, OUTCOME FIXED; sibling repo-side commit 9796f90f).

The repair doubles as a fourth independent reproduction of the correction, and it is the cleanest of the four because it removes the offline arithmetic entirely. Run on the existing `np=1` dump with **no offline correction at all**: the old convention prints the degenerate $\|A^T\psi - b\|/\|b\| = 2.000000$, the new convention prints $\|A^T\psi + b\|/\|b\| = 1.140697 \times 10^{-4}$ — equal to *every digit* of the published offline `res + 2b` value of Table 3’s floor. The offline corrections are therefore no longer only exact by derivation and reproducible by re-derivation; they are independently confirmed by a tool that never performs them. No published number moves, and none was re-derived from a post-fix output.

2.5 Localization

The 60.04 cross-residual is not spread over the domain. By state group, **59.78 of the 60.04 norm sits in U_0 (x -momentum) rows**. By cell, entries with $|r| > 0.5$ number 15, of which **13 sit on scotch partition-interface cells and 2 at graph distance one** — none farther. All 15 sit on `cellLevel 0` cells, away from the mesh’s refinement interfaces. The two worst (42.25 and 42.24) sit at $y = -0.1947$ and $y = +0.1949$: the y -normal cut signature.

Three structural facts sharpen this.

- **It is not a many-rank corner effect.** The worst cells each touch exactly *one* foreign rank through exactly *one* processor face; cells touching two to four foreign faces carry *smaller* residuals.
- **Cut orientation, not cut jaggedness alone, sets the magnitude.** The hierarchy in Table 2 is orientation-ordered: x -normal planar slabs 0.00054%, y -normal planar slabs 0.47%, *scotch*’s jagged mixed-orientation boundary 8.95% — and the worst cross-residual entries sit on y -normal processor faces.
- **The same instrument on the clean arm is five thousand times quieter.** Applied to the *simple* $4 \times 1 \times 1$ ψ , the serial operator maxes at **0.008439** on that arm’s own interface cells (9 of its top 10 entries sit on them) — $5,006\times$ below *scotch*’s 42.25. Had *simple*’s residual also been huge, the test would have proven nothing.

The hanging-node hypothesis, refuted backwards. The natural first guess on a refinement-interface mesh is that partition cuts crossing hanging nodes are the trigger. The supervisor sweep re-classified all 7,843 internal faces itself, from `owner/neighbour/cellLevel` and the archived `cellDecomposition` maps: 456 refinement-interface faces, of which *scotch* cuts **4** (0.88%) and *simple* $4 \times 1 \times 1$ cuts **68** (14.91%). *The decomposition that cuts seventeen times more refinement interfaces is the accurate one.* The hypothesis is refuted, not merely unconfirmed.

2.6 The mechanism, named at operator level

Named. Under decomposition, DAfoam v5’s reverse-mode AD product $(\partial R/\partial W)^T v$ — the global-tape shell operator the adjoint KSP iterates (`initializeGlobalADTape4dRdWT / dRdWT-MatVecMultFunction`, `DASolver.C:1364ff`) and the `calcJacTVecProduct` API that reverses the same

recorded computation — differs from the true transposed Jacobian of the discrete residual at partition-interface cells, by amounts that depend on the cut’s shape and orientation.

The two parallel AD paths sit *together* on the wrong side: they agree with each other to 1.2×10^{-3} of $\|b\|$ while both sit 3.3×10^2 of $\|b\|$ from the serially evaluated truth. The defect is therefore common to the recorded reverse computation, not to one call site. GMRES then solves the wrong system exactly; the adjoint inherits $O(1)$ error along the directions the wrong operator mislabels (hence the $16.5\times$ norm inflation); and the gradient contraction $\psi^\top \partial R / \partial X_v$ delivers the 8.95%. Every observed symptom follows: converged wrong answer, tolerance invariance, decomposition dependence, primal/FD/RHS invariance, and the near-invariance of the lab’s conformal-mesh cases.

Not claimed, at the time this was first written. The defective source line. The candidate surface was stated as the reverse-AD handling of inter-processor halo exchange inside the recorded residual evaluation — the tape over OpenFOAM processor-boundary updates — and pinning the line requires an instrumented rebuild of `libDASolver.so`. What was pinned: the subsystem, the spatial signature (interface-cell x -momentum rows), the orientation dependence, and the discriminating instrument that any candidate fix must satisfy.

What has since been pinned. The construct the tape records. A campaign registered to attack a different weakness came back with the carrier in the case’s own `fvSchemes`: the slope limiter’s `min/max` selection over a stencil that reaches halo cells. One word removes it, and removing it collapses the operator error by four and a half orders of magnitude (Section 2.9). The line is still not pinned — but the search space has narrowed from “the tape over halo exchange” to “the reverse sweep of `cellLimited/limitedGrad`”, and it now has a *single-process* reproducer to instrument (Section 3), which is far cheaper than a parallel one.

A continuum, not a binary. The FD-exact `simple 4x1x1  $\psi$` still leaves 9.4×10^{-2} of $\|b\|$ under the serial operator — roughly $800\times$ the `np=1` floor — while its gradient is correct to 0.00054%. Every `np=4` operator tested differs measurably from serial; `scotch`’s jagged cut is the catastrophic point on a continuum, not a separate species.

2.7 A usability finding recorded on the way

The last and most expensive discriminator — run with coloring disabled — could not deliver its number, and the reason is worth reporting to upstream in its own right.

`adjUseColoring: False crashes the v5 mphys Krylov path by construction:` `mphys_dafoam.solve_linear` skips `runColoring()` when the option is `False`, but `calcRdWT` unconditionally calls `readJacConColoring()`, which reads a file that was never written and aborts in `DAColoring::validateColoring`. The static chain was re-confirmed in the shipped container by the supervisor sweep: `mphys_dafoam.py:458` skips `runColoring()` unless `adjUseColoring` is `true`; `DASolver.C:1043` calls `readJacConColoring()` unconditionally; and `DAColoring::validateColoring` (beginning line 931) reaches its `FatalErrorIn ... abort` at approximately `DAColoring.C:1021`.

The identity-coloring branch (one colour per global column) is reachable only through `runColoring()`, so the arm was rerun with an explicit `runColoring()` call before `compute_totals`: identity coloring accepted, `nJacConColors: 26149`, preconditioner partials computed by pure per-column finite difference with no colour compression. **The forced path is memory-unbounded.** On this 2,777-cell case (colored path peaks under 2 GiB) the first forced attempt thrashed an 8 GiB container cap and was killed at 701 s; a second at a 20 GiB cap completed all 26,149 column evaluations in 258 s and then exhausted 20 GiB in the post-assembly stage — consistent with per-column FD columns of an elliptically coupled residual being nearly dense — and was killed thrashing at 591 s.

The pre-registered prediction for this arm (“the gradient will not move, because coloring builds only the preconditioner”) is scored **NOT SCORED**: the discriminator never met its measurement. The coloring exoneration does not rest on it; it rests on architecture plus M1, as set out above.

2.8 How far the defect reaches

The trigger condition is bracketed by measurement rather than asserted from one case. Four cases in this lab are already on record as decomposition-invariant — A1 (4,032-cell structured airfoil, np=1 vs np=4: relative error 3.795529×10^{-4} vs 3.807681×10^{-4}), A2 (38,304-cell conformal wing), A5 (4,800-cell conformal U-bend, six 27-component gradient groups all invariant at norm level, worst group-norm difference 3.8×10^{-4}) — and every clean case is conformal while the only dirty case is the only snappy-refined mesh tested. That confound is the target of a pre-registered breadth matrix, `DEFECT_REACH_decomposition_cases.md` (pre-registration committed at 99F5D41D before any arm ran).

arm	configuration	pre-registered prediction
N1	A4, np=4 simple 1x1x4 (z-normal slabs)	rel. err in [0.05%, 1.5%]: planar slabs of any orientation benign-to-moderate
N2	A4, np=4 simple 2x2x1 (4-rank corner line)	rel. err $\leq 1\%$. Decision rule: if $> 2\%$, “scotch-style jaggedness required” is REFUTED
N3	Ahmed-35, a <i>new</i> snappy-refined case built by A4’s own recipe, np=1	control: rel. err $\leq 1.5\%$
N4	Ahmed-35, np=4 scotch	headline : defect appears, rel. err $\geq 2\%$, analytic differing from N3 by $\geq 2\%$ while FD stays within $\sim 0.5\%$
N5	Ahmed-35, np=4 simple 4x1x1	clean: rel. err $\leq 1\%$ regardless of N4
N6	CBFS, 21,000-cell conformal blockMesh, 21,000 beta DVs, simple 4x1x1 vs the scotch record arm	decomposition-invariant: $\leq 1\%$ per component at the three recorded FD cells, norms to $\leq 2\%$
N7	Ahmed-35 cross-residual (contingent on N4)	$\geq 10 \times \ b\ $ under the np=1 operator, on momentum rows of interface cells

Table 4: The pre-registered reach matrix. Predictions committed before any solver ran; scoring is HELD / NOT HELD / NOT SCORED exactly as written, with no post-hoc bands.

Status at the time of writing (2026-08-05). All arms have run — the ledger at `/home/ubuntu/certonomous-runs/W4-defect-reach/ledger.txt` records thirteen entries from the Ahmed-35 mesh generation through the contingent N7 cross-residual — but the results section of the reach record is still being written and its predictions are not yet scored. One result has been relayed from that work in progress: **N6, CBFS, is decomposition-invariant at 1.1×10^{-4}** , consistent with the pre-registered prediction and with the conformal-mesh pattern. *The remaining arms are marked in progress here deliberately; this document will not pre-empt a pre-registered scoring with numbers read out of raw logs.*

Addendum, 2026-08-05: the reach matrix, scored and audited. The campaign has since been scored against its registered wording and adversarially audited (`DEFECT_REACH_decomposition_cases.md`, results appended; supervisor sweep `demo-output/website/dafoam/VERIFICATION_reach_matrix_supervisor_sweep.md`, 192c970c — five audit axes CONFIRMED, one CONFIRMED with a named evidential caveat). The trigger claim it confirms reads: a **freestreamVelocity** farfield U BC in the recorded tape is *necessary* for the defect, and the cut geometry modulates it (**scotch** np ≥ 3 worst, planar slabs benign). The wrong-operator defect reproduces on Ahmed-35 —

cross-residual $5.45 \times \|b\|$ under the serial operator against a 3.98×10^{-4} np=1 floor, recomputed by the sweep to every printed digit under a byte-identically rebuilt map validated at max $|\Delta xyz| = 0.0$ — and a within-case separating control (N9: `inletOutlet` farfield, *no* `patchVelocity` input, same mesh, same np=4 `scotch` cut, registered pre-run at 15:23:54Z with 20 s of margin) lands the analytic in the clean class, print-identical to the `patchVelocity`-registered arm. Two confounds travel with the confirmation and bound the breadth claim: (i) Ahmed-35 is a deliberately controlled *sibling* of A4, not an independent geometry — same 2,777-cell background mesh family, same 25,821-state serial system, same 328-face `scotch` cut hitting the same dominant serial cells — so the defect side of the seven-case BC survey is one case family, not two independent draws; (ii) across the survey’s external-flow cases `freestreamVelocity` is perfectly anti-correlated with `patchVelocity` registration, so the cross-case survey alone cannot separate the BC from the registration. The N9 within-case swap breaks both confounds, and the causal weight of the trigger claim sits there; the survey is corroboration, not proof. This addendum supersedes the in-progress markers for the reach matrix elsewhere in this document (the artifact index and the open-items list).

Two corrections to this addendum, both from work that came after it and both printed in the sections below. First, the necessity claim survives but sufficiency does not: the same BC installed on a clean case from a different mesh family produces no defect at all (Section 2.11), and the ingredient the survey was missing turns out to be the limiter. Second, and more consequentially, the N9 control was read here as putting the `inletOutlet` configuration “in the clean class”. That is true of its gradient and false of its operator, and the difference was measured (Section 2.10). The sentence to keep is not that `inletOutlet` is safe.

Two record facts the matrix already leans on need no new run. First, the np=2-clean / np=3-dirty contrast (0.26% vs 6.05%) rules out rank count as the trigger by itself. Second, the “cut through the refinement region” discriminator is already on record backwards, as set out in the localization above — and, since this paragraph was written, forwards as well, by a controlled pair (Section 2.11).

2.9 The branch, named to one word

The campaign that named the carrier was not looking for it. It was registered to attack the weakness the reach sweep had named — every defective measurement in this lab lived on one 2,777-cell Ahmed background mesh — by transplanting the trigger onto a second mesh family, by refining the first one, and by turning setup knobs that should not matter (`DEFECT_ROBUSTNESS_mesh_and_setup.md`: pre-registration 25F52868, amendments 671F40BB, C22B0F00, B0AA6102, 4EA782E3, results 49631723; run artifacts under `/home/ubuntu/certonomous-runs/W4-defect-robustness/`).

The knob arm was registered as a formality. R3a changed the discretization — convection scheme and default gradient scheme together — with the prediction that the error would stay $\geq 4\%$, because “the defect lives in the reverse tape of the parallel coupling, which every convection/gradient scheme feeds through the same processor-patch machinery.” It came back at **0.228%**, on the same mesh and the same `scotch` partition that reads 8.95%, and the registered decision rule — “error $\leq 1\%$ means the defect is scheme-specific — REOPENS the mechanism question, reported loudly” — fired. R3a had moved two knobs, so it convicted neither. Two separating arms were registered before either ran; then two sharper ones. The result is a ladder of four convection schemes — plus the gradient-scheme arm tested alongside them — in which every row keeps the same mesh, the same np=4 `scotch` cut and the same `freestreamVelocity` farfield, and is judged against its own in-run finite difference.

R6b is one word, and the sweep checked that literally. limited names A4’s `cellLimited Gauss linear 1` gradient scheme; `default` names its unlimited `Gauss linear`. A diff of the two case directories shows exactly one changed line, in `system/fvSchemes`, with `fvSolution`, `controlDict`, `decomposeParDict`, `blockMeshDict`, `snappyHexMeshDict`, the whole of `0.orig` and `runScript.py` all

div(ϕ ,U)	limiter	2nd	grad-corr.	analytic	FD (own run)	rel. err	KSP
linearUpwind limited (established)	yes	yes	yes	2.2086e-01	2.4258e-01	8.95%	719
linearUpwind default (R6b)	no	yes	yes	2.0572e-01	2.0399e-01	0.849%	41
Gauss linear (R6a)	no	yes	no	2.0299e-01	2.0267e-01	0.157%	223
upwind (R3a1)	no	no	no	2.7681e-01	2.7670e-01	0.041%	41
gradSchemes default limited only (R3a2)	yes	yes	yes	3.1706e-01	3.2562e-01	2.63%	780

Table 5: The scheme ladder. “limiter” is the $\min/\max$ selection over the stencil; “grad-corr.” is the face correction that needs a halo exchange of neighbour values in the forward sweep and its transpose in the reverse. Every row’s error is its own analytic against its own run’s finite difference; the discretizations differ, so the analytic columns legitimately differ too. R6b is the load-bearing row. Every number re-extracted from the logs by the supervisor sweep. The last row keeps the convection limiter and limits the *default* gradient scheme instead: it modulates the magnitude by 3.4x and cures nothing.

byte-identical. Same scheme family, same order, the same gradient-correction term still evaluated and still requiring its halo exchange. **The only thing removed is the selection.**

The registration named the outcome that would have refuted the hypothesis, in advance: “if R6b is DIRTY ($\geq 4\%$) while R3a1 was clean, the limiter branch is NOT the carrier — the second-order gradient-correction TERM is, branch or no branch, and the supervisor’s unifying hypothesis is REFUTED for this defect and must be reported as refuted.” R6b came back at 0.849%. Read across the rows, three candidates die by a row that keeps everything else: the scheme’s *order* is not it (R6a is second order and clean), the halo-exchanged *gradient correction* is not it (R6b keeps it and is clean), and the default gradient scheme is not the gate (R3a2 keeps the limiter and stays dirty at 64x the clean floor). And the conditioning travels with the same word: **719 Krylov iterations to 41, a 17.5x collapse**, on an operator whose only change is that a $\min/\max$ left the tape.

The control that refuses to call it a complete cure. A registered follow-up ran the identical unlimited discretization at $np=1$, predicting the surviving 0.849% would prove to be that discretization’s own protocol floor. It is not: $np=1$ reads **0.016%**, the prediction is NOT HELD, and a real decomposition effect at $\sim 0.8\%$ survives the limiter removal. Reading the two rows together locates it, and the location matters: the *analytic* is 2.0572e-01 at both $np=1$ and $np=4$ *scotch* — identical at all five printed digits — while the FD column moves 2.0568e-01 $\rightarrow$ 2.0399e-01. With the limiter gone the adjoint itself is decomposition-invariant at printed precision, and the remainder is an FD-column shift, not an analytic error.

The erratum, which runs in our favour. The adversarial sweep of this campaign found exactly one factual error anywhere in it, and it was ours: the established *scotch* $np=4$ Krylov count had been written **590** in six places. A fleet-wide grep places “590 iterations” in exactly one log — `a4_simple2x2x1.log`, a *different* arm, the *simple* 2x2x1 partition at 1.40% error. Every log that ran the established arm reads **719**, fifteen of them. Corrected at every site; the correction enlarges the claim, because the conditioning collapse is $719 \rightarrow 41 = \mathbf{17.5x}$, not the 14.4x that was reported (VERIFICATION_defect_robustness_supervisor_sweep.md, 8BD47B35: six axes, five CONFIRMED, one CONFIRMED WITH A NAMED ERRATUM).

2.10 Operator or contraction: the two levers are not the same kind of fix

By this point two edits were known to clean this gradient: the farfield BC swap (`freestreamVelocity` $\rightarrow$ `inletOutlet`, the reach matrix’s N9 control) and the limiter removal. Both had been measured only with `check_totals`. This lab’s own L-36 is the standing warning that a passing gradient check certifies a contraction, not an operator — so R5 was registered to ask which kind of fix each lever is, with the loud alternative written out in advance: *“if either lever leaves the cross-residual $\geq 50 \times \|b\|$ while its gradient is clean, then that lever does NOT fix the operator — it only rotates the objective’s adjoint direction away from the corrupted subspace, exactly L-36’s failure mode, the operator is still wrong for every other objective and DV on that case, and the upstream report’s framing must change from ‘these configurations are safe’ to ‘these configurations hide it.’”*

The instrument is Section 2.4’s cross-residual, unmodified, with each lever’s own configuration supplying its own serial operator and its own floor.

configuration (all np=4 <code>scotch</code> , same mesh, same cut)	gradient err	$\ A^\top \psi + b\ /\ b\ $	its own np=1 floor	ratio to floor
established (<code>freestreamVelocity</code> + <code>linearUpwind</code> limited)	8.95%	3.2881e+02	1.1407e-04	3.0×10^6
R5a — scheme lever (<code>upwind</code> , BC kept)	0.041%	1.354841e-02	1.998129e-06	6.8×10^3
R5b — BC lever (<code>inletOutlet</code> , limiter kept)	0.019%	1.047165e+00	6.402838e-06	1.6×10^5

Table 6: The two levers at operator level. Every figure recomputed by the supervisor sweep from the raw dumped vectors with its own scripts: the staged corrected residuals equal its own `res + 2b` to the last bit, its independently rebuilt np4 $\rightarrow$ np1 permutation equals the staged input exactly (`np.array_equal` true, both levers), and the sign convention was verified rather than assumed — the correction is the only reading under which the np=1 controls collapse.

The scheme lever fixes the operator. $328.81 \rightarrow 0.013548$ is a collapse by a factor of **24,270**. The np=4 analytic also matches np=1 to 2.5×10^{-5} . Removing the limiter branch does not rotate the objective away from the damage; the damage is gone.

The BC lever hides the defect. At $1.047 \times \|b\|$ the `inletOutlet` configuration’s `scotch` adjoint fails to satisfy its own serial adjoint system by a residual the size of the right-hand side itself — **163,548 times its own np=1 floor** — underneath a `check_totals` reading 0.019%. For scale, the reach campaign convicted the Ahmed-35 case of carrying the wrong-operator defect at $5.45 \times \|b\|$; 1.05 is the same order of wrongness. This is L-36’s failure mode caught in the act: the gradient is clean because the contraction is small, not because the operator is right. **The configurations this lab called safe are not safe; they are quiet.** That is the most consequential sentence the campaign produced, and it is a correction to this document’s own earlier text rather than a claim about anyone else’s.

Three disclosures the sweep required, printed rather than resolved. (a) The registered band was $\leq 5 \times \|b\|$, and by that wording alone *both* levers HELD — 1.047 is a collapse from 329. The substance, that one lever hides rather than fixes, rests on the floor-ratio criterion, which was *not* a registered band. The sweep says so plainly, and says the band was mis-calibrated by roughly five orders of magnitude against the actual np=1 floors (10^{-6} to 10^{-4}). The extra-registration reading runs against the lab’s own prior framing, so nothing is smuggled — but future cross-residual bands go in units of the configuration’s own floor, not of $\|b\|$. (b) No state-override control was run for either lever, so the linearization-state confound is bounded only indirectly:

by the established arm’s own precedent (328.808 with the confound removed against 328.814 without — a fourth-digit effect) and by the sibling scheme lever putting the entire protocol floor at 1.35×10^{-2} , 77x below the 1.047 signal. The conclusion survives; the ~ 0.4 core-min control is unbought and is on the open list. (c) One corroboration fell out free: the BC-lever dump prints `framework_dCDdshape=2.406182395247966e-01`, the N9 record value to all sixteen digits, from a separately staged case directory and a different task.

2.11 What the campaign eliminated, and the confound it did not

Hanging nodes, refuted forwards. Section 2’s localization refuted the refinement-interface hypothesis backwards — the accurate decomposition is the one that cuts seventeen times more refinement interfaces. R4 refuted it forwards, with the controlled pair the reach record had to list as untested: A4’s own recipe, same STL, same background `blockMesh`, same `0.orig`, same `fvSchemes` with the limiter present, and the two snappy refinement levels dropped to zero.

mesh	hanging nodes	analytic	FD (own run)	rel. err
A4 coarse (record)	502 cells at <code>cellLevel</code> 1, 456 interface faces	2.2086e-01	2.4258e-01	8.95%
A4 conformal (R4)	none (2,336 cells, <code>cellLevel</code> uniformly 0)	1.8315e-01	1.8847e-01	2.82%

Prediction $\geq 2\%$: HELD. Same geometry, refinement removed, nothing else changed, and the defect is still there at 70x the clean floor. The magnitude drop from 8.95% carries no mechanism claim — removing refinement changes the mesh, hence the cut, hence the contraction, which the reach campaign already measured as case- and objective-specific.

Refinement: the conditioning collapse travels with the limiter. One uniform refinement of the background mesh (19,619 cells, 1,040 at `cellLevel` 1) could not be scored as registered, for a reason worth more than the arm: in the established configuration the `np=4 scotch adjoint` *does not converge at all*. GMRES exhausts `gmresMaxIters` 1000 at KSP residual 1.7188e-02 with `PetscConvergedReason: -3, solve_linear` raises, and no analytic exists to compare against anything (33.60 core-min, charged in full). R2 is scored NOT SCORED — unmeasurable as staged — and the failure is itself the datum. It rhymes with the restart arm: with the limiter present the operator is badly enough conditioned that a *restarted* GMRES stagnates on the coarse mesh (`gmresRestart` 60, stalled at 2.5079e-02 with five digits of no progress over the last hundred iterations) and an *unrestarted* one stagnates on the refined mesh, while the clean-scheme arms converge in 38–68 iterations on the same coarse mesh.

The substitute arm asked the refinement question the remaining budget could answer, and answered it: the same 19,619-cell mesh with the single R6b word removed *converges* — `PetscConvergedReason: 2` at 149 iterations, where the limited configuration exhausted 1000 — at 7.2627e-02 against its own FD 7.3195e-02, **0.777%**. That is the same scale as the coarse mesh’s unlimited 0.849%. One refinement level up, the limiter is the difference between no adjoint solution at all and a converged one, and the small surviving effect neither vanishes nor grows.

The transplant that did not fire, and what it cost. R1 mirrored the reach matrix’s N9 in reverse: where N9 removed the farfield BC from the defective case, R1 installed it on a clean one — A1’s 4,032-cell structured conformal NACA0012, a different mesh family in every respect. Registered prediction: the defect fires, $\geq 2\%$. It did not. `np=1` reads 0.043%, `np=4 scotch` reads 0.043%, and the two analytics are 1.55×10^{-7} apart. The obvious escape — “A1’s cut is too benign to excite anything” — was closed at zero solver cost with the reach campaign’s own cut classifier: A1’s `np=4 scotch` cut is **131 internal faces of which 120 are oblique**, no axis-aligned normal to 0.999, which is the maximally jagged mixed-orientation family the reach

matrix identified as catastrophic; A4’s own cut is 328 faces (143 x / 7 y / 159 z / 19 oblique). And the transplant did land: baseline C_D moved 0.52% from the record configuration, the same order as the 0.44% shift N9 measured going the other way. **freestreamVelocity is necessary but not sufficient, now by measurement rather than by the absence of a test.**

The survey that retro-explains the null, and the two correlates it cannot rank. A zero-cost read of all seven cases’ on-disk system/fvSchemes shows why R1 was always going to be clean: A1’s convection gradient is *unlimited*, so the arm was missing an ingredient. The limiter separates the seven cases exactly as the BC does — both defective cases carry `linearUpwind limited with cellLimited Gauss linear 1`; all five clean cases carry an unlimited `linearUpwind/linearUpwindV`. So does a third variable: `primalMinResTol` is 1e-4 on both defective cases and 1e-6 or 1e-8 on all five clean ones. *Three perfect correlates, one seven-case survey, no ranking available from it.* What breaks the symmetry is the within-case controls, and they exist for two of the three: N9 swaps the BC alone on A4 and cleans the gradient; R6b removes the limiter alone on A4 and cleans the *operator* (Table 6). The primal-tolerance lever has no within-case control at all, because DAFoam treats an unreached `primalMinResTol` as a primal *failure* rather than a depth setting — the registered R3b arm raises **AnalysisError: Primal solution failed!** before any adjoint runs. The knob is a gate, not a control. **That confound stays open and is recorded as open.** One candidate is eliminated outright: the turbulence model is not a correlate — `kOmegaSST` sits on both sides, defective on A4 and Ahmed-35, clean on CBFS.

The limit of what the campaign establishes. Every arm that moved the defect is a within-case control on A4: one mesh, one geometry. The cross-case survey corroborates and is itself a confounded correlate. The one experiment that would have broken the one-family confound — both ingredients installed on a clean case from another mesh family — was named in the record as the next arm and deliberately not improvised, because it was in no registered edit set and the budget did not cover its full protocol. It was registered and run two days later. It is the subject of the next section, and it did not do what either registered branch expected.

3 The same branch, in serial

The outstanding experiment was named in the record before it was approved: install *both* measured ingredients — the freestream farfield BC and the limited convection scheme — on the clean structured NACA0012, and measure whether the case *acquires* the defect. It was registered as such, with its edit set, its bands, and its escape clauses (Amendment 5, 0263D950, committed seven seconds before the first arm launched), and run on 2026-08-07. The registration named two outcomes: acquisition, which would give the defect a second mesh family, or a null, which would confirm the one-family confound and force the upstream report to scope its reproducer. It produced a third.

3.1 The arm

The registration’s clause (i) was a *control*: the np=1 arm should read $\leq 0.5\%$, as its unlimited sibling did (0.043%). It reads 92.8%. The registered escape clause therefore fired — “if the np=1 control itself is dirty ... the arm is reported unmeasurable, not scored” — and R7 is NOT SCORED against its FD-relative bands; clause (ii) is unevaluable as worded. Two facts then have to be pulled apart, and the amendment that does it (AA7FCDDC) was committed with the two numbers in hand and before either separating arm ran, which is stated here rather than glossed.

The acquisition question, answered: no. On the decomposition-invariance instrument the two rows are the same run twice. The analytics are 3.9×10^{-4} apart in vector-relative terms

A1 + <code>freestreamVelocity</code> + limiter	C_D /shape analytic	FD (own run)	rel. err	C_L /shape	adjoint KSP
np=1 (<code>a1lim_np1.log</code>)	1.066449e-01	6.194868e-02	92.8%	7.86%	95/96, reason 2
np=4 <code>scotch</code> (<code>a1lim_ np4scotch.log</code>)	1.066142e-01	6.194874e-02	92.8%	7.87%	96/97, reason 2

Table 7: The acquisition arm. Both arms `rc=0`; every one of the seventeen primal solves per log (baseline plus eight design variables at $\pm h$) reached the case’s own `primalMinResTol` $1e-8$, worst final residual 9.99×10^{-9} , none exiting on the iteration cap. Staging asserted in-driver; the np=4 log reads `Decomposition method scotch` [4].

(per-component max 0.197%, median 0.040%); the FD columns 4.3×10^{-6} apart, 1.9×10^{-7} in the worst single component; the C_L analytics 3.8×10^{-5} ; the Krylov counts 95/96 against 96/97 — *no conditioning excitation at all*, where A4’s defective configuration reads 719 against its clean arms’ 41. Whatever is wrong here is wrong identically at both decompositions. **The defect did not acquire on the second mesh family.** The trigger conjunction — limiter $\times$ `freestreamVelocity` $\times$ jagged `scotch` cut — is measured *insufficient* off the Ahmed background family, and the one-family confound on the defect side stands. The remaining separating candidates sharpen rather than disappear: the Ahmed mesh family or its cut topology, or the loose `primalMinResTol`. A1 runs $1e-8$, and with both ingredients installed at $1e-8$ it is decomposition-clean — so the loose-primal co-ingredient hypothesis *gains* weight here, where a fired defect would have killed it.

One caveat is owed and the supervisor sweep required it be stated as a word rather than a hedge: **the invariance instrument on which this verdict is scored was adopted after the numbers were in hand.** It echoes a subclause the R1 registration already used, and no registered band could read a 3.9×10^{-4} shift as acquisition, so the inference is forced — but the instrument choice is post-hoc relative to the R7 registration, and this document says so.

3.2 Convicting the analytic, not the finite difference

arm	edit vs <code>a1lim_np1</code>	analytic	FD (own run)	rel. err
R7f1 <code>a1lim_np1_h3e3</code>	<code>check_totals</code> step $1e-3 \rightarrow 3e-3$	1.066449e-01	6.216543e-02	93.7%
R7f2 <code>a1limdef_np1</code>	<code>linearUpwind</code> limited $\rightarrow$... default (one word)	6.516374e-02	6.522800e-02	0.121%

Table 8: The separating arms, both registered before either ran. R7f1’s analytic is byte-identical to the $1e-3$ arm’s at every printed digit — only the FD column moves. The sweep’s byte audit: R7f1’s only changed input is one line of `runScript.py`; R7f2’s is one word of `system/fvSchemes`.

The step check was registered to convict the finite difference: the prediction was that the FD would *not* be step-stable, the limiter having kinked the shape response at the FD scale, and that the 93% would be an unresolved-FD artifact rather than a conviction of the serial AD. It came back at 2.10% vector-relative (per-component max 15.14%), scored NOT HELD in the middle band by a tenth of a point — and read in substance rather than spun, a 2.1% step wobble is **44x too small** to account for a 92.8% gap, the FD magnitude is $6.2e-02$ at both steps, and the analytic is byte-identical across them. The FD reference is essentially resolved; the discrepancy lives in the analytic.

R7f2 then puts one word between 0.121% and 92.8%, *at np=1*: the same `limited  $\rightarrow$  default` lever that cleaned the operator on A4, now convicting the limiter branch in serial on the second mesh family (C_L 0.018%, KSP 32). Everything else the staging touched is exonerated by the same arm, because R7f2 carries it all: the `linearUpwindV  $\rightarrow$  linearUpwind` family swap, the added `gradSchemes` entry, and the R1 boundary-condition edit set (0.043% on record).

3.3 The attack that could have killed it

There is a classic finite-difference trap for a branchy scheme, and it had to be closed. If the limiter’s selection flips between the $+h$ and $-h$ legs, a central difference measures a kink average rather than a slope — and a *symmetric* kink at the base point is step-invariant under central differencing, so the two-step check above cannot exclude it by itself. The record registered the kink hypothesis and then tested it only by step stability. The adversarial sweep named that gap and closed it at zero solver cost, from data the record’s own logs already carried: each FD leg prints its converged C_D , so the *one-sided* slopes were free all along.

component	central FD	forward	backward	fwd/bwd asym.	analytic
0	−0.009054	−0.008926	−0.009182	2.8%	−0.034900
1	−0.020342	−0.019338	−0.021345	9.9%	−0.037237
4	+0.037311	+0.037465	+0.037158	0.8%	+0.061443
5	+0.040489	+0.040656	+0.040322	0.8%	+0.064415
6	+0.008497	+0.009660	+0.007334	27.4%	−0.025912

Table 9: One-sided slopes at $h = 10^{-3}$, reconstructed from the FD legs’ own printed converged C_D values. The same reconstruction rebuilds the log’s printed central-difference vector to 4.7×10^{-9} absolute at both step sizes, which validates the leg ordering, the step, and the framework’s arithmetic before anything is inferred from it.

Three readings, and the finding survives all three.

- **On the components carrying most of the gap there is no kink to blame.** For 0, 1, 4 and 5 the two one-sided slopes agree to within 10%: the function is locally smooth at the FD scale, and the analytic still misses by 65–285%.
- **On the one component that *is* branchy, the kink cannot rescue the adjoint.** Component 6 — the sign-flipped one — shows 27.4% asymmetry at $h = 10^{-3}$, growing to 91% at 3×10^{-3} : real limiter chatter. But *both* one-sided slopes are positive at *both* steps, all four secants falling between +0.0053 and +0.0143, while the analytic reads −0.025912. The analytic lies outside the [backward, forward] bracket, on the wrong side of zero, at three times the magnitude. No branch selection available to the finite difference produces a negative slope there.
- **Triangulation, also free.** The limited case’s FD vector sits at cosine **0.994** to the *unlimited* case’s analytic — which R7f2 validated against its own FD at 0.121% — while the limited analytic sits at cosine 0.90 to its own FD. Three of the four instruments agree with each other; the limited serial adjoint is alone.

The sweep’s recommendation is adopted here as protocol, because it is strictly stronger than a second step size and costs nothing: for this symptom class, reconstruct the one-sided slopes from the FD legs’ printed objective values. It belongs in the registration, not in the post-mortem.

3.4 What the serial variant is, and what it changes

- **It is decomposition-independent, and it broke a family property before anyone had written that property down.** Every defect this lab had traced in this toolchain vanished at np=1. This one does not.
- **It is the same object as D1.** cellLimited’s min/max stencil selection, recorded on the reverse tape, wrong through a different excitation: a parallel inconsistency on the Ahmed family, a serial contraction error on the airfoil. On A4 the identical token pair reads only 0.34% at np=1 — the serial excitation is *case-dependent*, which is exactly what a

wrongly-differentiated *data-dependent* branch would produce, since different meshes activate different branches at the converged state.

- **It is far cheaper to file.** A maintainer needs one process, one 4,032-cell structured case, and one word of `fvSchemes`. No MPI in the loop, no partitioner, no mapping instrument, no cross-residual protocol. The instrumented-rebuild search space is unchanged — the `cellLimited/limitedGrad` reverse sweep — and now has a single-process reproducer to instrument.
- **It costs the campaign’s framing a clause.** “The branch defect is a parallel inconsistency” is now measured to have an exception: the branch can be wrong serially, with the parallel machinery innocent.
- **It cost this lab an instrument’s warrant.** The decomposition-invariance gate adopted after A4 passes here to 3.9×10^{-4} on a case that is 92.8% wrong. *Invariance instruments detect inconsistency between two evaluations of the same tape; a tape that is wrong identically in both sails through every one of them.* Never promote an invariance pass to a correctness verdict unless at least one side is anchored to an external reference — an own-run finite difference with a step check, or an independently evaluated operator.

4 Why it was never caught

Two independent bodies of evidence answer this: the published verification protocols of the toolchain’s own method papers, and the community record.

4.1 The papers’ own protocols

Three method papers were read in full and their verification protocols tabulated (`DAFOAM_PAPERS_VERIFICATION_PROTOCOLS.md`, `BF6AC53B`; reading notes with per-claim source tables under `demo-output/website/campaign/`).

- He, Mader, Martins, Maki, “DAFoam: An Open-Source Adjoint Framework for Multidisciplinary Design Optimization with OpenFOAM,” *AIAA Journal*, doi:10.2514/1.J058853.
- He, Mader, Martins, Maki, “An aerodynamic design optimization framework using a discrete adjoint approach with OpenFOAM,” *Computers & Fluids* 168 (2018) 285–303, doi:10.1016/j.compfluid.2018.04.012.
- Kenway, Mader, He, Martins, “Effective Adjoint Approaches for Computational Fluid Dynamics,” *Progress in Aerospace Sciences*, doi:10.1016/j.paerosci.2019.05.002.

Three facts fall straight out of Table 10.

1. **No paper ever varies the decomposition of anything.** Decomposition is not an experimental axis anywhere in the corpus — not at fixed process count, not across process counts. The two papers that verify DAFoam’s v1 operator in (implied) parallel never state what the decomposition was.
2. **The operator our defect lives in has exactly one published accuracy measurement, and it is serial.** The survey states the reason in its §5.1 — running on one CPU core “allows us to isolate the impact of parallel communication on the performance” — and confirms the gap in its conclusions: “*We do not have scalability data for the Jacobian-free (operator overloading) and the full-code AD (operator overloading) options because we run the adjoint computation only in serial in the ADODG Case 3 (Sec. 5.1).*”

paper	adjoint operator verified	case / mesh class	cells	decomposition	result
C&F 2018	v1 explicit FD-coloring Jacobian	Ahmed body 25°, snappy- HexMesh	~1M	unstated	avg <0.1%, worst 0.252%
AIAA J 2020	v1 explicit FD-coloring Jacobian (Jacobian-free explicitly deferred to future work)	ADODG Case 3 wing, structured hex	102,912	unstated ; “decompose”/ “processor”/“scotch” absent from the paper	avg <0.1%, worst flow row 0.195%
PAS 2019	Jacobian-free reverse AD (operator overloading) — the v2+/v5 architecture family	ADODG Case 3 wing, structured hex	102,912	serial. np=1, stated twice, by design	10-digit agreement
PAS 2019	Jacobian-free source- transformation AD — <i>ADflow, not DAfoam</i>	CRM wing / wing-body-tail	3.6M / 10.4M	block-based, fixed (48 / 96 cores)	11 / 6 digits

Table 10: Every gradient-accuracy verification in the method-paper corpus.

3. **No verification anywhere combines a refinement-interface mesh with a stated graph partitioning.** The one snappy-mesh verification — C&F 2018’s Ahmed body, our defect’s own geometry family — is of the v1 explicitly FD-assembled Jacobian, an architecture in which a tape-recording defect cannot exist, at an unstated decomposition.

The corpus checks the derivative class this defect spares. The serial variant of Section 3 looks at first like it should embarrass point 2 rather than support it: if the limiter’s tape can be 92.8% wrong in a single process, why did the corpus’s one *serial* verification of the shipped operator family return ten-digit agreement? Two answers, both readable straight off Table 10, and together they are the sharpest statement of the gap this section can make.

First, the corpus’s parallel evidence is entirely about the other architecture. The only two verifications run in (implied) parallel — and the only one run on a refinement-interface mesh, C&F 2018’s Ahmed body, our defect’s own geometry family — measure the v1 explicitly FD-assembled Jacobian. A finite-difference column re-evaluates the residual, branches and all: branch *selection* is differenced there, not recorded. A mis-taped branch cannot exist in that architecture. Every parallel accuracy number in the corpus is a measurement of the derivative class this defect spares, taken in the configuration where it could not appear.

Second, the one measurement that does exercise the recorded tape is a single case at a single converged state, and our own arms show why that cannot generalise for a data-dependent branch. The same `cellLimited/linearUpwind` token pair reads 92.8% at np=1 on the structured NACA0012 and 0.34% at np=1 on the Ahmed body. A branch that is not activated asymmetrically at the state you verify at is a branch you verify past. Ten digits on one structured wing certify that wing’s tape at that state — which is precisely what the survey claims and no more.

A published claim, corrected against ourselves. An earlier draft of our own upstream report asserted that “the toolchain’s journal paper reports average adjoint derivative error under 0.1% at up to 1536 cores” and that our measurement therefore contradicts a peer-reviewed number. Reading the paper in full showed that sentence to be *wrong twice*. The abstract joins two disjoint experiments with an “and”: the 1536 cores is a runtime-only table on a 10.1M-cell structured mesh; the <0.1% is a separate ~103k-cell accuracy study at an unstated processor count. **No accuracy number at any stated core count exists in that paper.** And both <0.1% figures attach to the v1 explicit-FD-Jacobian architecture — the same paper pairs them with “Implementing the Jacobian-free approach in DAfoam ... will be conducted in the future work.” The 0.1% scale is

the FD-partial error floor, not a parallel-consistency scale.

This *strengthens* the case rather than weakening it: the report does not contradict a published measurement, it fills a hole the survey’s own conclusions declare.

No archival method paper covers the shipped architecture. The DA Foam publications page lists no framework paper after 2020; the `mdolab/dafoam` README’s citation section names exactly the two He et al. papers, tied to no version; a JOSS search finds nothing; Zenodo records 15636000 (v4.0.2) and 20045081 (v5.0.0) are software deposits, not papers. The v2.2 release note that shipped the Jacobian-free adjoint cites no paper for it either. *The v4/v5 rewrite we measured has no archival method paper at all*; the citation trail for the shipped matrix-free adjoint ends at papers describing other architectures.

Was the defect class disclosed anywhere? Two quotes were fetched for exactly this question, and neither is a tape caveat. C&F 2018 §2.9 names interprocessor boundary-patch state updates as “*essential for accurately computing the adjoint derivative*” — for the v1 FD path, where they are handled by executing `U.correctBoundaryConditions()` per perturbation. The v5 tape must *record* that same coupling, and its interface-cell rows are exactly where our cross-residual concentrates. PAS 2019 §A.4 acknowledges that “*some code structures remain a challenge to differentiate, including pointers and MPI calls*”, and that ADflow’s parallel derivative correctness rests on manual assembly using properties “that the AD code cannot, in general, safely assume” — a statement about AD-across-MPI as a hazard class, made about ADflow’s development choices.

No paper states a halo-exchange tape caveat or a partition-consistency assumption for DA Foam’s operator-overloading path. There is no sentence a maintainer could point to as prior disclosure. What the quotes support is gentler and more useful: **the defect sits in a subsystem whose verification the papers explicitly scoped out, in a region their own earlier work flagged as accuracy-critical.**

4.2 The community record

A read-only novelty sweep ran **63 recorded searches across 10 venues** on 2026-08-04, with roughly 25 further pages read in full: the `mdolab/dafoam` issue tracker and Discussions, `mdolab/id-warp` issues and discussions, `mdolab/pyofm`, `mdolab/MACH-Aero`, `mdolab/adflow`, `OpenMDAO/OpenMDAO`, GitHub global search, web and scholar search, and the PETSc lists (`LIAISON_NOVELTY_SWEEP_decomposition_defect.md`, `3c74dc03`). Nothing was posted; every URL cited in that memo was fetched in-session, and quotes are marked verbatim or via-summary.

Verdict: no prior report. No one has ever reported a DA Foam gradient that is converged and wrong under one decomposition and correct under another; any **scotch-versus-simple** dependence; any transpose-operator inconsistency; or anything of this class on a plain wall-bounded case without coupled patches. The word “**scotch**” **appears in zero issues and zero discussions in the project’s history**, and `getRotationMatrix3d` appears in zero issues or pull requests anywhere on GitHub.

What the record *does* contain makes the case sharper, not weaker:

Three consequences for how the finding should be presented upstream.

1. **The “known issue with coupled patches” reply is pre-answered.** Every upstream acknowledgement of parallel-wrong derivatives attributes the wrongness to a coupled patch (periodic, AMI), prescribes avoidance, and names no mechanism. Our Ahmed-body case has *no* coupled patches: it shows the same class of wrongness arising from ordinary processor boundaries — precisely the configuration upstream’s own lore calls safe.

thread	what it establishes
#946 (2026-02)	The strongest near-miss found anywhere. A user’s <code>check_totals</code> shows 93.92% analytic-vs-FD error on the PeriodicHill case; the maintainer replies, verbatim: “ <i>The PH case’s derivatives are not accurate when running in parallel. You have to run it in serial.</i> ” No mechanism offered, scoped to the periodic-boundary case
#379 (2022-12)	The same acknowledgement four years earlier — the periodic hill case “is known to have issues when running in parallel (because it has periodic boundaries; we haven’t gotten a chance to debug it yet)”, while other cases “can run in parallel without an issue”
#972 (2026-04)	Nearest <i>mechanistic</i> relative: cyclicAMI adjoint false-converges then produces 1e11–1e15 sensitivities; remedy is to force all AMI faces onto one rank. A coupled interface whose parallel adjoint treatment is wrong — but AMI patches, v4 assembly path, and garbage output rather than a cleanly converged 9%-wrong gradient
#885 (2025-09), #952 (2026-03)	Upstream’s clearest acknowledgement that decomposition affects the adjoint — but strictly its <i>convergence rate</i> . KAHIP partitioning improves the preconditioner; kahip support was added upstream 2025-12. Nobody in either thread checks the converged gradient <i>value</i> against decomposition
#101/#102 (2021-03)	The one time serial-vs-parallel gradient equality was actually checked upstream, it passed: the reported discrepancy was cell ordering in a written field, not the gradient. v1-era topology case
#914 (2025-11)	Documents upstream’s acceptance threshold: a 1.27% analytic-vs-FD deviation draws “the gradient accuracy is very acceptable for practical aerodynamic optimization”

Table 11: The community record on parallel adjoint correctness in DAfoam.

2. **The “decomposition only affects convergence” reply is pre-answered.** #885/#952 concern convergence rate; our KSP *converges* (reason 2, true residual 1.7×10^{-7}) to a wrong solution, which is exactly what the cross-residual measurement discriminates.
3. **The triggering configuration is the default.** The docs make no parallel-consistency claim and give no decomposition guidance at all, while the shipped default is `scotch` (pyDAfoam.py lines 590–591, main at E77F0C0C, fetched raw 2026-08-04: `self.decomposeParDict = {"method": "scotch", ...}`).

The remaining gaps are not novelty gaps. They are, unchanged: a reproducer on a stock upstream tutorial rather than a lab-built case, and mechanism to a line.

One novelty gap that is new, and is stated rather than assumed away. The sweep above ran on 2026-08-04 and was aimed at the decomposition *phenomenon*: whether anyone had reported a DAfoam gradient that is converged and wrong under one decomposition and correct under another. The limiter result post-dates it by a day and the serial variant by three, and **neither has been through a novelty sweep of its own**. Nothing here claims priority for “the slope limiter’s reverse tape is wrong”; what is claimed about it is measured, not surveyed. Closing that gap is cheap and belongs before any filing — the more so because the serial variant is the entry point a maintainer would actually run.

5 The IDWarp rotation guard

This defect is independent of Section 2 in code and in provenance — it is geometric, serial, and lives in a different repository — but it was found in the same arc, it contaminated the same

gradients, and it had to be removed before the decomposition measurements could be trusted. It is not independent *in kind*: a guard is a branch, and Section 2.9 arrives at the same object from the other end. It is also the cleanest member of the class: root cause named to a line, confirmed by repair.

5.1 The line

IDWarp **2.6.2** is the exact version in `dafoam/opt-packages:latest` (image digest `sha256:9d45679d55fd47f5ca7afd99cabb86c7c2729cf2acf34c438eb33af5290f07fc`); the installed `idwarp/*.py` files are md5-identical to the v2.6.2 tag of <https://github.com/mdolab/idwarp>, tag commit 647FD8FC.

The warp displaces each volume node by an inverse-distance weighted sum in which the weights W_i are *constant* with respect to the surface coordinates; all the surface dependence lives in a rotation matrix M_i and a displacement b_i , with $S_i = M_i \cdot (r - X_{u0,i}) + (X_{u,i} - r)$. **The rotation acts on the lever arm from the surface node to the volume node.** M_i is built at `src/modules/kd_tree.F90:1639-1640` as $R(n_0 \rightarrow n)$, by:

```
real(kind=realType), parameter :: tol = 1.4901161193847656e-08    ! line 44 (= sqrt(eps))
call cross_product_3d(v1, v2, axis)
call getMag(axis, axisMag)
! When axisMag is less than sqrt(eps), the acos 'arg' value will be
! exactly one which will give a nan in complex mode.
if (axisMag < tol) then                                           ! line 58   <<< HERE
    angle = zero
    axis = zero
    axis(1) = one
else
    axis = axis / axisMag
    vv1 = v1 / magv1 ; vv2 = v2 / magv2
    arg = min(one, vv1(1)*vv2(1) + vv1(2)*vv2(2) + vv1(3)*vv2(3))
    angle = acos(arg)
end if
Mi = Mi + sin(angle) * A + (one - cos(angle)) * C                ! Rodrigues
```

The map $M_i(n_0, n)$ is perfectly smooth at $n = n_0$, with the finite, non-zero derivative $dM_i = [n_0 \times dn]_{\times}$. But *this parameterisation* — normalise the cross product, take `acos` of the dot product, apply Rodrigues — has a removable coordinate singularity there: `axis` is 0/0 and $d(\arccos)/d(\arg) \rightarrow \infty$. The guard exists to stop that becoming a NaN, and the source comment says so. It replaces the singularity with `angle = 0`.

What Tapenade does with the guard. In the generated reverse file `src/adjoint/outputReverse/vectorUtils_b.f90:123-128`, when the guard branch was taken the reverse sweep *zeroes every adjoint on the path back to v2*; the subsequent `GETMAG_B` and `CROSS_PRODUCT_3D_B` calls therefore propagate exactly zero, and $dM_i/d(\text{normals}) = 0$ exactly. Tapenade is not at fault: it differentiated the branch that was taken. The defect is that the branch taken is a *constant* where the true function has a non-zero first-order term.

The guard fires with certainty at every gradient evaluation. `getMag` regularises with $+10^{-30}$ under the square root. At an undeformed baseline `normals == normals0` bit-for-bit (same code, same inputs), so `axis =  $n_0 \times n = 0$` and `axisMag =  $\sqrt{10^{-30}} = 10^{-15} < 1.4901 \times 10^{-8}$` . **Branch 0, guaranteed, at every non-corner surface node.** A `check_totals`, a first design iteration, and every gradient probe are evaluated at exactly that state. The finite difference, meanwhile, perturbs the surface by h and tilts adjacent faces by $\theta \sim h/L$, far above the threshold — so the FD sees a first-order rotation term the AD has set identically to zero. *That difference is the gap.*

5.2 Narrowing the claim, against ourselves

The obvious statement — “the rotation term is mis-differentiated” — is too broad, and measurement refuted it. Across five of IDWarp’s own regression meshes, run with the project’s own displacements:

mesh	deformation	worst err, rotations ON	worst err, rotations OFF	$\ dX_s\ $ on vs off
symm_block (inflate_cube)	axis scaling	212.6%	1.06e-05%	identical
o_mesh	shear sweep	1.7e-05%	5.8e-03%	6.42e+03 / 9.46e+02
co_mesh	shear sweep	3.9e-05%	5.1e-05%	7.27e+04 / 2.16e+03
sym_mesh	shear sweep	6.9e-06%	9.1e-04%	8.85e+04 / 2.45e+03
onera_m6	shear sweep	1.26%	9.7e-07%	1.69e+04 / 3.06e+03

On the three shear cases the deformation genuinely rotates normals, the guard does *not* fire, the AD *does* change when rotations are switched off — and with rotations **on** those cases are clean at $10^{-5}\%$ to $10^{-6}\%$. **When the rotation branch is live, warpDeriv is correct.** The catastrophic failure is confined to $n \approx n_0$. This is a sharper and more falsifiable statement than the one we started with, and it is the correct one.

The capstone that could have refuted it. If the failure is the degenerate branch specifically, then moving a case off its undeformed baseline should make the guard stop firing, at which point warpDeriv must become sensitive to useRotations and get dramatically more accurate. With all 27 shape design variables pre-set to 0.02 on the U-bend case, the two sign flips vanish and the worst error falls from 207.0% to 8.2% with nothing else changed — and the fingerprint confirms the branch is now live: $\|dX_s\|$ is $1.495168856286067 \times 10^3$ at the undeformed baseline whether rotations are on or off (*identical* — branch dead), and $1.566420804338303 \times 10^3$ vs $1.495168856286067 \times 10^3$ pre-deformed (*differs* — branch live). Had the mechanism been “rotations are mis-differentiated”, this experiment would have failed flatly.

5.3 Why upstream’s CI cannot see it

Four independent mechanisms, each verified against primary source.

1. **The assertion is missing.** tests/test_USMesh.py:86 calls mesh.verifyWarpDeriv(...) and *discards the result*. Nothing is asserted on it — which is exactly what upstream issue #57 says.
2. **The only thing asserted is blind by construction.** The regression check is handler.root_add_val("... Sum of dxs:", val, tol=1e-8). Across five meshes the rotation term contributes *exactly zero* to that sum (identical to 0–1 ulp) while changing $\|dX_s\|$ by factors of 5 to 34. Five for five, upstream’s check cannot detect a change that alters the derivative vector by a factor of 34.
3. **A dot-product test cannot detect it either, and is not run.** examples/structured/-dotprod.py lives in examples/, not tests/. And a dot-product test only proves forward-AD and reverse-AD are mutual transposes: if both linearise a *different* function than the warp, they do so identically and the dot product is still zero.
4. **The verification tool defaults to a random seed.** UnstructuredMesh.py:460: def verifyWarpDeriv(self, dXv=None, ..., randomSeed=314). A random direction contracts the error against a direction it barely overlaps (0.32–1.30% where the real objective seed reads 207%, flipped).

Not a documented approximation. No documentation, release note, docstring, or source comment qualifies `warpDeriv` as approximate. The docs say only “Compute the warping derivative $(dX_v/dX_s^T)^T \text{Vec}$ ”; the `warpDeriv.F90` header says only “Compute the following matrix vector product: $dX_v/dX_s^T \cdot X_{vbar}$ ”. Both present it as the exact transpose Jacobian. *One disclosed gap:* the primary IDWarp reference (Secco, Kenway, He, Mader, Martins, *AIAA J.* 59(4):1151–1168, 2021, doi:10.2514/1.J059491) could not be retrieved — five umich URLs return 403 including the authors’ own bibliography link, and OpenAlex (`is_oa: false`, `oa_status: "closed"`) and Semantic Scholar independently report no open copy. No claim is made about what that paper says, and the “undocumented” characterisation rests on code, docs and release notes only.

5.4 The upstream issue this answers

`mdolab/idwarp#57`, “`inflate_cube` test appears to fail”, opened 2021-07-14, labelled **bug**, **open**, **zero comments**, **five years untouched**. Its timeline contains exactly one event: the label, applied 8 seconds after creation. Its author wrote: “*the output from this test appears to show large errors in some of the derivative terms. However, these error values are not actually used as a unit test and so the entire test suite still passes*”, and “*Given how weird the FD values are for the two terms with big errors, it may be that the issue is in those rather than the AD values, but it’s hard to see how that could be the case.*”

We reproduced the issue exactly — IDWarp alone, no DA Foam, no OpenFOAM, no CFD, using IDWarp’s own test mesh, options, displacement and verification routine — and answered its open question adversarially. A step-size study on the two failing degrees of freedom gives FD values stable to six significant figures across three decades ($h = 10^{-4}$ to 10^{-6}), with subtractive roundoff appearing only at 10^{-8} . **The finite difference is converged; the AD is the wrong one.**

Why degrees of freedom 0 and 3 and not 1, 2, 4, 5: `inflate_cube` scales an axis-aligned block by (1.1, 1.2, 1.3), which leaves every face normal *exactly* unchanged, so the guard fires even at the deformed state. Perturbing a node *along* its face normal tilts adjacent faces at first order and switches the rotation term on; perturbing it *in* the face plane does not. That is a clean instance of our own predicted-clean control, appearing unbidden inside upstream’s five-year-old data.

5.5 The patch, and what it proves

The derivation was written *before* the code, and every sign was checked by a controlled experiment with a hand-computed known answer before any system-level run. With $u_1 = v_1/|v_1|$, $u_2 = v_2/|v_2|$: $\sin \theta [k]_{\times} = [u_1 \times u_2]_{\times}$ exactly, and $(1 - \cos \theta)[k]_{\times}^2 = O(\theta^2)$, so $M_i = I + [u_1 \times u_2]_{\times} + O(\theta^2)$ and, at $u_2 = u_1$,

$$dM_i = [w]_{\times}, \quad w = \frac{v_1 \times dv_2}{|v_1| |v_2|} \quad (\text{forward})$$

$$v_{2b} += \frac{\text{axial}(m_{ib} - m_{ib}^T) \times v_1}{|v_1| |v_2|} \quad (\text{reverse, by the cyclic triple product})$$

$|v_1|$ and $|v_2|$ are already recomputed in the reverse routine’s forward sweep and are in scope inside the branch. **The whole fix is four lines of assignment in the branch .EQ. 0 block**, replacing nothing: the three zeroed intermediates stay zeroed, because the dummy-axis path they belong to really does carry zero; the patch adds the term that path lost. The diff is 2 files, +44 lines, all in `src/adjoint/output{Reverse,Forward}/vectorUtils_{b,d}.f90`.

On the A2 wing the patch improves three of four `check_totals` rows by 33.9x, 53.2x and 115.1x (CD/shape $1.713791 \times 10^{-2} \rightarrow 5.059114 \times 10^{-4}$; CL/shape $1.165163 \times 10^{-2} \rightarrow 2.189473 \times 10^{-4}$; CL/twist $1.120617 \times 10^{-2} \rightarrow 9.737824 \times 10^{-5}$) — and *degrades* the fourth, CD/twist, from 0.389% to 0.505%, both deep inside the pass band. Any one-line summary quoting “34–115x” must carry that footnote.

acceptance test (stated before the runs)	before	after	verdict
Upstream <code>inflate_cube</code> (#57), rotations ON, DOFs 0 / 3	210.16% / 212.62%	8.56e-06% / 3.35e-05%	PASS
A1 NACA0012, real <code>dCD/dXv</code> seed, <code>idx6</code> (leading-edge combo)	634.0%, sign flipped	5.54e-04%	PASS
A5 U-bend, pressure-loss real seed, <code>idx8</code> / <code>idx17</code>	207.0% / 121.6%, both flipped	3.0e-06 / 7.0e-06 (relative)	PASS
Regression control: <code>o_mesh/co_mesh/sym_mesh/onera_m6</code> , guard dead	—	bit-identical <code>verifyWarpDeriv</code> output, every printed digit	PASS
Primal invariance: 310,284-coordinate warped grid	—	md5-identical (8fafe12f848a f490a5041c865112b5fb), <code>max diff </code> = 0.0	PASS

Table 12: Acceptance results. Note the *direction*: on #57 the AD moved to the already-converged FD (2.87055669475 $\rightarrow$ -115.875779229 against an unchanged FD of -115.875789153) — the direction a genuine fix must move and a comparison-zeroing artifact could not produce. The in-plane DOFs 1, 2, 4, 5 are untouched to every printed digit.

Two limits, stated plainly. First, **retuning the tolerance cannot fix this.** `tol` is exactly $\sqrt{\varepsilon}$, but the `acos(min(1, v1.v2))` construction is numerically flat on its own out to $\sqrt{2\varepsilon} = 2.1073 \times 10^{-8}$ rad: $1 - v_1 \cdot v_2 = \theta^2/2$ drops below ε , `arg` rounds to bit-exact 1.0, and `acos(1.0)` is exactly 0, guard or no guard. Measured at $\theta = 10^{-8}$: `arg` bit-exactly 1.0, $1 - \text{arg}$ bit-exactly 0.0. *The first thing a maintainer will reach for is foreclosed.* Second, **the AD’s zero is the exact derivative of the implemented function at that point.** The guard makes the warp genuinely flat in a ball of angular radius $\sim 1.5 \times 10^{-8}$ rad, and a finite difference taken entirely inside that ball returns exactly 0.0, agreeing with the AD to every digit. Tapenade is correct about the code as written. The defect is precisely that `warpDeriv` returns *the derivative of a flat spot that no optimizer can traverse and no finite difference can resolve* — reaching it needs $h < 1.5 \times 10^{-8}L$, and the step study already shows roundoff destroying the FD at $h = 10^{-8}$. Any filing should pre-empt the response “the code is differentiable and your step is too coarse.”

5.6 The second regime, and the fix that covers both

Away from the baseline, with `axisMag` just above `tol`, the guard does not fire and the derivative *is* computed — but through an `acos` whose argument has lost its information to cancellation. This regime is real, measured on ONERA M6 at $\sim 1.26\%$, and **survives the patch, exactly as predicted** before the acceptance runs. Its cause was *tested* rather than asserted, by repairing each candidate line in isolation:

θ	V0 shipped	V1 denominator only	V1b angle only	V2 both	V3 singularity-free
1e-06	6.658e-05	5.561e-05	3.661e-05	1.974e-10	1.538e-12
1e-07	3.998e-04	3.339e-04	2.199e-04	7.221e-10	1.710e-12
5e-08	1.161e-02	9.663e-03	6.436e-03	3.083e-09	2.229e-12
2e-08	5.286e-02	4.483e-02	2.801e-02	4.844e-09	1.068e-12
$\leq 1.49\text{e-}08$ (guard)	1.0	1.0	1.0	1.0	9.3e-13

Table 13: Relative error of the reverse routine against the true derivative of the smooth map. V1 is the single-line repair an earlier draft of our own record named as the carrier: it buys $1.0\times$ to $1.6\times$, i.e. it is indistinguishable from doing nothing. The pair buys 2.1×10^3 to 3.7×10^7 .

The dominant term is `angle = acos(arg)` in the *primal*: `arg` = $1 - \theta^2/2$ loses the information

at $\theta \sim \sqrt{\varepsilon}$ before `acos` is ever called, and the reverse inherits it. Neither line is sufficient alone and the two do not add. Only `angle = atan2(axisMag/(magv1*magv2), arg)` *together with* $\sqrt{1 - \arg^2} \rightarrow \text{axisMag}/(\text{magv1} \cdot \text{magv2})$ removes the regime — and the substitute is free, because `axisMag` is already computed two lines earlier in the primal.

The recommended upstream fix is neither of our patches but the reparameterisation $R = I + [v]_{\times} + [v]_{\times}^2/(1+c)$ with $v = a \times b$, $c = a \cdot b$: a polynomial in v with the smooth coefficient $1/(1+c)$, giving I with the correct derivative at $a = b$, needing no threshold, and unable to NaN except at exact reversal — a genuine singularity of the problem rather than of the parameterisation. It is measured here at 6.3×10^{-13} to 2.2×10^{-12} across every angle tested *including inside the guard ball*, where every `acos`-based variant reads exactly 1.0. It is the only single change that covers both regimes.

Two caveats attach. A regime-2 repair **cannot** be primal-bit-identical — it changes `angle` in the forward warp (M_i moves by 1 ulp at $\theta = 1$) — so any such patch must be re-graded on the primal as well as on the gradient. And our shipped patch is a *proof of concept on the generated file*: upstream should fix the primal’s parameterisation and regenerate, not merge a hand-edit of Tapenade output.

5.7 Workaround, with its price named

`useRotations: False` in `meshOptions` restores gradient consistency to roundoff on all evidence here — it makes the warp exactly linear in the surface coordinates. It is not free: upstream documents rotation interpolation as what “will help preserve boundary layer orthogonality” under large orientation changes, so mesh quality under large deformation is the price. **That trade has not been measured by this lab**, and no optimisation has been re-run under it. Recommending it as a default would be premature.

6 The conditioning wall, and the sub-LU remedy

6.1 The wall

On a 21,000-cell curved-backward-facing-step (CBFS) field-inversion case, the DAFoam adjoint linear solve fails at iteration 0 with `PetscConvergedReason: -9 (DIVERGED_NANORINF)` and printed iteration-0 residual $7.091590452305 \times 10^{-4}$. It fails identically under all five available `jacMatReOrdering` values — two produce `-9`, three produce `-3` flat stagnation, none converges — and the printed initial residual is identical to 13 digits across all of them, which is the tell that the failure is in the factorization, not the Krylov space.

6.2 The offline reproducer, which made every candidate a ten-second experiment

The dumped CBFS system (`dRdWTPC`, 210,592², nnz 13,710,468, plus the right-hand side whose norm matches the printed iteration-0 residual to 13 digits) was solved with a `petsc4py` harness that replicates `DALinearEqn::createMLRKSP`’s call sequence *in order* — `KSPSetFromOptions` first, hard-coded `GMRES/ASM/sub-ILU` settings second — under `mpirun -np 4`, the real layout:

```
RESULT reason -9 its 0 finalres 7.091590452305e-04 wall 7.01s
```

Bit-for-bit DAFoam’s failure — same reason, same iteration count, same residual — with `PCView` confirming full parity (ILU fill 1, ordering `rcm`, shift `NONZERO`, pivot-in-blocks, `zeropivot 2.22e-14`).

6.3 Runtime options: reachable, measured, and empty

A standing belief in three independent internal readings held that no PETSc runtime option can reach DAFoam’s ASM sub-block factorization. That was half wrong, and the half matters. PETSc’s PCSetUp_ASM calls KSPSetFromOptions on each sub-KSP it creates *inside* KSPSetUp — after DAFoam’s one visible call site, before DAFoam’s hard-coded per-block overrides. Measured, not argued:

variant (all via PETSC_OPTIONS)	deployed-factor evidence	result
control (parity with DAFoam)	zeropivot 2.22e-14, rcm, shift NONZERO	-9 , iter 0
-sub_pc_factor_nonzeros_along_diagonal	diagonal already has no zeros	-9 , iter 0
-sub_pc_factor_zeropivot 1e-8	“tolerance for zero pivot 1e-08” in PCView — <i>the option reached the factor</i>	-9 , iter 0
-sub_pc_factor_diagonal_fill	applied	-9 , iter 0

Two conclusions, one per column. **Reachability:** factor options do land, so “compiled-in, nothing reachable” was too strong. *Added 2026-08-10:* the boundary is now exact, and it is narrower than this paragraph once implied — factor options land because PETSc applies them after DAFoam’s own overrides, while every option that selects the Krylov method or the preconditioner family is discarded at twelve call sites (Section 7.12). That boundary is a property of the shipped build and is stated here as one: it has since been moved on a locally patched image, which changes nothing about what a user of the shipped build can reach. **Mechanism:** a 10^{-8} zero-pivot threshold with MAT_SHIFT_NONZERO active still produces the NaN, so this is *not* a small-pivot-below-threshold event the shift machinery could catch. The failure is factor growth — enormous elements in L/U that overflow when applied — which no shift and no diagonal permutation fixes; only pivoting or completeness does. This also explains, mechanically, the earlier observation that `spilu` reported “exactly singular” at every drop tolerance on the same dumped system while `splu` succeeded at every pivot threshold.

6.4 The one-word change, offline and then in-solver

sub-PC	ordering	reason	iterations	wall	per-block factor size
ILU(1) (DAFoam)	rcm	-9	0	7.0 s	6,963,844 nnz (before NaN)
LU	rcm	2 (converged)	347	68.0 s	100,311,134 nnz (~0.8 GB)
LU	nd	did not finish	—	killed at 600 s	—

Table 14: Offline mechanism test on the dumped system: one change, PCSetType(subpc, PCLU) where DAFoam hard-codes PCILU. The `nd` row is an operational finding worth keeping: PETSc’s native LU under nested dissection is pathologically slow on these blocks, so `rcm` is the ordering that works with complete LU here — the exact reverse of its role under ILU, where `rcm` is the ordering that surfaces the NaN.

PCSetType(MLRsubpc, PCILU) is hard-coded at `DALinearEqn.C:266-267`, so complete LU needs a recompile. The patch adds an environment-variable switch at that line: when `DAFOAM_SUBPC_TYPE=lu` is set, the local PC type becomes PCLU; otherwise not one line of behaviour changes. In-solver, on B3’s exact `-9` configuration:

```
DAFOAM_SUBPC_TYPE=lu: ASM sub-block PC set to complete LU
Main iteration 0    KSP Residual norm 7.091590452305e-04
Main iteration 400  KSP Residual norm 6.488778512831e-04
Main iteration 500  KSP Residual norm 1.778417281485e-05
Main iteration 667  KSP Residual norm 6.922418564747e-10
**Completed**! Total iterations: 667. PetscConvergedReason: 2. 232.2 s
```

The residual history is worth reading rather than skipping: the first 400 iterations crawl, because the assembled `dRdWTPC` the LU inverts is not the matrix-free operator GMRES applies; then the Krylov space catches the dominant cluster and the residual falls six decades in 267 iterations. Slow-then-superlinear is what a correct strong preconditioner on a mismatched operator looks like. Flat-to-thirteen-digits is what the dead ILU looked like. They are not the same thing.

6.5 The gradient, and its verification

The adjoint linear system depends on the objective, not on the design variable, so the converged solve carries over directly to the field-inversion design variable: 21,000 per-cell β components, same 667 iterations, same reason 2, gradient norm $1.4558046603 \times 10^{-5}$. **This is the first converged field-inversion gradient on a closure-relevant case in this lab.**

cell	h	central FD	adjoint $g[i]$	rel. err
5491 (largest $ g $)	0.05	1.914384790951e-06	1.916018813330e-06	0.085%
5491	0.1	1.907246786675e-06	1.916018813330e-06	0.460%
6740	0.05	1.679642395377e-06	1.678653382471e-06	0.059%
12486	0.05	1.472401682783e-06	1.469472928907e-06	0.199%
6490 (<i>chosen by the verification sweep, never published by the lab</i>)	0.05	1.520027166028e-06	1.519705840939e-06	0.0211%

Table 15: Central-difference verification of the 21,000-component gradient, fresh process per point, cold reset of the processor directories. The unperturbed control run reproduces the adjoint run’s objective to *exactly zero difference* ($1.5279278906359758 \times 10^{-2}$, all 17 digits) — which doubles as a primal-invariance proof across the switch. The doubling of the error from $h = 0.05$ to $h = 0.1$ on cell 5491, with the FD moving *away* from the adjoint value as the step grows, is central-difference truncation behaving exactly as $O(h^2)$ predicts: the agreement is step-limited, not noise-limited.

6.6 Where the remedy comes from, and why it is not exotic

Switching failing ILU sub-blocks to LU is the PETSc developers’ own canonical escalation, not an invention of this lab. From the `petsc-users` archive, verbatim: “*The default preconditioner with ILU(0) on each process is not appropriate for your problem and is producing overflow. Try -sub_pc_type lu and see if that produces a different result*” (Barry Smith, `msg24474`, 2015-03-26); and `msg26973` (Hong, 2015-10-27) demonstrates on a zero-diagonal system that ILU with any shift “does not converge” while `-sub_pc_type lu -sub_pc_factor_shift_type nonzero` converges in 24 iterations. The PETSc FAQ states, verbatim: “*The convergence of many of the preconditioners in PETSc including the default parallel preconditioner block Jacobi depends on the number of processes.*”

The complete upstream DAfoam remedy ladder for a non-converging adjoint, assembled from every place it is stated (the run-script guide, the FAQ, the doxygen `DAOPTION` page, and the live discussion threads) is: `renumberMesh -overwrite; pcFillLevel 1 → 2; jacMatReOrdering rcm → nd/natural; gmresRestart/gmresMaxIters up; asmOverlap up; upwind schemes; (since 2025-12) kahip decomposition; and transonicPCOption` for transonic cases. That last rung has a trapdoor of its own, reported in Section 7: the value this lab’s transonic scripts carried on it is dead code for the solver they ran, accepted without a warning and echoed back as though live. **Nothing in that ladder changes the sub-PC type; the ILU choice itself is never questioned upstream**, and no upstream doc, issue or discussion mentions ILU singularity, zero pivots, or a direct/sub-direct factorization at all — under 8 targeted searches plus 4 doc-page reads. Meanwhile `mdolab/dafoam` discussion #1002, converted to issue #1011 on 2026-07-29 and still open, shows users exhausting the documented ladder on *official tutorials* and stalling: the failure mode this switch was built for is live upstream today.

The shape of a filing. An upstream pull request exposing the sub-PC type as an `adjEqnOption` key — default `ilu`, behaviour unchanged — would be novel, is one option plumb, and is backed by both a measured unblock and named-PETSc-developer precedent. Our local implementation is one hunk at `DALinearEqn.C:266` (`demo-output/website/dafoam/subpclu_patch/DALinearEqn_subpclu.patch`, regenerated at 9796f90f); its default-off behaviour was verified by re-running the unpatched configuration on the patched image and reproducing the `-9` at iteration 0 with all 13 residual digits intact. *Added 2026-08-07:* the committed diff is now mechanically applicable. The version this document originally cited carried absolute headers into a scratch directory that no longer exists, so `git apply` failed on the very file cited as the reproduction diff — filed by a family supervision pass and fixed by regenerating against the stock `DALinearEqn.C` extracted from the shipped image with `a/src/adjoint/DALinearEqn/DALinearEqn.C` headers. **git apply -check now PASSES** from the `DAFoam` repository root, and the 19-line applied block was verified byte-identical to the change in the deployed `dafoam-subpclu:v1` image. One declared delta: the regenerated patch adds a warning on a set-but-unrecognized `DAFOAM_SUBPC_TYPE` value (`LU`, `Lu`, a trailing space — all of which previously fell through to stock `ILU` in silence), and the `v1` image is *not* rebuilt, so that warning exists in patch text only and drivers must keep asserting the sub-LU log line until a rebuild lands.

6.7 What the change does *not* fix

On the NASA hump (51,626 cells, 51,626 β design variables) the same one-word change moves the failure from catastrophic to honest, and no further. The `-9` is gone — the complete-LU factorization of blocks `ILU` cannot factor completes, and `GMRES` iterates with a monotonically descending residual, 1.094138002900 at iteration 0 to 0.9544468674795 at iteration 900 — but it descends slowly, 12.8% in 900 iterations (accelerating: 0.75%, 1.1%, 1.2%, 1.6% per successive hundred) and far from the target. The run was stopped deliberately at roughly iteration 900 when host available memory fell from 30.5 GB to a low-water **1.62 GB** under about 19 GB of LU factors plus a still-filling 2000-vector restart space, on a shared box.

Stated for whoever picks this up: **the hump blocker is no longer the singular factorization — it is Krylov convergence rate against a memory envelope**, which is a different and honestly-named problem.

Added 2026-08-08. The same envelope bounds where this remedy can be carried at all. Taken to a compressible transonic case — `ONERA M6`, 21,840 cells, `np=4`, six transported fields, $\sim 38k$ unknowns per `ASM` sub-block — the complete-LU factorization was OOM-killed at a 10 GB cap and, at 20 GB, held 19.94 GiB resident plus 12 GB of host swap while completing *zero* hundred-iteration `KSP` blocks in 27.9 minutes, against the same case's stock-`ILU` run at 5,876.6 MiB total and 82 s per hundred iterations. Feasibility of the sub-LU switch is block-size- and field-count-bound, not case-count-bound: `CBFS` fits, `M6` at `np=4` does not. That case turned out not to need it — Section 7.

7 The option that parsed and did nothing

Section 6 reported a wall that was where it looked, and a remedy for it. This section reports the case that remedy was carried to next, where the wall was real and its cause was not what the record said. The misattribution was this lab's own, in this lab's own records, and the correction is printed here at the same length as the claim it replaces.

7.1 The record this section corrects

From 2026-07-29 the record read as follows. The `ONERA M6` family — compressible, transonic, `DARhoSimpleCFoam` — returns `PetscConvergedReason: -5 (DIVERGED_BREAKDOWN)` on *both* adjoint solves at every mesh size tried: 21,840, 42,120, 79,560 and 99,840 cells, while an

incompressible case of this lab’s converges cleanly at 63,920. Memory was excluded — the smallest failing case peaks at 5.9 GB with more than 20 GB free — and a dedicated conditioning campaign (R5_ADJOINT_CONDITIONING.md) measured what could be measured about the operator:

- The assembled preconditioner matrix `dRaWTPC` ($200,360^2$, 22,017,324 nonzeros), dumped with a stock `-ksp_view_pmat` and analysed offline, carries a **diagonal magnitude spread of 1.48×10^{14} — 14.17 decades**, essentially the dynamic range of a double.
- `normalizeResiduals`: `None` converts the denormal collapse into honest `-3` stagnation, three times, on both objectives independently — and *worsens* the measured diagonal spread to 16.14 decades, which is how the record came to name “a second, still-live layer of ill-conditioning.”
- Both attempts to strengthen the preconditioner from that sane baseline — `pcFillLevel 0 → 1`, and the literature-precedented nested-Richardson wrap (`globalPCIters/localPCIters = 3`) — reintroduce the collapse, in four independent solves, at exactly iteration 1000: the GMRES restart boundary.
- Jacobian colour count does not separate the converging cases from the diverging ones (1,999 colours on a converging incompressible case against 1,233–1,391 on every diverging compressible one).

The failure also lies about itself, and the record caught that much: when the residual denormal-collapses toward zero the relative-ratio gate in `DALinearEqn::solveLinearEqn` is trivially satisfied, so a `-5` run prints “*Residual tolerance satisfied, solution finished!*” and carries on to the next objective. Every claim above was measured, is reproducible, and still stands as measured. What was wrong was the sentence they were assembled into: *the M6 adjoint is blocked by conditioning at every mesh size.*

7.2 The lever that was never pulled

Every archived M6 run — the ladder logs, all seven R5 campaign logs at line 489, the reordering pair, the 21,840-cell sweep baseline at line 410 — echoes `transonicPCOption 2`; in its own `daOptions` dump. Read the solver that consumed it, verbatim from the shipped image:

```
// src/adjoint/DAResidual/DAResidualRhoSimpleCFoam.C:171-176
// for PC we do not include the div(phid, p) term, this improves the convergence
if (isPC && daOption_.getOption<label>("transonicPCOption") == 1)
{
    pEqn -= fvm::div(phid, p_, divPhidPScheme);
}
```

That is the only comparison against `transonicPCOption` in the file. A branch on `== 2` does exist — at `DAResidualTurboFoam.C:176`, in a different solver, doing something else entirely (dropping the off-diagonal terms of the ϕ residual). `pyDAFoam.py:396` initialises the option to `-1` and documents it, verbatim: “*Which options to use to improve the adjoint equation convergence of transonic conditions / This is used only for transonic solvers such as DARhoSimpleCFoam*” — naming as its example the solver for which the value our scripts carried does nothing at all.

The value is read through a plain `getOption<label>` lookup. It is not validated, not range-checked, and not reported: the run prints `transonicPCOption 2`; in its settings echo — which is a faithful report of the *dictionary*, and says nothing about the *solver* — and then takes no branch. So the sentence the record could not write until 2026-08-08: **no archived M6 adjoint, at any mesh size, ever ran with an active transonic preconditioner.** The wall was characterised in full with the mitigation built for exactly that symptom silently off — and that mitigation is a rung on the upstream remedy ladder quoted in Section 6 (“`transonicPCOption` for transonic cases”), the rung a `DARhoSimpleCFoam` user in trouble is told to try.

The search space, stated, because a deadness claim without one is not a finding. This lab learned that rule the expensive way two days later (Section 8.3), so the claim above is stated with its own reach attached. In the shipped image, `grep -rn transonicPCOption` over the entire `src/adjoint` tree and `pyDAFoam.py` — not one file, and including `.H` headers — returns exactly four source occurrences: `DAResidualRhoSimpleCFoam.C:173 (== 1)`, `DAResidualTurboFoam.C:161 (== 1)` and `:176 (== 2)`, and the `pyDAFoam.py:396` default of `-1`. Line 173 is the only occurrence in the file that governs the solver these cases ran, and it tests `== 1`. The claim is a claim about that search space, and the search space is the build’s.

The finding is due to the pre-registration of an unrelated arm (`A3_SUBLU_PREREGISTRATION.md §2, 12D3A7A3`), which went to the source to state what the lever *was* before pulling it, and found it inert. It cost nothing and it invalidated an attribution nine days old.

7.3 Two not-evaluables, printed at their cost

The arc from that discovery to a converged adjoint ran through two arms that produced no answer to the question asked, and both are printed here because their blockers are findings.

The mesh that was born broken. The first arm targeted the ladder’s cheapest rung, an archived 24,960-cell “vcoarse” M6 mesh. DAFoam’s own quality gate rejected it before iteration zero: “*Failed 5 mesh checks*” — 23 negative-volume cells (minimum $-3.302747834736008 \times 10^{-9}$), 25 cells at aspect ratio 2.077×10^{95} , non-orthogonality 135.3° , 144 incorrectly oriented face pyramids, skewness 55.4. Adversarially defended two ways: stock `checkMesh`, outside DAFoam, reproduces every figure; and the preserved July-28 decomposition of the *pre-repair* geometry carries the identical 23 negative cells to the last digit under an offline divergence-theorem volume computation. The mesh was born broken — a pyHyp extrusion collapse at the wing-tip trailing edge on a $4\times$ -coarsened surface, which makes it a second, cross-geometry specimen of the generator pathology this lab already had on record for NACA 0012 (`GENERATOR_FINDING_pyhyp_aspect_ratio.md`). It also rediagnosed the archived “SEGV during `decomposePar`”: two orphan `autoPatch` patches (`auto4`, 176 faces; `auto5`, 11 faces) unmapped by the tutorial `createPatchDict`, so `decomposePar` failed *cleanly* and the old wrapper launched the solver anyway against half-written `processor*` directories. Cost: 0.53 core-min of 30. The mesh birth-certificate rule in the lab’s verification charter (v1.5 §9) comes from this arm.

The remedy of Section 6, which does not fit here. The re-file carried both levers — `DAFOAM_SUBPC_TYPE=lu` and `transonicPCOption: 1` — onto the 21,840-cell sweep mesh, which passes the gate (asserted *before* the pre-registration was committed). Both were proven active in-log, the first time on any M6 run. The sub-LU factorization then proved memory-infeasible on this case: OOM-killed at a 10 GB cap, and at a 20 GB cap 19.94 GiB resident plus 12 GB of host swap with **zero completed 100-iteration KSP blocks in 27.9 minutes** — against the record ILU run’s 5,876.6 MiB total and 82 s per hundred iterations. The lever’s feasibility is block-size- and field-count-bound: CBFS at `np=4` is 21,000 incompressible cells and fits easily (Section 6); M6 at `np=4` is ~ 38 k unknowns per ASM sub-block across six transported fields, and does not. The arm spent 127.5 core-min against 30 and was ended by a host-memory floor, both stated on its face. Its verdict was recorded as *not evaluable, memory*: mapping a memory death onto “the conditioning wall is confirmed” would have been a false confirmation of the very sentence that was about to be overturned.

7.4 The epilogue: one token, on the identical configuration

The third arm changed one token against the archived `runScript.py` — `transonicPCOption 2` $\rightarrow$ `1`, the complete functional diff — with no sub-LU (`DAFOAM_SUBPC_TYPE` unset, stock ASM/ILU(0)),

the same mesh, the same partition, the same coloring cache, the same **natural** ordering and 200-vector restart, cold-started from uniform fields.

solve	configuration	its	KSP residual path (printed every 100 iterations)
C_D	tpc 2 (record)	400	$2.120880199369 \times 10^{-2}$ flat through 300, then denormal $6.467896765713 \times 10^{-310}$; reason -5
C_D	tpc 1	368	$\rightarrow 2.045016030763 \times 10^{-6}$, 70.6 s; reason 2
C_L	tpc 2 (record)	600	$1.839195903440 \times 10^{-1} \rightarrow$ stall at $1.839160961071 \times 10^{-1}$ for 300+ iterations, then denormal $3.945602898014 \times 10^{-308}$; reason -5
C_L	tpc 1	383	$1.839195903440 \times 10^{-1} \rightarrow 6.86 \times 10^{-2} \rightarrow 4.47 \times 10^{-3} \rightarrow 1.89 \times 10^{-4} \rightarrow 1.831279779570 \times 10^{-5}$, 101.7 s; reason 2

Table 16: The one-token arm against the 2026-07-30 record, same case directory, same 21,840-cell mesh. Record: `run_opt5_onera_n15_21840.log`. Converged arm: `tpc1_computetotals.log`, pre-registration 13C244CD, result 11B90D25. Healthy norms throughout, no NaN anywhere, `rc=0`, and the full totals dictionary printed — **the first converged M6-family adjoint, and the first M6 gradient, in this lab’s history**, at 7.33 core-min of the ~ 28 approved.

The line worth reading twice is the C_L start value. The record’s solve sat at $1.839160961071 \times 10^{-1}$ for three hundred iterations before collapsing; the converged solve *starts* at $1.839195903440 \times 10^{-1}$ — the same linearization state to four significant digits — and descends straight through the record’s stall point to convergence. Same operator, same right-hand side, same starting residual: what changed is a term dropped from the preconditioner’s pressure equation.

7.5 The control that could have taken it back

A first-in-history convergence claim on a nine-day-old failure has one obvious confound: host and case-state drift between 2026-07-30 and today. The pre-registered negative control (31EEDE16, filed before launch) ran the archived `runScript.py` itself, zero edits, on today’s host, cold-started, with the expectation written down in advance — stall at the known value, `DIVERGED_BREAKDOWN` — and the falsifier stated: *if the control converges, the token story is wrong and the finding reverts to open*.

It did not converge. It reproduced the record **bit for bit** (`ctrl1_computetotals.log`, 551A7BA5):

solve	printed sequence, control vs the 2026-07-30 record	match
C_D	$2.120880199369\text{e-}2$, $2.115399810107\text{e-}2$, $2.118080629916\text{e-}2$, $2.111512927865\text{e-}2$ (iterations 0–300), then denormal	every digit
C_L	$1.839195903440\text{e-}1$, $1.839114940558\text{e-}1$, $1.839524885689\text{e-}1$, $1.839160961071\text{e-}1$, $1.865300757639\text{e-}1$, $1.841505669984\text{e-}1$, terminal denormal $3.945602898014\text{e-}308$	every digit

Table 17: The negative control against the record. Both solves -5 , at 400 and 600 iterations. The elapsed times are not comparable and are not compared — the record run computed its Jacobian coloring from scratch and this one reused the cached `dRdWColoring_4.bin` — but every printed residual is. The terminal C_D denormals differ ($6.544171804409 \times 10^{-310}$ against $6.467896765713 \times 10^{-310}$): that is sub-normal garbage below any meaningful precision, and it is reported rather than rounded into agreement. Cold-start signature 0.5969274433533561, bit-identical to the converged arm’s. 8.53 core-min.

The causal statement this earns, and no more: on one host, one day, one cold-start protocol, with the two arms’ C_L solves starting from the same printed number, the only difference between

double DIVERGED_BREAKDOWN and double reason 2 is transonicPCOption 2 $\rightarrow$ 1. Identical state, opposite outcomes, one token.

One incidental result falls out of the control: both M6 arms ran on dafoam-subpclu:v1 — the patched image of Section 6 — with the switch unset, and the control reproduced, to every printed digit, a record measured on the stock image. That is a default-off regression for the sub-PC patch on a second case, a different solver, and a failure mode the patch was never aimed at.

7.6 The gradient, and the two components it does not certify

The converged solve produces a gradient; by this lab’s charter (§7) that gradient is an existence proof until it has a finite-difference table. The pre-registration (4AEF4A2F, addendum B9F42631) fixed the components, the steps, the noise-floor rule and the pass band before either attempt ran, under the tight-primal discipline (primalMinResTo1 10^{-8} ; disclosed on its face: the primal plateaus below the 10^{-4} hard gate without reaching 10^{-8}).

Attempt 1 at $h = 10^{-3}$ landed in the pre-registered *not evaluable* branch — every component’s objective delta sat below ten times the measured baseline drift (2.107×10^{-6}) — and the steps were re-sized from the measured floor rather than argued. It also fired this lab’s own index-label lesson: the pre-registration’s reference value for `shape[5]` was a mis-parse of a line-wrapped 120-column numpy row, and the true maximum-magnitude component is `shape[115]`. The perturbation was always applied to the component named, by construction, so no number moved; the mis-parse was disclosed and the real maximum component added to attempt 2.

component	adjoint	FD(h)	FD($2h$)	Δ/floor	step-cons.	verdict
<code>patchV[1]</code> (AoA)	7.64611788e-3	7.66002800e-3	7.62500232e-3	94	0.46%	PASS, 0.18%
<code>shape[115]</code> (max g)	−1.24189676e-1	−1.23048667e-1	−1.22797280e-1	1513	0.20%	PASS, 0.93%
<code>twist[1]</code>	1.73254475e-3	1.75500957e-3	1.66219195e-3	22	5.29%	<i>no verdict</i> (1.28%)
<code>shape[5]</code>	7.34091832e-3	7.82812275e-3	8.47020820e-3	96	8.20%	<i>no verdict</i> (6.22%)

Table 18: Attempt 2, $h = 10^{-2}$ and 2×10^{-2} , noise floor 1.626×10^{-6} , adjoint reason 2 at 368 iterations — iteration-identical to the converged arm, third occurrence. Arm verdict per the pre-registered rule (≥ 2 evaluable components, all evaluable components inside the 5% band): **PASS**. Δ/floor is the objective delta in multiples of the measured noise floor; step-cons. is the change between the two steps, whose pre-registered gate is 5%. The two step-inconsistent rows are the two smallest signals; their bracketed relative errors are printed for the reader and carry **no verdict in either direction** — the honest reading is that the step-consistency gate, not the agreement, is what failed there. A3_FD3_PREREGISTRATION.md, result E4280B79.

Two components spanning two design-variable groups, one of them the largest gradient component in the vector, agree with central differences at 0.18% and 0.93%. Alongside that: the adjoint is bit-reproducible across three independent runs (`shape[115]` reads −1.24189680e-1 and −1.24189676e-1 on separate runs; every compared component agrees to all printed digits; all three converge in exactly 368 iterations). What this table licenses is stated narrowly: the gradient may be used as a number for `patchV`- and `shape`-class work *at this rung, at the tested steps*. `twist`-class components remain unverdicted.

7.7 The second rung, and a negative reason code refused

Added 2026-08-10. The rung above — the D3 sweep’s 42,120-cell member, admitted on a `checkMesh` certificate written before the pre-registration — now converges too, in a staged copy that left the archived member pristine. Both solves reason 2: C_D at **987** iterations, C_L at **1171**, the C_L residual descending monotonically from $1.839192419993 \times 10^{-1}$ to $1.824629776972 \times 10^{-5}$. The archived record for this same member is a double −5, measured — like every other archived M6 run — with `transonicPCOption 2`.

Its finite-difference table is *stronger* than rung 1’s, and in the way that matters:

component	adjoint	FD($h = 10^{-2}$)	FD($2h$)	step-cons.	verdict
patchV[1] (AoA)	7.90292883e-3	7.90232345e-3	7.90521694e-3	0.0366%	PASS, 0.0077%
twist[1]	1.80386732e-3	1.80882369e-3	1.80327914e-3	0.3065%	PASS, 0.2740%
shape[115] (runtime arg max g)	−1.30055677e-1	−1.30078078e-1	−1.30084297e-1	0.0048%	PASS, 0.0172%

Table 19: Rung 2, 42,120 cells. Noise floor 8.093×10^{-7} , evaluability threshold 8.09×10^{-6} ; every component’s objective delta clears it by $45\times$ to $3,213\times$. **All three components are evaluable and step-consistent** — the unverdicted rows of Table 18 do not recur at this rung, which locates their cause in rung 1’s smaller signals rather than in the gradient. Pre-registration 22c8E988, addendum 2B3E6517, result 4E982B4A.

One hazard closed mechanically rather than by care: the maximum-magnitude component was selected by the *runtime*, by $\arg \max|g|$ over the printed vector, with no hand-transcribed index anywhere in the arm. It chose **shape[115]** — the same FFD component that rung 1’s mis-parse had missed and its addendum recovered. The index-label failure of rung 1 cannot recur in this instrument, which is a better outcome than having been more careful.

The −3 that was refused, and then proven. Attempt 1 returned C_D at reason 2 in 987 iterations and C_L at −3 (KSP_DIVERGED_ITS) at *exactly* DA Foam’s default `gmresMaxIters` of 1000, with a residual of $1.307615741156 \times 10^{-4}$ after a monotone 1,407-fold descent. The pre-registration’s letter would have graded that DIVERGED, and the mapping under it would have read *the wall is real above this rung* — which would have been a false statement, and was refused as one, on the same ground as the sub-LU memory death two days earlier. The addendum, committed before attempt 2, staked the refusal on a falsifiable claim: *a raised budget alone will finish it*.

Attempt 2 — identical but for `gmresMaxIters` $1000 \rightarrow 2000$ — reached iteration 1000 at $1.307615740828 \times 10^{-4}$ against attempt 1’s $1.307615741156 \times 10^{-4}$: **ten significant digits**, the same solve with one of them truncated, and then converged at 1171. A cap is not a wall. This is the third refusal in this arc to be vindicated by a later measurement — the mesh-gate not-evaluable, the memory death, and now the −3 — and it is the cleanest of the three, because the vindication is a ten-digit identity rather than an argument.

What two rungs buy, and what they price. Iteration count scales *superlinearly* in cells across the pair: C_D 368→987 and C_L 383→1171 for a $1.93\times$ cell increase — exponents **1.50** and **1.70** — while per-iteration cost scales about linearly (0.144 s/iteration at rung 2). Two points define an exponent and cannot test it; the next rung is where it holds or breaks. The rung’s own cost is reported against its estimate without softening: 99.5 core-min against ~ 47 , a $2.1\times$ overrun whose cause *is* the finding — the arm was priced off cells when iterations scale at cells^{1.5–1.7}.

A second agent, dispatched on a mistaken belief that this one had been killed, staged nothing, stood down, and produced three independent items instead: a **checkMesh** replication agreeing to every printed digit, an outside reading of the arm log’s lever-activity and cold-start proofs, and — before attempt 2 returned — a written prediction of convergence near iteration 1,250 from the observed decades-per-hundred-iterations slope. Measured: 1,171, correct in direction and conservative by 7%.

7.8 The anomaly that was over-generalised, not invented

Added 2026-08-10; corrected the same day, and the correction is the point of the subsection. The dead lever cost a second sentence as well — though not the whole of it, as this document briefly claimed. R5 recorded a second result, four independent solves deep: from the sane baseline,

strengthening the preconditioner — `pcFillLevel 0 → 1`, or the literature-precedented nested Richardson wrap — **reintroduced** the catastrophic collapse, at exactly the restart boundary. That is a strange thing for a preconditioner to do, and the record treated it as evidence: it became the concrete mechanism for “a second, still-live layer of ill-conditioning” — the reading that a stronger local solve resolves the operator accurately enough to expose a scale disparity a weaker one never reaches. The lab reasoned around that anomaly for weeks.

A pre-registered triage (CFD6B5FF) put the three iteration-count levers back on the *working* configuration at rung 1, with the R5 prior named in the filing as the real test it had become, and with the collapse outcome pre-declared as a finding in its own right rather than a failure:

arm	lever echo, in-log	C_D its	C_L its	wall	verdict
baseline (11B90D25)	Fill Level: 0, Restart: 200	368	383	110 s	—
fill 1	ILU PC Fill Level: 1	236 (−35.9%)	250 (−34.7%)	105 s (−4.5%)	winner
restart 1000	GMRES Restart: 1000	311 (−15.5%)	306 (−20.1%)	103 s (−6.4%)	winner
Richardson	Global/Local PC ITERS: 3	209 (−43.2%)	211 (−44.9%)	133 s (+20.9%)	mixed

Table 20: The three levers on the PC-active configuration at rung 1, staged one case copy per arm, each carrying its four required in-log proofs. **All three converged, on both solves, at reason 2. None collapsed.** Verdicts are the pre-registered ones: a winner cuts iterations $> 10\%$ on both solves without increasing wall; *mixed* cuts iterations and costs wall. 22.7 core-min against 35–45 approved. A3_TRIAGE_LEVERS_PREREGISTRATION.md §8, 3AC8257E.

The two levers R5 recorded as collapse-inducing came back as the two largest iteration cuts on the board, and at 15:41Z on 2026-08-10 the record was amended to say what that appeared to mean: that the collapse had been an artifact of the dead configuration all along.

Three minutes later, that amendment was falsified by its own author. At 15:44Z the stage-0 transfer arm of the next rung returned. It is the Richardson lever at **42,120 cells with the transonic preconditioner ACTIVE** — the only change from the arm that had converged there at 987/1171 iterations, with `transonicPCOption 1`; in the dump, the cold signature, and no sub-LU banner:

```
Main iteration 200 KSP Residual norm 9.490658670647e-154
**Completed**! Total iterations: 200. PetscConvergedReason: -5. 152.66 s (CD)
Main iteration 200 KSP Residual norm 0.000000000000e+00
**Completed**! Total iterations: 200. PetscConvergedReason: -5. 228.32 s (CL)
```

Collapse to *exactly* zero at iteration 200 — which is `gmresRestart`, the first restart boundary. That is R5’s recorded signature reproduced verbatim, on a live preconditioner. The retraction was withdrawn on the record’s own face within the hour, at the same prominence it had been written (5AA3A142), and this document carries the corrected claim rather than the one it would have preferred.

What is actually true, stated at the width the measurements support. R5’s *observation* stands and is now reproduced under a working configuration. Only its *scope* was too broad. The corrected sentence is that the strengthening collapse is **real and mesh-dependent**: with the preconditioner active, Richardson helps at 21,840 cells and collapses at 42,120.

A candidate mechanism, labelled unproven. Both rung-1 strengthened arms converged in 209–250 iterations — inside roughly one 200-iteration restart cycle — and so met their first restart with an already-tiny residual. The rung-2 Richardson arm met its first restart with the residual still near 1.4×10^{-1} , and died there. The candidate rule is that a strengthened preconditioner destabilises the GMRES restart recomputation, and whether that bites depends on whether

mesh	transonic PC	fill 1	Richardson
21,840 (rung 1)	active	reason 2, 236/250	reason 2, 209/211
42,120 (rung 2)	active	not tested	−5, collapse at iteration 200
21,840 (R5, as recorded)	dead	−5, collapse	−5, collapse

Table 21: The three measurements that fix the scope. The dead lever cost the causal sentence about the *wall*; it did not invent the collapse. R5_ADJOINT_CONDITIONING.md, 2026-08-10 correction at 5AA3A142.

convergence completes before the restart matters — which is a function of mesh size. On that reading R5 saw collapse at 21,840 cells because a dead preconditioner left it far from convergence at its own restart. Two supporting points, one falsifiable prediction — *fill 1 should also collapse at 42,120 cells* — and that arm has not been run. It is stated here as a hypothesis and is not established.

So the accounting on this defect has a second line, and it is narrower than the one this document carried before this rung: the silently-ignored option produced a false causal sentence about the wall, and it caused a true observation about remedies to be recorded at the wrong width. The lab then reasoned around an over-general anomaly for weeks — every documented escalation looked, on the record, like it made things worse everywhere, when it makes things worse only above some mesh size the ladder has not located. A wrong wall stops work; a correct observation with the wrong scope redirects it.

One operational consequence, and it is the reason the transfer arm existed: the pre-registration’s plan to adopt Richardson for the next rung is **void**. Stage 0 was mandatory before adoption, it failed, and it stopped that rung from launching on a lever that would have collapsed it — for 15.93 core-min.

7.9 The third rung, and the ceiling it found

Added 2026-08-10. Rung 3, 79,560 cells, ran the baseline configuration after stage 0 voided the lever, with the cap raised to 4,000 — 1.6 times the pre-registered requirement. It did not converge, and this time the negative reason code is the finding rather than a budget:

	rung 2 C_L : −3 as <i>budget</i>	rung 3 C_D : −3 as <i>wall</i>
total residual reduction	1,407 ×, monotone	1.31 ×
behaviour at the cap	still descending $\sim 3\times$ per 100 iterations	flat
residual, iteration 1300 $\rightarrow$ 4000	— (converged at 1171)	1.615246604817e-2 $\rightarrow$ 1.615245992220e-2: 3.79×10^{-7} relative over 2,700 iterations
resolved by raising the cap?	yes — converged at 1171, iteration-1000 residual matching to ten digits	no — at the observed rate the 10^{-4} relative target is unreachable

Table 22: Why one −3 is a knob and the other is a wall. **The distinction was pre-registered before either was seen** (rung 2’s filing: “a −3 at *exactly the cap with monotone descent is reported as budget-limited, not as a wall*”), which is what licenses using it now. Rung 3’s C_D residual moves in the sixth significant figure across the last 2,700 iterations; C_L was never attempted, DAfoam raising `AnalysisError("Adjoint solution failed!")` after C_D . A3_RUNG3_N52_RESULT.md.

And it is not a memory death, which is exactly what makes a conditioning verdict sayable here. Peak container usage 11.65 GiB against a 22 GiB cap, host `MemAvailable` never below 17 GB, no swap growth, no OOM: the memory guard never approached tripping. This lab’s standing rule — a memory death is NOT EVALUABLE and never a conditioning verdict, the

rule that governed the sub-LU arm — cuts the other way here. Memory was comfortable and the solver still would not converge.

The ceiling, bracketed and deliberately not located. Two rungs converge and pass their FD gates with the transonic preconditioner active; the third stagnates on the same configuration with memory to spare. **The reopened ladder’s ceiling lies between 42,120 and 79,560 cells.** Nothing here says 79,560 *is* the boundary — the bracket is two measured points, not a location — and nothing here says the M6 adjoint is unreachable at that size by any means; it says the configuration that works at two rungs stagnates at this one. Nor is it claimed that stagnation and the earlier breakdown share a mechanism: a flat stall at a cap and a collapse to a denormal are different signatures, and this arm identifies neither.

The exponent breaks by kind. Rung 2’s measured scaling predicted $C_D \approx 2,570$ iterations here. The measurement is *no convergence at all* — a falsification not by 20% or 50% but by kind. **A two-point slope in this family is a description of two points, not a cost law**, and any extrapolation toward the 399,360-cell class is now foreclosed by measurement rather than by caution. That is worth more than a third confirming rung would have been, and it is the answer the rung was launched to get.

The conditional FD arm was correctly not launched — stage 1 did not converge, so its ~60–70 core-min were not spent — and the rung closed at 111.0 core-min against 170–190 approved, under budget because a pre-registered gate did its job.

7.10 What the ceiling is not: twelve eliminations and the number that survived

Added 2026-08-10; extended the same day with two entries the shipped build could not have produced (Section 7.13). Section 7.9 left the mechanism unidentified. A pre-registered diagnosis campaign then went after it in the one way that produces a durable result when it fails — by naming candidate causes in advance, deriving each one’s implied remedy, and testing them — and it failed. What it produced instead is an elimination table, which is worth printing because the eliminations are measurements rather than arguments.

The hypothesis the campaign most expected to confirm, killed by its own decisive test. The obvious reading of a stalling adjoint on a badly-scaled operator is that the assembled preconditioner matrix’s diagonal spread has grown past what ILU can carry. It has not:

	rung 2 (42,120 cells)	rung 3 (79,560 cells)
unknowns	384,518	724,609
nonzeros per row	111.8	112.8
diagonal magnitude spread	14.40 decades	14.47 decades
adjoint outcome	converges, C_D 987 iterations	stagnates

Table 23: Two rungs, 0.07 decades apart, and only one of them converges. **Diagonal spread does not discriminate between the rung that works and the rung that walls.** A 14.4-decade spread is survivable at 42,120 cells and fatal at 79,560, so whatever changed is not the operator’s diagonal scaling — which also retires the 14.17-decade figure of Section 7 as an *explanation*, while leaving it true as a measurement. A3_SAAD_DELIBERATE_CONDITIONING_PREREGISTRATION.md §A5, DDA82819.

The campaign records that this was the hypothesis it most expected to confirm, and the one a motivated reading would have confirmed. It is also the route this lab’s own capability strategy named — *diagonal-spread diagnosis* → *chosen preconditioner* → *converged* — so the finding is that the strategy’s own stated route is a measured dead end on this case.

candidate cause	how it was refuted, by measurement
Krylov method breakdown (orthogonality loss)	recursive and true residuals track to 7–12 digits through the stall (iteration 212: 1.618871461180e-2 both, twelve digits). The arithmetic is faithful; the operator is offering no progress
Krylov subspace too small	a 5× window (restart 200 → 1000) bought 1.647× against a pre-registered 10× bar
field-block separation (→ field-split)	the 2,000 largest diagonals are flat across all six state slots (301/357/331/324/351/336); every slot individually spans 13.1–14.5 decades; per-slot medians within 1.7×. There is no field to split off
geometric localisation (the tip-TE pathology)	the 500 largest diagonals have spatial standard deviation (1.32, 1.77, 0.60) against the whole mesh’s (1.62, 1.59, 1.19), and are <i>less</i> near-wing than average (21.8% vs 46.6% at $ y < 0.05$). The extremes are diffuse
mesh volume spread imprinted on the diagonal	$r = -0.135$, fitted slope -0.100 (-1.0 would mean $\text{diag} \propto 1/V$) across ten decades of cell volume
diagonal spread magnitude	Table 23: 14.40 decades converges, 14.47 does not
one-level Schwarz deficiency (overlap)	κ moved 0.979× and 1.062× at doubled <code>asmOverlap</code> , and the residual came back 3% <i>worse</i>
stronger preconditioner application (Richardson)	collapses at rung 2 (Section 7.8)
ILU fill level	<code>pcFillLevel</code> 1 at rung 3: 1.859× against the same 10× bar, flat after iteration 400
non-normality of the assembled operator	ν_F 2.65e-3 at the stalling rung against 4.30e-3 at the converging one — ratio 0.617 against a pre-registered bar of 3.0, confidence intervals non-overlapping. The stalling rung is <i>less</i> non-normal
augmented-restart Krylov (<code>-ksp_type lgmres</code>) — <i>reachable only after the patch</i>	–3 at 400 iterations, residual 8.236875832365e-2: 5.10× <i>worse</i> than the control and 3.88× above its own iteration-0 value, with restart carried explicitly at 200 so the comparison is with the control’s window (Section 7.13)
algebraic multigrid (<code>-pc_type gamg</code> <code>-pc_gamg_sym_graph true</code>) — <i>reachable only after the patch</i>	–5 at 200 iterations, residual 7.554080154832e+179 after barely moving through iteration 100; a 4-level multiplicative hierarchy confirmed in the readback, memory never a factor at 7.0 GiB. Off-the-shelf AMG at documented defaults , not the coarse-space class (Section 7.13)

Table 24: Twelve candidate causes, each refuted by a measurement rather than by an argument. The last two were unreachable from the shipped build at all and were run only after the defect of Section 7.12 was patched locally; both reported the control’s own iteration-0 residual, $2.121343646203 \times 10^{-2}$, which is what makes them commensurable with the ten above them. What survives is a negative characterisation with a number attached: the rung-3 adjoint operator, as ASM+ILU(0) preconditioning sees it, carries $\kappa \approx 10^{11}$ ($s_{\max}/s_{\min} = 9.57 \times 10^{10}$ at iteration 400), and that value is insensitive to every preconditioner parameter this build exposes.

The falsifier fired, and it was pre-stated. The campaign’s chosen arm was `asmOverlap 1 → 2` — the one parameter of the hardcoded PCASM that classical theory says controls κ — with the prediction written down before launch: $s_{\max}/s_{\min}$ must fall materially below 9.57×10^{10} and the residual must descend further than the overlap-1 control. Neither happened ($0.979\times$ and $1.062\times$ on κ ; residual $1.031\times$ worse), so the arm’s own stated alternative fires verbatim: the ill-conditioning is not Schwarz-mediated. The record reports the proof clause as NOT MET in the same words a success would have used.

Two disclosures travel with that arm and are reproduced here rather than dropped. The choice of lever was made *after* the diagnosis was in hand, which the record labels one notch weaker than its pre-registered ideal — “diagnosis-derived and outcome-pre-registered, not rule-pre-registered end to end” — and the cost ran to 61.6 core-min against ~ 35 approved, flagged at 45 before the diagnosis and 58 before the arm rather than discovered afterwards.

And the remedies the evidence does name are the ones the toolchain blocks. The elimination table points where a numerical analyst would expect: a two-level method with a coarse space, a different preconditioner family (PCFIELDSPLIT, PCGAMG), or an augmented-restart Krylov method (`lgmres`, `dgmres`). Every one of them is selected through the PETSc option channel, and every one of them is silently discarded by the solver — which is Section 7.12, and is the reason that defect is in this report rather than in a usability note. Two of them have since been run, on a build patched to honour that channel, and both failed (Section 7.13); that they could be run at all is the defect’s cost made concrete rather than a revision of anything measured here.

The one number that cuts against all of this, printed rather than smoothed. `pcFillLevel 1` at rung 3 was run as an engineering attempt with no diagnostic claim attached. It stalled, at $1.859\times$. But the condition estimate underneath it did not:

mid-cycle, iteration 300	control, fill 0	fill 1
$s_{\max} / s_{\min}$	1.379e10 / 0.294228	254.03 / 0.153029
$s_{\max}/s_{\min}$	4.69×10^{10}	1.66×10^3
residual at iteration 400	1.615428631404e-2	8.687780770929e-3 ($1.859\times$)

Table 25: **Seven orders of magnitude off the preconditioned condition estimate, and the solve still stalls at $1.9\times$.** Measured 100 iterations into cycle two, at the identical point of the identical cycle; readings at exact restart boundaries print 1/1 because the Hessenberg estimate is empty there, so only mid-cycle values are comparable (fill 1 reads 1.76×10^3 at iteration 700 and 1.08×10^4 at 800). `A3_RUNG3_FILL1_ENGINEERING_PREREGISTRATION.md` §5.1, 48B34CBF.

Stated plainly because it cuts against the tidy story: **the condition number estimated from the Krylov cycle is not predicting convergence here.** Two readings fit, and the arm that produced the number could not separate them.

1. **The estimator is measuring the cycle, not the operator.** `KSPComputeExtremeSingularValues` reports the extremes of the *current cycle’s* Hessenberg matrix — a lower bound over the subspace actually visited, not the operator’s spectrum. On this reading the control’s 4.69×10^{10} says how badly that cycle is spanning the space, and says less about conditioning than its magnitude suggests.
2. **The obstruction is not spectral at all.** If the operator is markedly non-normal, eigenvalue or singular-value clustering does not govern GMRES convergence — Saad’s own caution — and a seven-decade improvement in a spectral proxy buying $1.9\times$ is exactly what that looks like. This reading would explain the *entire* elimination table at once, because non-normality is invisible to every scale-, structure- and decomposition-based remedy in it.

Both readings were then tested, and both are refuted. The arm that raised the hypothesis named it as the outcome it should be most willing to report, pre-registered the measurement and its decision bar (7783F603), and refuted itself (15B64E81).

Reading (1), at zero cost, from logs already on disk. The estimator’s own subspace-to-subspace variability is bounded by its non-degenerate readings within each run: $2.22\times$ on the fill-0 control, $6.51\times$ on fill 1, $2.32\times$ on the overlap-2 arm — at most $6.5\times$. The fill-0-to-fill-1 jump is $\sim 2.5 \times 10^7$. **The jump exceeds the artifact scale by six to seven orders of magnitude**, so estimator noise cannot account for it. An unplanned control fell out of the same arithmetic: the fill-0 and overlap-2 runs, launched hours apart, return near-identical estimates at matched iterations ($4.31\text{e}10$ vs $4.39\text{e}10$; $4.69\text{e}10$ vs $4.59\text{e}10$; $9.57\text{e}10$ vs $1.02\text{e}11$), which validates the instrument’s reproducibility and independently re-confirms that doubling the Schwarz overlap did nothing.

Reading (2), by measurement, and it goes the other way. Departure from normality was measured as the normalised Frobenius commutator $\nu_F = \|P^\top P - PP^\top\|_F / \|P\|_F^2$, by a Hutchinson estimator with $k = 200$ Rademacher probes at a fixed seed — scale-invariant by construction, which is the diagonal-spread lesson applied — against rung 2 as the *converging* control, because the number means nothing without one.

	rung 2 — converges	rung 3 — stalls
unknowns / nonzeros	384,518 / 42,988,956	724,609 / 81,718,327
$\ P\ _F$	4.261146e12	7.587909e12
ν_F	4.298842e-3	2.651122e-3
95% confidence interval	[4.2813e-3, 4.3163e-3]	[2.6377e-3, 2.6645e-3]

Table 26: Ratio 0.617 against a bar of 3.0 fixed before the numbers existed. **The stalling rung is measurably less non-normal than the converging one, by $1.62\times$, with non-overlapping confidence intervals** — a resolved difference, not a null from noise. Both operators are only mildly non-normal in absolute terms ($\nu_F \sim 10^{-3}$ on a measure running to order 1). 0 of a 25 core-min cap: both operators were already on disk. A3_NONNORMALITY_DIAGNOSTIC_PREREGISTRATION.md §5, 15B64E81.

The limitation bounds the claim in both directions and is not dropped here. What was measured is DA Foam’s assembled dRdWTPC — not the matrix-free transpose Jacobian, and not the preconditioned operator AM^{-1} whose normality actually governs GMRES convergence, which this build never assembles. So the honest statement is that *the non-normality of the operator DA Foam builds its preconditioner from does not distinguish the rung that converges from the rung that does not*, and therefore fails to explain the table. It does not prove the preconditioned operator is well behaved.

Where that leaves the chapter. Twelve candidate causes have been tested and refuted by measurement: method breakdown, Krylov subspace size, field separation, geometric localisation, mesh volume scaling, diagonal-spread magnitude, one-level Schwarz overlap, stronger preconditioner application, ILU fill level, non-normality, and — once the option channel was patched open — augmented-restart Krylov and off-the-shelf algebraic multigrid. The surviving statement is unchanged and better defended than when it had nine: **the rung-3 wall is real, it is not memory, and nothing this build exposes — nor anything measurable on the operator it assembles, nor either of the two forbidden remedies tried at their defaults — distinguishes it from the rung that converges.** What remains unexplained is now precisely bounded rather than vaguely open, which is a different and more useful kind of ignorance. The boundary that used to close the investigation — that the remedy classes the evidence names cannot be selected at all from this build — has been crossed locally rather than argued away, and what lies beyond it is two more eliminations and one honestly untested remainder: a physics-appropriate coarse space, which is neither off the shelf nor cheap.

7.11 The second silent configuration: the time-0 rewrite

The same arc surfaced a second way for a configuration to be silently not what it says. At the end of a primal, `pyDAFoam` writes the end state back into the time-0 directory: on this case 0/U came out bit-identical to 1000/U (F77B2607). Nothing in the log announces it. Every subsequent run of that case directory therefore starts from the previous run’s converged fields — and the wrapper’s own `renameSolution` then hard-raises (`pyDAFoam.py:1543`) on the leftover 0.0001 directory it created, so the natural repair (delete the leftover time directory, rerun) clears the obstacle while leaving the overwritten time 0 in place. The failure mode is the dangerous kind: the run proceeds, and the cold start you assumed is not the start that ran.

Caught inside this campaign — an arm whose “cold” primal ran its full 1000 steps in 10 s from the previous run’s converged fields, finishing 2.2% in $\|U\|$ from the record’s own writeout — it was then audited backwards across the lab’s DA Foam record (`WARMSTART_AUDIT.md`, 9BB948B2; zero solver cost, log forensics only). The audit’s own priced discriminator failed first and is reported as such: “a warm primal is an order of magnitude faster” is *invalid* on this rung — cold and warm primals both take ~ 10 s, and the contrast the proposal had priced was coloring-computation time. The surviving discriminator is the first **Time step continuity errors** value: cold-from-uniform 0.597 against warm 0.0107, a $55\times$ separation, and bit-reproducible — *six* independent cold starts print 0.5969274433533561 to all 16 digits.

Nine conclusion-bearing records were graded against it. Seven are **cold-clean**, most of them by construction: the W4 studies ran each arm in its own staged case copy, which is structurally immune to the hazard, and their first continuity errors are bit-identical across arms — including the reordering ladder and the sub-LU unblock of Section 6, whose comparisons are therefore provably same-state. Two carry flags, neither load-bearing: a set of M6 lever nulls whose logs were lost with a session scratchpad (formally indeterminate, mechanism certain — and closed on 2026-08-10 by retiring all eleven as superseded rather than rerunning them: each tested a lever against a preconditioner that was never on, so a cold rerun would have certified nulls about a configuration no future run will use), and the R5 matrix dumps quoted at the head of this section, which were assembled about a warm state $\sim 2\%$ off in $\|U\|$ and whose conclusions are order-of-magnitude spreads (14 decades) that such a drift cannot move. No record-grade conclusion was overturned, and the audit rewrote none of them; the flags are printed here for the same reason they were printed there.

7.12 The third silent configuration: an option channel that answers and does not obey

*Added 2026-08-10. Prepared as a filing-ready defect candidate (`DEFECT_CANDIDATE_ksp_options_override.md`, F29378D9) and, like everything else here, **not filed**.*

`DALinearEqn::createMLRKSP` calls `KSPSetFromOptions(ksp)` — the function whose entire purpose is to let a PETSc user reconfigure a solver from the command line — and then overrides almost everything that call could have configured. Twelve call sites, all executing after it, so under PETSc’s last-setter-wins ordering the user’s options lose:

So these are all accepted without error and all have no effect: `-ksp_type lgmres` (augmented restarting), `-ksp_type dgmres` (deflated), `-ksp_type fgmres`, `-pc_type gamg`, `-pc_type fieldsplit`, `-sub_pc_type lu`, `-ksp_gmres_restart 1000`. The run proceeds with GMRES + ASM + sub-ILU regardless, and `-ksp_view` prints the hardcoded stack — so even the user’s verification step agrees with the code rather than with the user.

The detail that makes this a defect and not a documentation gap: the monitors work. Viewers and monitors are *not* overridden. `-ksp_monitor_true_residual`, `-ksp_view_pmat binary:` and `-ksp_view_rhs binary:` all behave exactly as PETSc documents. A user therefore receives **correct diagnostics and silently discarded remedies through the same channel** — which is the worst available combination, because the diagnostic path constitutes a working

line	call	what it overrides
138	KSPSetFromOptions(ksp)	<i>the point at which every <code>-ksp_*</code> / <code>-pc_*</code> option is applied</i>
144	KSPSetType(ksp, KSPGMRES)	<code>-ksp_type</code> — hardcoded to GMRES
158	KSPGMRESRestart	<code>-ksp_gmres_restart</code>
170	KSPSetPCSide(ksp, PC_RIGHT)	<code>-ksp_pc_side</code>
185	PCSetType(MLRMasterPC, PCKSP)	<code>-pc_type</code> (outer)
192	KSPSetType(MLRMasterPCKSP, KSPRICHARDSON)	inner KSP type
212	PCSetType(MLRGlobalPC, PCASM)	<code>-pc_type</code> — hardcoded to additive Schwarz
216	PCASMSetOverlap	<code>-pc_asm_overlap</code>
249 / 259	KSPSetType(MLRsubksp[i], ...)	<code>-sub_ksp_type</code>
286	PCSetType(MLRsubpc, localPCType)	<code>-sub_pc_type</code> — hardcoded to ILU
328	PCFactorSetLevels	<code>-sub_pc_factor_levels</code>
343	KSPSetTolerances	<code>-ksp_rtol</code> , <code>-ksp_atol</code> , <code>-ksp_max_it</code>

Table 27: `src/adjoint/DALinearEqn/DALinearEqn.C`, read in both images. The comment at line 137 — `// First, KSPSetFromOptions MUST be called` — shows the ordering is deliberate, which is what makes the *silence* the defect rather than the ordering.

demonstration that the option channel is live. Every signal available to that user says their command line was received. It was.

This lab is the worked example, twice over, and both halves were measured before the source was read. The whole diagnosis of Section 7.10 — the dumped preconditioner matrices, the true-residual tracking, the preconditioned $\kappa = 9.57 \times 10^{10}$ from `KSPComputeExtremeSingularValues` — was obtained *through this option channel*, and every remedy that diagnosis points to is selected through the same channel and discarded by it. A user who does the hard part correctly is left holding a diagnosis they cannot act on without recompiling `libDASolver.so`.

It also sharpens, rather than contradicts, the measurement in Section 6: factor options such as `-sub_pc_factor_zeropivot` *do* land and were proven to land by `PCView`, because PETSc applies them inside `PCSetUp_ASM` — after `DAFoam`’s overrides. The boundary is now exact. Options consumed after line 138 by objects `DAFoam` does not touch again take effect; options that select the Krylov method or the preconditioner family do not. “Compiled-in, nothing reachable” was too strong, and “the option channel works” is too strong in the other direction; what is true is that the channel carries diagnostics and drops strategy.

Three graded fixes, cheapest last.

1. **Respect the channel: move `KSPSetFromOptions` last.** Set the `daOptions` values first and call `KSPSetFromOptions` (plus `PCSetFromOptions`) as the final step, which is the ordering PETSc’s own examples use. Then `daOptions` act as defaults and runtime options as overrides — what both APIs promise. Cost: a reorder, plus explicitly re-applying any setting that genuinely must not be user-overridable, with a comment saying why.
2. **If the override is intentional, say so, in two places.** One `Info` line at setup naming the fixed stack and the ignored options, and the same sentence in the `adjEqnOption` reference. This costs nothing and converts a silent failure into a stated limitation.
3. **Cheapest partial: namespace the objects.** `KSPSetOptionsPrefix(ksp, "dafoam_")` and the same on the sub-KSPs and PCs, so the objects never consume unprefix user options and the boundary stops being invisible.

Fix (a) is now implemented, and the maintainer’s first objection is answered by measurement. *Added 2026-08-10, after the above was written.* The first fix was applied to a local image — `dafoam-ksptests:v1`, built from the `dafoam-subpclu:v1` lineage the whole A3 chain ran on, with both prior images left untouched and reachable — as **one call relocated, 28**

diff lines: `KSPSetFromOptions(ksp)` moved from the top of `createMLRKSP` to the end of it, so that `daOptions` act as defaults and runtime options as overrides. All three AD targets rebuilt in 29 s at -j 8. Two gates were pre-registered before the build, with the failure branch of the first written down as the more valuable outcome (A3_KSPOPTS_PATCH_PREREGISTRATION.md, results §6 at 0FCC98A0; patch archived at `kspopts_patch/DALinearEqn_kspopts.patch`, 3F631C4D).

gate	question, fixed before the build	measured
A — regression	Does honouring the option channel change behaviour when no options are set? Pass is rung 1's exact counts	CD 368, CL 383, PetscConvergedReason: 2 on both , cold-start signature 0.5969274433533561 to all 16 digits — the exact counts measured on the unpatched lineage (11B90D25) and reproduced twice since. 8.67 core-min
B — restoration	Does the reorder actually reopen the channel? Pass is <code>-ksp_view</code> reporting the requested type	<code>PETSC_OPTIONS="-ksp_type fgmres -ksp_view"</code> → type: fgmres , where the shipped build reports type: gmres . 8.40 core-min

Table 28: The two gates on fix (a). Gate A is the load-bearing one for a filing: **iteration counts are the sharpest available equality test**, and they did not move at all — so **the override is not load-bearing**, and this document's own recommended fix survives a maintainer's first objection by measurement rather than by argument. The pre-registration stated the opposite outcome as the more valuable one: a moved count would have meant the ordering matters beyond configuration, that fix (a) is wrong as recommended, and that fix (c) replaces it.

Gate B upgraded the recommendation, and the reason belongs in this report's thesis. With `-ksp_type fgmres` genuinely in force, the build's own diagnostic echo still prints `Solver Type: gmres` and `GMRES Restart: 200` — the `daOptions` values, not the effective ones. That is L-40 — *the switch you set is not the switch that ran* — occurring inside the upstream code and printed by the program itself, in a report whose entire argument is that class of defect. Fix (a) restores the user's control; it does not restore their ability to verify it, and control without verifiability is half a fix. **The revised recommendation is therefore (a) plus an effective-value echo:** query `KSPGetType` and `PCGetType` after the options call and print what the solver will actually run, instead of what the dictionary asked for. That is a few lines, and it closes the “which switch actually ran” gap that motivated this report in the first place.

A secondary finding from the same gate, which is not a bug and must still be documented. Under `-ksp_type fgmres` the readback reports `restart=30` — PETSc's default — rather than the `gmresRestart: 200` of `daOptions`, because `KSPSetType` rebuilds the object's internal state and DA Foam's `KSPGMRESSetRestart(200)` had been applied to the previous one. This is correct “user override wins” semantics. Its consequence is not benign: **a user overriding the Krylov type must also pass `-ksp_gmres_restart`**, and in this very test the inherited restart of 30 turned rung 1's 368-iteration convergence into a 1000-iteration -3. That failure is the small restart's doing and not the patch's — and a user who did not know to look would have read it as the method's.

Severity is **medium** and the class is what matters rather than the severity: no wrong number is produced, and everything DA Foam does configure works. It costs nothing until a user has a hard problem, and then it costs them the ability to act on their own correct analysis, in a way no log will show. A user of a discrete-adjoint code who reaches for `-pc_type gamg` is, by construction, a user who has already done the hard part.

7.13 The class the build forbade, tested at last: the eleventh and twelfth eliminations

Added 2026-08-10. With the channel open, the remedy class Section 7.10 named and could not reach became runnable. Two members were run — the two the evidence pointed at hardest — under a pre-registration whose first binding was written before any number existed and is quoted here because it governs how the result is stated: *a failure is the eleventh elimination; a success is one reachable remedy working, and not vindication of anything.* The shipped-toolchain ceiling verdict of Section 7.9 was measured against the shipped toolchain, and nothing run on a locally patched image revises it, in either direction (A3_STAGE2_UNREACHABLE_CLASS_PREREGISTRATION.md, A15B839D, results E1755100).

Both arms ran the diagnostic control’s own script — 400 iterations, `transonicPCOption 1`, `pcFillLevel 0`, restart 200 — so the only difference from the control is `PETSC_OPTIONS`. **Both reported the control’s exact iteration-0 residual**, $2.121343646203 \times 10^{-2}$, which establishes commensurability with the ten eliminations before them by measurement rather than by assumption.

arm	readback, quoted before grading	result at the graded solve
-ksp_type lgmres (augmented restarting)	type: lgmres, restart=200, aug. dimension=2, right preconditioning, unpreconditioned norm — the control’s window and the control’s norm	−3 at 400 iterations, residual $8.236875832365 \times 10^{-2}$: 5.10× worse than the control and $3.88\times$ above its own iteration-0 value. 19.4 core-min
-pc_type gamg -pc_gamg_sym_graph true	type: gamg, type is MULTIPLICATIVE, levels=4 cycles=v	−5 at 200 iterations, residual $7.554080154832 \times 10^{+179}$: barely moved by iteration 100 ($2.120914790716 \times 10^{-2}$) and then 180 orders of divergence. Memory never a factor , peak 7.0 GiB. 105.0 core-min

Table 29: The two previously-unreachable remedies, at documented defaults, against the standing control (residual $1.615428631404 \times 10^{-2}$ at iteration 400). Restart 200 was carried *explicitly* on arm 1 because Gate B had established that `-ksp_type` resets the GMRES-family settings — without it the arm would have inherited PETSc’s restart of 30 and the restart’s failure would have been read as the method’s. Both graded STALL against a pre-registered $10\times$ bar.

The anomaly on arm 1, flagged rather than binned. A residual ending $3.88\times$ above its own starting value is not what a GMRES-family method should produce. Two readings fit — genuine divergence, or LGMRES’s augmented recurrence reporting an intermediate norm differently in this PETSc build — and **this arm cannot separate them**. It does not need to for its verdict: under either reading the method neither converged nor descended. The anomaly is printed here for the same reason the record printed it, and no mechanism is claimed from it.

Bounded exactly as the pre-registration requires. What was tested is **two off-the-shelf members of the class, at documented defaults**, on a convection-dominated nonsymmetric transonic adjoint. GAMG’s defaults target elliptic/SPD-like operators even with `sym_graph` set, and this operator is outside that envelope — so the honest claim from arm 2 is that **off-the-shelf AMG fails here**, and explicitly *not* that a coarse-space method cannot work on this problem. A physics-appropriate coarse space, or AMG with smoothers chosen for this operator, remains untested, and it is not cheap. Arm 1 is a stronger elimination in one respect and weaker in another: augmented restarting is the arm the campaign’s own most reproduced observation pointed at — progress dies at the restart boundary, three independent times — and it is one member of that family at its defaults, not the family.

What the arms proved beyond their own verdicts. The readbacks are the measurement that makes Section 7.12 concrete: a Krylov type *and an entire preconditioner family* — a four-level multiplicative V-cycle hierarchy, built and reported by the solver itself — were selected at runtime on a build where both were previously unreachable and unreported. The defect costs a user exactly these two experiments, and they are the two a user in this position would try first. Two disclosures travel with the arms and are reproduced rather than dropped: arm 2 was stopped after its graded C_D solve rather than continuing C_L down this family’s known false-success path, and it billed 105.0 core-min against 25.9 estimated — a $4.1\times$ overrun whose cause is nameable, both arms having been priced off the ILU baseline’s per-iteration cost when GAMG’s hierarchy setup bears no relation to it. Separately, the shared launcher hardcodes a 16 GiB container cap where the pre-registration had stated 22 GiB; the cap was never approached (7.0 of 16), so it had no effect on the result, and the mismatch between what was registered and what ran is recorded rather than passed over.

7.14 A diagnostic that does not diagnose: three arms on the condition estimate

Added 2026-08-10. Section 7.10 printed one number that cut against the tidy story — a seven-decade improvement in the preconditioned condition estimate buying $1.9\times$ in residual. With stage 2 in hand, that is no longer one arm’s oddity. It is a result in its own right, and it is the one finding of this chapter that generalises past this case.

arm	$s_{\max}/s_{\min}$	behaviour
lgmres , iteration 400	2.70×10^5	−3, residual $5.10\times$ worse than the control
control (fill 0), iteration 400	9.57×10^{10}	−3, flat stall
gamg , iteration 100	7.71×10^{17}	−5, divergence to 10^{179}
pcFillLevel 1 , iteration 300	1.66×10^3 against the control’s 4.69×10^{10} at the same point of the same cycle	stalls at $1.859\times$ (Table 25)

Table 30: The standard diagnostic, read across four arms of one campaign. The first three span **five and a half orders below the control and seven above it** — twelve and a half orders end to end — across arms whose behaviour ranges only from *stalling* to *worse*. The fourth improves the estimate by seven decades and buys $1.9\times$.

The claim this licenses is narrow and worth stating anyway, because a practitioner meeting this operator class will reach for exactly this number: **the Krylov condition estimate is not the instrument for this question, and there are now three independent measurements saying so** — the fill-level arm improving it seven orders for $1.9\times$, and two stage-2 arms landing five and seven orders on either side of a control they both failed to beat. An analyst ranking the first three arms of Table 30 by their condition estimates would put **lgmres** ahead of the control, which is the reverse of how their residuals came out.

The limitation, stated with the result rather than after it. These are `KSPComputeExtremeSingularValues` readings — the extremes of the current cycle’s Hessian matrix, a *lower bound* over the subspace actually visited, not the operator’s spectrum. They are not all taken at the same point of the same cycle (the readings above are each arm’s own graded iteration, and readings at exact restart boundaries degenerate to $1/1$), and arm 2’s estimate is of a *different* preconditioned operator entirely, since GAMG replaced the preconditioner the other three shared. So this is not a claim that these operators’ spectra differ by twelve orders. It is the narrower and more useful claim: the number a user can actually obtain from a running solve, at the points a running solve offers it, does not order these configurations by how they behave. The estimator’s own within-run variability was separately bounded at $\leq 6.5\times$ (Section 7.10), which is what rules out reading the spread as sampling noise.

7.15 What is reportable here, and what is not

Three behaviours in this section are properties of the shipped tools rather than of this lab's cases, and they are the reason it sits in a defect report at all:

1. **A configuration option that parses cleanly and does nothing for the solver reading it.** `transonicPCOption: 2` is a legal value — for `DAResidualTurboFoam`. Given to `DARhoSimpleCFoam` it raises nothing, warns nothing, and is echoed back in the settings dump exactly as a live option would be. The information needed to reject it is available at the point of use: the solver knows its own type. A warning at option-read time on an out-of-range value, or a documented per-solver applicability table, would have converted nine days of misattributed conditioning work into a one-line log message. This is not the branch class of Sections 2–5 and it is not a wrong number; it is a usability and diagnosability defect, and its blast radius is measured: every M6 conclusion in this lab's archive, one of which was wrong.
2. **A state rewrite with no log line.** The time-0 overwrite is defensible as a restart convenience and indefensible as a silent one, particularly in a wrapper whose own rename step then fails on the artifact it just produced. Anyone A/B-testing an archived `DAFoam` case is exposed, and the exposure is invisible in the log unless you know to read the first continuity error.
3. **An option channel that carries diagnostics and drops strategy** (Section 7.12). Of the three this is the one with the largest audience, because it costs nothing until a user has a hard problem and then costs them exactly the move their analysis earned. It is also the only one of the three whose evidence is entirely in upstream's own source rather than in this lab's cases: twelve call sites, a five-minute reproducer on any `compute_totals` case, and three graded fixes of which the cheapest is one line per object.

None is filed. D4 and D5 have no prepared upstream report of the kind Sections 2 and 5 carry — what exists is the measured record cited throughout this section — while D6 has one, complete and **filing-ready** (`DEFECT_CANDIDATE_ksp_options_override.md`, F29378D9, extended with the implemented patch and its two gates at 5E720034), held with everything else. The shape a filing would take is small in all three cases — a validation warning, a log line, and a reorder — and that smallness is the argument for it. D6's is now smaller than the argument for it: the reorder exists, it is 28 diff lines, and it is measured behaviour-neutral by default on the sharpest equality test the case affords.

7.16 Scope, stated against ourselves

- **Two rungs, and then a ceiling.** 21,840 and 42,120 cells both converge and both pass their FD gate with the preconditioner active, where the archived record had a double `–5` at each. 79,560 cells does not converge, on ample memory (Section 7.9), so “one token was the wall” is a statement about two rungs and explicitly not about the family. The ceiling is *bracketed* between those two sizes and **not located**: no intermediate mesh has been run, 99,840 and the production-class meshes are untouched, and nothing here claims 79,560 cells is unreachable by other means. The iteration-scaling exponents (1.50 and 1.70) are withdrawn as a predictive law by their own test; they describe two points. **Twelve eliminations are not a proof of the thirteenth**: the mechanism is still unidentified, and the two newest entries were bought by patching the build rather than by understanding the operator.
- **The patched-image results stand beside the shipped-toolchain verdicts, never in place of them.** Everything in Sections 7.13 and 7.14 was measured on a local image

carrying this document’s own recommended fix. That image exists on one host, is pushed nowhere, and revises nothing: the ceiling verdict, the elimination table’s first ten entries and every reason code in this section were measured against the shipped build and are untouched. What the patched arms establish is what the shipped build *forbids* — and, for the two members tested, that reaching it would not have helped.

- **“Off-the-shelf AMG fails here” is the claim; “the coarse-space class fails” is not.** Arm 2 ran GAMG at documented defaults with `sym_graph` set, on a convection-dominated nonsymmetric adjoint that is outside the envelope those defaults target. A physics-appropriate coarse space is untested, is not cheap, and is not indicted here. The same caution applies one notch weaker to arm 1: one member of the augmented-restart family at its defaults, not the family.
- **The regression evidence for fix (a) is one case, and it is stated as one case.** Gate A is rung 1’s converged configuration on one host, cold-started: bit-identical iteration counts and cold signature, which is strong evidence *on that configuration* that the override is not load-bearing, and not a proof that no DAfoam configuration anywhere depends on the ordering. An upstream reviewer would want the same check on an incompressible case and on a Richardson-wrapped one; neither has been run. The effective-value echo the revised recommendation adds is drafted and *not* implemented, and fixes (b) and (c) remain untested.
- **The old measurements stand; one sentence built on them is withdrawn and one is rescoped.** The 14.17-decade diagonal spread, the `normalizeResiduals` signature change, the restart-boundary collapses and the colour counts are all still true as measured, on the configuration they were measured on. Withdrawn: the causal sentence they were assembled into. Rescoped, not withdrawn: that strengthening the preconditioner reintroduces collapse — real, reproduced on a live preconditioner at rung 2, and mesh-dependent (Section 7.8). This document asserted the stronger, wrong version of that correction for part of one day and prints both the claim and its falsification.
- **The gradient claim is Tables 18 and 19 and nothing wider.** Two verified components at rung 1 (two unverified), three at rung 2; no claim about the other components of a vector with hundreds of them, at either rung, and no optimization run behind either.
- **The lever triage is a rung-1 measurement.** Direction and rough magnitude at 21,840 cells. Transfer to a taller rung is unproven by design and is stage 0 of the unlaunched rung 3.
- **The upstream ladder claim is unchanged.** Section 6’s finding — that no documented remedy changes the sub-PC type — is not affected by this section. The `transonicPCOption` rung of that same ladder is live and useful; the defect is that a wrong value on it fails silently.
- **Nothing here is filed**, and the lever-activity discipline that produced it is now this lab’s rule rather than its discovery (L-40, verification charter v1.5 §9): a load-bearing option is evidence only when a runtime log proves it ran.

8 Methods: adversarial verification as practice

Every headline claim in this document has survived an independent adversarial sweep. That phrase is used precisely, and this section says what it buys, because a reader assessing seven unfilled defect claims is entitled to know what standard they were held to.

8.1 What a supervisor sweep is

A verification agent is dispatched with the posture that **the claim is wrong until it survives attack**. It does not review the record; it re-derives the record. Concretely, across the six sweeps behind this document:

- **Independent instruments, not re-runs of the lab’s scripts.** The mechanism sweep rebuilt the processor-to-serial permutation from scratch with its own parsers, from `cellProcAddressing/faceProcAddressing` and the `AdjointIndexing` dumps, then compared against the stored map: identical bit-for-bit in target and sign. The rotation sweep wrote its own NumPy check of the corrected derivative (250 draws, against Richardson central differences of the singularity-free reference) and its own DAfoam driver that wraps `idwarp.USMesh.warpDeriv` itself rather than the lab’s hook.
- **A new validation channel the original never used.** The `AdjointIndexing` dumps carry cell and face centre coordinates; mapping `np=4` coordinates onto `np=1` coordinates gives max deviation **0.0** over all 26,149 and 26,227 entries, validating every entry of the permutation exactly and independently of any solved field.
- **Re-measurement from scratch, in the container.** One cell of the phenomenon table (`np=2`, patched) was re-run with the sweep’s own driver, own staging, own container invocation: analytic, FD, absolute error, relative error, baseline C_D and GMRES iteration count *all identical to every printed digit*, and carrying the provenance stamp the archived run lacked. Cost: 86 s wall, 2.9 solver core-min.
- **A point the lab did not choose.** The conditioning sweep verified a finite-difference cell (6490) the lab never published, at a different rank band, and got 0.0211% — tighter than any published cell. A wrong indexing, a stale gradient file, or a lucky triple would not survive that.
- **Cold reproduction of the control, not just the treatment.** The env-off regression was re-run cold by the sweep, on a fresh case copy: reason, iteration count, and all 13 residual digits matched independently of anything the original session wrote.
- **Recomputation from raw vectors, trusting no staged intermediate.** The robustness sweep recomputed both of Table 6’s cross-residuals with its own NumPy from the dumped vectors ($1.354841\text{e-}02$ and $1.047165\text{e+}00$), rebuilt the `np4→np1` permutation from the raw addressing files — its reconstruction equals the staged input under `np.array_equal`, for both levers — verified the sign convention rather than assuming it, and confirmed *in code* that the instrument’s operator is the matrix-free AD product the KSP uses and never the assembled matrix dumped alongside it.
- **A byte-level audit of the “one word” claims, because they are load-bearing.** `diff` of the R6b case against the established source: one changed line, in `fvSchemes`, everything else `cmp-clean`. `diff -r` of the serial arm against the clean case it was built from: the single differing input file is `system/fvSchemes`, two lines; and its one-word control differs by exactly one word.
- **A free check the record never ran, found by attacking the record’s own logic.** The serial-limiter sweep judged the record’s step-stability test logically insufficient against a symmetric FD kink — correctly — and then closed the hole at zero cost by mining the one-sided slopes out of the FD legs’ own printed converged C_D values (Table 9), first validating the reconstruction by rebuilding the printed central-difference vector from the same values to 4.7×10^{-9} .

- **The ledger is audited to the second.** Billed walls were reconciled against log mtimes and internal timings — e.g. five offline runs plus a 600 s kill summing to 693.04 s against 693 billed; a memory trace whose first and last samples bracket 816 s against 819 billed; nine finite-difference runs whose per-run walls sum to *exactly* 642; and 21 robustness-campaign lines re-summing to exactly 128.28 with all 21 per-line `wall×cpus` arithmetic checks passing.

8.2 What the sweeps actually found

They did not rubber-stamp. Across the six sweeps the claims survived on every axis — one of them “CONFIRMED WITH A NAMED ERRATUM” — and the sweeps still returned a defect list, which is printed here rather than repaired quietly. The mechanism sweep found five bookkeeping and provenance defects; the phenomenon sweep found two documentation defects and three wording over-reads; the rotation and conditioning sweeps found no defect and two presentation caveats between them; the robustness sweep found one factual error and named three protocol gaps; the serial-limiter sweep found no error, named two, and ran the check whose absence was one of them. The material ones appear in the body of this document; the consolidated list:

8.3 Two audits larger than a sweep — one of which was itself wrong

A supervisor sweep re-derives one claim. Twice in this arc a single finding was large enough to force a sweep of the *archive*, and both are reported with their negative results because that is what an audit is for. One of the two then produced a false positive of its own, which is reported here at more length than its successes.

The dead-lever audit (`DEAD_LEVER_AUDIT_2026-08-08.md`, 946E4A26) ran the D4 question against every conclusion-bearing record the lab holds: each sentence leaning on a named solver option, and each such option hunted in the archived *runtime* logs rather than the input dictionaries. Zero solver cost, read-only. Result: about 125 distinct lever/conclusion pairs **verified** against a quoted log line, 16 reported **unverifiable-from-logs** (the logs are gone or never printed the banner — and per the charter those conclusions now carry that caveat on their faces), and four **found dead**. Only one of the four reopened a conclusion, and it is D4 itself. The headline is a null: *no new conclusion-reopening dead lever was found anywhere in the archive*. That null is worth more than the finding that prompted it, and it is the reason D4 is reported as one specimen rather than as a pattern in this lab’s work.

One of those four found-dead verdicts was itself manufactured, and it is the more instructive half of the audit. On 2026-08-10 a container-side re-read refuted the entry recording that `DASimpleFoam` silently ignores consistent `yes: SIMPLEC` *is* implemented, in the primal at `pEqnSimple.H:27` and in the adjoint at `DAResidualSimpleFoam.C:189`, both carrying the textbook `rAtU = 1/(1/rAU - UEqn.H1())` form, in both images. The original search scanned `DASimpleFoam.C` and `DASolver.C` and found only two unrelated comments — **the file list, not the search string, was the defect**: the equations live in a `.H` include and in the `DAResidual` class. The blast radius is real and is recorded rather than absorbed: that false deadness had already been adopted as the stated cause of a 22.05% cross-code discrepancy on another case in this lab, and that cause is now retracted (7CA80F8D) — the discrepancy stands and is unexplained again, which is worse than never having explained it, because the search for the real cause stopped for weeks. A false **found-dead** is not a missing finding; it is an invented one, and it propagates. D4’s own verdict was re-verified against that standard, with its search space printed on its face (Section 7), and the audit’s own record now carries the split: 35A91E0E, one confirmed with provenance, one refuted.

The warm-start audit (`WARMSTART_AUDIT.md`, 9BB948B2) did the same for D5’s state hazard across nine conclusion-bearing records, also at zero solver cost, and is described in Section 7. It is the one audit here that began by invalidating its own priced instrument — the “warm primals are an order of magnitude faster” signature does not exist on this case — and proceeded on a discriminator it validated instead.

found	disposition
The cross-residual table's numbers appear in no log; the instrument carried a sign error and every arm's log prints a degenerate <code>ratio=2.000000e+00</code>	Stated in full, Section 2.4; values are exact offline corrections and reproduce three ways including a fresh serial run. <i>The instrument itself was repaired 2026-08-07</i> — and reproduces the floor to every digit with no correction at all, which makes the repair a fourth reproduction
The instrument was left unrepaired after the sweep said to repair it, and was then reused twice with two same-family filename conventions and nothing declaring which	Filed by a family supervision pass as standing and high-severity; fixed at both sites with a convention banner and a distinct post-fix output prefix. No published number moves
The committed sub-PC diff carried absolute headers into a directory that no longer exists, so <code>git apply</code> failed on the file this document cites as its reproduction diff	Regenerated with repo-relative headers, <code>git apply -check</code> PASS, applied block verified byte-identical to the deployed image
The either-copy confound bound silently used the assembled matrix — the instrument the same record discredits	Disclosed here; immaterial by a 30× margin even when inflated by the full assembled-vs-AD discrepancy
The cited coloring-crash log no longer exists (the runner truncates per attempt; three attempts shared a tag)	Claim re-established statically in the shipped container, by code rather than by the cited log
The residual-function-identity loophole was left unacknowledged	Closed from the prior record's per-run FD column, and now stated in the body as its own subsection
The case's own ladder record still asserted the superseded verdict, in both directions	Filed as a lesson (L-32); satellites had moved, the primary record had not
A misprinted relative spread (1.7e-06 for 1.7e-05) and an over-broad “any group” invariance sentence (one noise-floor component differs by 11.64%)	Corrected; neither touches a number in any claim
A cost figure of 12.63 core-min that should read 12.80, repeated in a commit message	Corrected in place, with the arithmetic shown
Two headline numbers quoted in different units in a one-line summary; an “improves 34–115x” claim that passes over a fourth row degrading 0.389% to 0.505%	Both carried into this document with the footnote attached
Provenance stamps missing from the headline sweep's logs	Mitigated by a stamped independent re-run of one cell and a stamped pair reproducing the same two numbers
The established arm's adjoint Krylov count was written 590 in six places; 590 belongs to a different arm (<code>simple</code> 2x2x1, 1.40% error), and fifteen logs of the established arm read 719	Corrected at every site, here and in the source record. The correction enlarges the claim: the conditioning collapse is 17.5x, not the 14.4x reported
The R5 verdict bands were registered in absolute multiples of $\ b\ $ and mis-calibrated by ~ 5 orders against the actual $n_p=1$ floors; by the registered wording alone, the lever that <i>hides</i> the defect counts as a collapse	Printed with the split stated (Section 2.10): the substance rests on a floor-ratio criterion that was not a registered band, and it runs against the lab's own prior framing. Future cross-residual bands go in units of the configuration's own floor
No state-override control was run for either R5 lever, so the linearization-state confound is bounded only indirectly	Named in the body with both bounds; the ~ 0.4 core-min control is unbought and is on the open list
The serial arm's step-stability check cannot, by itself, exclude a symmetric FD kink — and the closing check was already free inside the record's own logs	Run by the sweep and printed as Table 9; adopted as the registered closing move for this symptom class
The “no acquisition” verdict is scored on an instrument adopted after the arm's numbers were in hand	Labelled post-hoc in the body in so many words, per the sweep, rather than merely caveated

8.4 Pre-registration

Predictions are committed to the repository *before* the runs that test them, and scored exactly as written.

- The cross-residual discriminator was proposed with the prediction “the cross residual will be large.” **HELD**: 3.3×10^2 against a 1.1×10^{-4} floor.
- The coloring-off discriminator was pre-registered with “the gradient will not change, because coloring builds only the preconditioner.” **NOT SCORED** — the discriminator proved unrunnable, so the prediction never met its measurement. It is scored NOT SCORED rather than quietly dropped or retroactively counted.
- The rotation root-cause session committed seven falsification conditions (P1–P7) before any diagnostic ran, including two — “evalMode=exact must *not* help” and “matrix reordering must not move the result at all” — whose failure would have refuted the mechanism. P5 and P6 both held; P7’s designed control came out only partially clean and is reported at its real strength, with the genuinely clean instance found elsewhere.
- The reach matrix (Section 2) registered seven arms with numeric bands and an explicit decision rule for refutation, at 99F5D41D, before any arm ran.
- The robustness campaign registered nine arms across a pre-registration and four amendments, every amendment committed *seconds* before its arms launched — the sweep reconciled each commit timestamp against ledger completion minus wall (17:14:47Z before a 17:15:08 start, and three more of the same shape) and found the ordering intact every time. Its scoring runs harsh: **four NOT HELD and two NOT SCORED**, including the arm whose failure named the carrier (R3a: registered $\geq 4\%$, measured 0.228%) and one whose “direction held, magnitude under the band” is reported as NOT HELD rather than upgraded. The load-bearing clean clause, R6b, was registered together with the result that would have refuted the whole hypothesis.
- The acquisition arm registered an escape clause for a dirty control, and the clause fired: the np=1 control read 92.8%, so the arm is reported NOT SCORED against its own bands rather than re-read into a result. Its contingent arms were authorized only on a fire that did not happen, and were correctly not run.
- The M6 arc of Section 7 ran five pre-registrations in one day (12D3A7A3, 054D10B2, 13C244CD, 31EEDE16, 4AEF4A2F), each committed before the launch it governs, two of them carrying addenda committed before the attempt *they* govern. Its scoring is the point: the registered *not evaluable* branch fired **three times** — twice on arms that never reached an adjoint, once on a finite-difference step below its own measured noise floor — and each is reported as not evaluable with its blocker named, rather than read into the verdict the campaign wanted. The negative control was registered *with its falsifier* (“if the control converges, the token story is wrong and the finding reverts to open”), and the FD arm’s pass rule (≥ 2 evaluable components, all evaluable ones inside 5%) was fixed before the components were.
- Rung 2 (22C8E988) registered its own mapping and then hit the case the mapping got wrong: a -3 at the iteration cap, which its letter would have graded DIVERGED. The addendum (2B3E6517) was committed *before* attempt 2 and staked the refusal on a prediction — a raised budget alone finishes it — which the ten-digit residual match then confirmed. A pre-registration that has to be corrected mid-arm, in writing and in advance, is working as designed.
- The lever triage (CFD6B5FF) named this lab’s own standing prior — R5’s collapse-on-strengthening — in the filing, as the thing the arm was really testing, and pre-declared

the outcome that would have *confirmed* it (any collapse under a strengthened PC) as a reportable finding rather than a failure. The prior lost. Registering the outcome that would have vindicated the record is the only reason its loss counts for anything.

- The diagnosis campaign of Section 7.10 wrote its decision rule — five candidate diagnoses, each mapped to the preconditioner it would imply — *before the diagnosis existed* (A9BB202D), and then reported that the rule was incomplete: the measurement returned a sixth signature it had not anticipated. The lever finally chosen was therefore selected after the diagnosis was in hand, and the record labels that branch one notch weaker than its pre-registered ideal in so many words rather than presenting it as pre-committed. Its falsifier then fired.
- The fill-1 arm (D4C706E8) pre-registered the clause that mattered most for an attempt with no theory behind it: that a convergence there would be a knob nobody could explain and *must not* be allowed to retro-justify the diagnosis. It stalled, so the clause was never tested — and it is recorded as having been in place beforehand regardless, which is the only time such a clause can be worth anything.
- The sharpest instance in this arc is an agent refuting itself. The fill-level arm generated the non-normality hypothesis as a side effect of a result it did not expect, named it in advance as the outcome it should be most willing to report, pre-registered the measurement and its $3\times$ decision bar (7783F603), and then measured the stalling rung as *less* non-normal than the converging one — killing its own hypothesis at zero solver cost and adding the tenth row to the table instead of the explanation it was hoping for.
- The two arcs added last (Sections 7.12 and 7.13) both pre-registered the outcome that would have cost this document something. The patch’s Gate A (0FCC98A0) wrote down, before the build existed, that a *moved* iteration count would mean the override is load-bearing and **this report’s own recommended fix is wrong and must be re-graded before anyone upstream acts on it**, with fix (c) named as its replacement and stage 2 abandoned. Stage 2 (A15B839D) then fixed its interpretation before any number existed — *a failure is the eleventh elimination; a success is one reachable remedy working and not vindication of anything* — and, in the same filing, declined two arms with reasons on the record, one of them (`fieldsplit`) because the campaign’s own diagnosis had already refuted its precondition. It also carried Gate B’s lesson forward as a binding: every arm changing the Krylov type must restate the restart, or the restart’s failure gets read as the method’s.
- One prediction in this arc was made by an agent with no stake in it: an independent reader, before rung 2’s attempt 2 returned, projected convergence near iteration 1,250 from the residual’s decades-per-hundred slope. Measured 1,171 — correct in direction, conservative by 7%, and logged in the calibration cohort as a hit.

8.5 Cost, stated including the overruns

Every run is billed at full cap occupancy for its whole wall clock, including both kills and including the refined-mesh arm that returned a failure instead of a number. The two most recent sweeps cost *nothing*: both re-derived their verdicts from artifacts already on disk, and the check that decided the serial finding (Table 9) came out of logs the record had already written and not read.

The table’s last two lines are the newest and follow the same rule. Stage 1 ran $1.4\times$ its estimate because Gate B was priced as a seconds-long one-iteration `-ksp_view` check and run as a full solve instead — which is where the `restart=30` finding and the stale-echo evidence came from, both of which go upstream, and it is still recorded as an overrun rather than folded into the estimate. Stage 2’s $2.4\times$ is arm 2 alone: both arms were priced off the ILU baseline’s per-iteration cost, and GAMG’s hierarchy setup bears no relation to it. The correction is written into the record for the next AMG arm — price the setup phase separately, and cap on wall rather than on iterations when the per-iteration cost is unknown.

work	spent (core-min)	budget
Decomposition discriminators (8 container runs)	56.77	45
— of which the five arms that decided the item	12.80	
— of which three attempts at the coloring-off arm, all ending in a measured failure mode rather than a number	43.97	
Mechanism supervisor sweep (one 9 s container run; everything else arithmetic on dumps)	0.30	
Phenomenon supervisor sweep (one np=2 check_ totals)	2.90	~30
Conditioning unblock (7 entries: offline ladder, regression control, two converged solves, hump attempt, 9-point FD sweep)	185.0	240
Conditioning supervisor sweep	18.3	22
Robustness campaign (21 ledger lines: two refined-mesh arms, four scheme knobs, the conformal pair, two levers at three container tasks each)	128.28	150
— of which one arm that produced no gradient at all (refined mesh, limiter present, adjoint diverged), charged in full	33.60	
Acquisition arm and its two separating arms (4 runs, all rc=0)	15.16	30
Robustness supervisor sweep (offline: raw-vector recomputation, byte diffs, two cut re-classifications)	0.00	
Serial-limiter supervisor sweep (offline: log mining, one-sided-slope reconstruction)	0.00	
<i>The M6 arc of Section 7, 2026-08-08 to 2026-08-10, eight pre-registrations:</i>		
vcoarse arm (mesh gate, no iteration run)	0.53	30
sub-LU + PC re-file, three attempts (OOM, host-floor kill)	127.0	30
the one-token arm — both adjoints converged	7.33	~28
negative control (record configuration, reproduced)	8.53	~30
FD verification, two attempts	29.5	~30
dead-lever audit and warm-start audit (log forensics only)	0.00	
rung 2 (42,120 cells): two convergence attempts and the FD arm	99.5	~47
lever triage, three staged arms on the working configuration	22.7	35–45
rung 3 (79,560 cells): stage-0 transfer test 15.9 + stagnation arm 95.1	111.0	170–190
restart challenge at rung 3 (5× window, 1.647×)	47.7	66–82
Saad diagnosis campaign and its chosen arm (elimination table)	61.6	~35
fill-1 engineering arm (1.859×, the κ surprise)	33.9	~15
non-normality diagnostic (offline; both operators already on disk)	0.00	25
option-channel patch, stage 1: rebuild plus Gates A and B	21.5	~15
stage 2, the previously-unreachable class: lgmres 19.4 + gang 105.0	124.4	~52

Table 31: Solver cost, with every overrun named and its cause named beside it. The first — 26% on the discriminators — is confined entirely to the coloring-off arm, and the record volunteers that the second forced attempt should not have been launched after the first showed unbounded memory growth rather than a slow finish. The second is larger and worse-looking: the M6 sub-LU re-file spent $4.25\times$ its budget, inside a pre-registered commitment to run to its reason codes, and was stopped by a host-memory floor rather than by its own budget. It bought a measured infeasibility and no reason code. The arm that answered the question cost 7.33 core-min. Rung 2 then overran by $2.1\times$ for a reason worth more than the overrun: it was priced off cell count, and iteration count on this family scales at cells^{1.5–1.7}. That mis-pricing is corrected in the next rung’s filing rather than absorbed. Rung 3 then came in *under* budget for the only good reason — a pre-registered gate refused to spend the conditional FD arm’s 60–70 core-min on a stage that had not converged — and its 15.9-core-min stage-0 test is the cheapest line in this table relative to what it prevented: a 95-core-min arm launched on a lever now known to collapse at that size.

8.6 The lessons this arc produced

The method itself is on the record, as numbered lessons in `LESSONS.md`:

- **L-29.** A guard that replaces a removable singularity with a constant differentiates to a hard zero — at exactly the point every gradient evaluation uses. Audit every such guard on a differentiated path; require the guarded branch to carry the analytic limit, or reparameterise so no guard is needed. And when *testing* derivative code, evaluate at the degenerate/baseline point deliberately and seed with the real objective direction.
- **L-30.** A gate whose threshold lies between two constraint values reports the constraint, not the quantity. Before gating on a reported extremum, find out whether the tool was *told* that number.
- **L-31.** A verification instrument must itself be verified at the component it condemns. When an analytic derivative disagrees with a finite difference, re-measure the finite difference by an independent path — with a neighbouring component as a control — before attributing the gap to the derivative.
- **L-32.** When a verdict moves, the case’s own record must move first; satellites are not the record.
- **L-33.** A correction trained inside a gated regime rebuilds the gate’s own failure at the regime’s floor.
- **L-34.** “Unreachable at runtime” is a claim about the deployed object, not about one call site. Deploy a probe that prints the final object state (for PETSc, PCView on the sub-PC after setup) before declaring a configuration layer unreachable.
- **L-35.** *A converged Krylov solve certifies the operator it was given, not the operator you meant.* When an iterative solve converges cleanly and the downstream answer is still wrong, stop tuning the solve and test the operator: apply the converged solution under an independently evaluated instance of the operator — serial where the suspect is parallel, assembled where the suspect is matrix-free — and compare the residual to that instance’s own floor. It is a mat-vec, not a solve: seconds, not core-minutes. Two footnotes: map vectors across decompositions with the integer addressing files (sign on flipped faces), never coordinate matching, and validate the map on the primal state first; and diff operator *actions* on mapped vectors, never operator *entries*.
- **L-36.** *A gradient check certifies a contraction, not an operator* — and the two do not scale together. Passing `check_totals` at one configuration certifies the contraction of the operator’s error with that one objective’s adjoint direction: almost no information about the operator, none about other objectives, other design variables, or other decompositions. The converse holds too: a large measured operator defect does not imply the published gradients are badly wrong. Never extrapolate a passing gradient check across configurations; when an operator defect is suspected, measure the operator.
- **L-37.** When a differentiated code is wrong in parallel, *suspect a recorded branch first* — limiters, switching or mixed boundary conditions, guards, `min/max/abs` — and one-knob them before theorizing about halo exchange in general, because a branch is removable by a one-line dictionary edit and is therefore the cheapest discriminator available. And any lever that turns a gradient error off must be re-verified at *operator* level before the configuration is described as unaffected: a lever can fix the operator or merely rotate the contraction away from the damage, and the gradient number cannot tell them apart. State which one you measured.

- **L-38.** *An invariance check can pass because both sides are wrong together.* Decomposition-, ordering-, rank-count- and restart-invariance detect *inconsistency* between two evaluations of the same tape; a tape wrong identically in both sails through all of them. Never promote an invariance pass to a correctness verdict unless one side is anchored to an external reference. And the converse of L-37’s trigger: a recorded branch can corrupt the reverse sweep in *serial*, with the parallel machinery innocent — so when a serial gradient check fails against a step-stable finite difference, put the tape’s branches at the top of the suspect list there too, before FD-quality theories.
- **L-40.** *The switch you set is not the switch that ran.* Any option, flag, model choice or lever that is load-bearing for a conclusion must have its activity confirmed in the *runtime log* — not its presence in the input dictionary — before the conclusion ships. “Configured” and “active” are different claims and only the second is evidence: a dictionary entry can be dead code for the solver that executed it, and the settings echo will print it back either way. Where the archived log cannot settle it, the conclusion ships marked *unverifiable-from-logs* rather than quietly assumed. The companion rule from the same arc: a mesh entering an archive, a pre-registration or a ladder rung carries its `checkMesh` record from creation — born clean or it does not enter.
- **L-43, second corollary.** *A found-dead verdict is a claim about implementation, and implementation hides in include files.* Where L-40 says a lever is evidence only when the log proves it ran, this is the failure in the opposite direction and it does more damage: a false “the code ignores this” is not a missing finding but an invented one, and it propagates into whatever it was used to explain. A deadness claim about C++ requires the build’s view rather than a file’s — the whole source tree, `.H` includes and derived classes included — and any verdict of the form “the code ignores X” is downgraded to “not found in the files searched, which were ...” unless the search space is stated. A negative claim without its search space is not a finding.

A Reproduction appendix

A.1 Environments

image / package	identity
dafoam/opt-packages:latest IDWarp 2.6.2 (inside the above)	DAFoam 5.0.0, OpenFOAM v2506, PETSc 3.15.5. The stock toolchain; all phenomenon and mechanism measurements digest <code>sha256:9d45679d55fd47f5ca7afd99cabb86c7c2729cf2acf34c438eb33af5290f07fc</code> ; installed <code>idwarp/*.py</code> md5-identical to tag v2.6.2, tag commit 647FD8FC
patched IDWarp	scratch clone at <code>/home/ubuntu/certonomous-runs/W5-patch/idwarp</code> ; <code>git diff</code> byte-identical to <code>rotation_branch/idwarp_v2.6.2_degenerate_branch_fix.patch</code> (2 files, +44 lines). Mounted, never installed in place
dafoam-subpclu:v1	local-only <code>docker</code> commit of the stock image with one hunk at <code>DALinearEqn.C:266</code> rebuilt in all three AD modes. Nothing pushed. All three M6 arms of Section 7 ran on this image with <code>DAFOAM_SUBPC_TYPE unset</code> — and the negative control reproduced, to every printed digit, a record measured on the stock image: a default-off regression on a second case

A.2 Switches and stamps

```

# Which IDWarp is actually loaded -- stamped into every log by the driver.
IDWARP_IMPORTED_FROM: /patch/idwarp/idwarp/__init__.py      # patched arm
IDWARP_IMPORTED_FROM: <container site-packages>              # stock arm

# Sub-block preconditioner switch (patched image only; default off).
docker run -e DAFOAM_SUBPC_TYPE=lu ... dafoam-subpclu:v1
# Log line emitted on block 0 when active:
#   DAFOAM_SUBPC_TYPE=lu: ASM sub-block PC set to complete LU

# The IDWarp workaround, for anyone who needs a consistent gradient today
# (mesh-quality price NOT measured -- see Section 5.7):
meshOptions = {..., "useRotations": False, ...}

```

A.3 Varying the decomposition without falling into the trap

Editing `system/decomposeParDict` directly is silently ineffective: `pyDAFoam` regenerates it. Set the `daOption` instead, and *read the decomposition back out of the log*:

```

daOptions = {
  # ...
  "decomposeParDict": {"method": "simple", "simpleCoeffs": {"n": [4, 1, 1]}},
  # default when unset is {"method": "scotch"} -- pyDAFoam.py:590-591
}

```

```

# Verify the edit took effect -- this line is the check that catches the trap:
grep "Decomposition method" run.log
#   Decomposition method scotch [4] (region region0)
#   Decomposition method simple [4] (region region0)

```

A.4 The cross-residual protocol

Pure `DAFoam` public API; no rebuild required. This is the instrument any candidate fix must satisfy.

```

# 1. Per decomposition: run the primal + adjoint, then dump
#   psi   = DAFoamSolver.psi
#   state = the converged primal state vector
#   b     = dFdW (the adjoint RHS)
# with, in daOptions:
#   "writeJacobians": ["adjointIndexing"]
# which emits AdjointIndexing_<rank>_of_<n>.txt (indices + cell/face centres).

# 2. Build the exact integer permutation to serial ordering from
#   processor*/constant/polyMesh/cellProcAddressing
#   processor*/constant/polyMesh/faceProcAddressing (sign => flipped face)
# NEVER by coordinate matching. Validate it on the PRIMAL state first:
# duplicated processor-face phi copies must agree to machine precision
# (measured here: 2.7e-15). The AdjointIndexing centres give a second,
# exact channel: max |dxyz| must be 0.0.

# 3. Evaluate the true residual under the SERIAL operator, matrix-free:
#   r = solverAD.calcJacTVecProduct(state -> residual) applied to mapped psi
# SIGN CONVENTION: OpenMDAO seeds -dF/dW, so the system is A^T psi = -b.
# An instrument computing (A^T psi - b) prints a degenerate ratio of exactly
# 2.0 for its own-operator control -- that is the tell that the sign is wrong.
# Correct offline as res + 2b, or -- better, and done here on 2026-08-07 --
# fix the instrument: both reused tasks now compute (A^T psi + b), every log
# line prints a convention banner, and the fixed task writes a distinct
# w4xf_res_*.npy prefix so pre-fix dumps (needing +2b) cannot be conflated

```

```
# with true residuals. The repaired tool reproduces the np=1 floor
# 1.140697e-04 with NO offline correction, to every digit.

# 4. Judge against the SERIAL operator's own floor, not against zero:
# np=1 control ..... 1.14e-04 of ||b|| <- the floor
# np=4 simple 4x1x1 .... 9.37e-02
# np=4 scotch ..... 3.288e+02 <- the defect
```

Lab scripts implementing this: W4-a4-discriminators/runScript_w4.py (tasks w4_dump, w4_crossres, w4_crossres2), build_maps.py, analyze_mats.py.

A.5 The two one-word reproducers

Both defects of the branch class in DAfoam are switched on and off by a single token in system/fvSchemes. Neither needs a rebuild, a patch, or a lab artifact. The token is the same in both.

```
// system/fvSchemes -- the switch, both cases
gradSchemes
{
    default Gauss linear; // unlimited
    limited cellLimited Gauss linear 1; // the branch: min/max over the stencil
}
divSchemes
{
    div(phi,U) bounded Gauss linearUpwind limited; // defect ON
// div(phi,U) bounded Gauss linearUpwind default; // defect OFF -- one word
}
// Same scheme family, same order, the same gradient-correction term still evaluated
// and still needing its halo exchange. Only the selection leaves the tape.
```

(A) Parallel — the decomposition-dependent adjoint. A4, 2,777-cell snappy-refined Ahmed body, freestreamVelocity farfield, np=4 shipped-default scotch, check_totals at step 1e-3 central:

```
limited: dCD/dshape 2.2086e-01 vs own FD 2.4258e-01 -> 8.95 % KSP 719
default: dCD/dshape 2.0572e-01 vs own FD 2.0399e-01 -> 0.849% KSP 41
# and at operator level, by the cross-residual protocol above:
limited: ||A^T psi + b|| = 3.288e+02 x ||b|| (serial floor 1.14e-04)
default: ||A^T psi + b|| = 1.355e-02 x ||b|| (serial floor 2.00e-06)
# Control, same discretization at np=1: 0.016 % -- the ~0.8 % that survives
# is an FD-column shift; the analytic reads 2.0572e-01 at BOTH decompositions.
```

(B) Serial — one process, no MPI in the loop. A1, 4,032-cell structured NACA0012 with the freestreamVelocity farfield installed (inout patch; freestreamValue uniform (9.959800351 0.895754972 0) at the case's own α_0), primalMinResTol 1e-8, np=1:

```
limited: dCD/dshape 1.066449e-01 vs own FD 6.194868e-02 -> 92.8 % (CL 7.86%)
default: dCD/dshape 6.516374e-02 vs own FD 6.522800e-02 -> 0.121% (CL 0.018%)
# One of the eight components (index 6) is sign-flipped: AN -0.02591213
# against FD +0.00849673. Krylov 95/96 vs 32. No decomposition anywhere.
```

The closing check this symptom class needs, and it is free. Before attributing a serial analytic-vs-FD gap to a limiter, rule out the finite difference — and rule it out *properly*, because a symmetric kink at the base point is invariant under a change of central-difference step and will survive a two-step check. The FD legs have already printed what is needed:

```

# Each central-difference leg prints its own converged objective. Mine them:
#   forward slope = (J(+h) - J(0)) / h
#   backward slope = (J(0) - J(-h)) / h
# 1. Validate the reconstruction first: rebuilding the framework's printed
#   central-difference vector from the same mined values must agree (here:
#   4.7e-09 absolute, at both h = 1e-3 and 3e-3).
# 2. A kink splits the two one-sided slopes apart. A smooth function does not.
# 3. The verdict is whether the ANALYTIC lies inside [backward, forward].
#   Measured here on the sign-flipped component: forward +0.009660,
#   backward +0.007334 (27.4% asymmetry -- real limiter chatter), all four
#   secants across both steps in [+0.0053, +0.0143], analytic -0.025912.
#   Outside the bracket, wrong side of zero, 3x the magnitude: no branch
#   selection available to the FD produces that. The adjoint is convicted.

```

A.6 The third one-token reproducer, and how to prove it ran

Section 7's switch is one integer in `daOptions`, on a transonic case run with `DARhoSimpleCFoam`. It needs no patch, no rebuild and no image:

```

daOptions = {
  # ...
  "transonicPCOption": 2, # accepted, echoed, and DEAD for this solver
#  "transonicPCOption": 1, # the live branch: DAResidualRhoSimpleCFoam.C:173
}
# Only == 1 is implemented for DARhoSimpleCFoam; == 2 is DAResidualTurboFoam.C:176.
# pyDAFoam.py:396 initialises it to -1 and documents it for "transonic solvers
# such as DARhoSimpleCFoam". Nothing validates the value at read time.

```

```

# The echo below is NOT proof the branch ran -- it is a dump of the dictionary,
# and it prints the dead value exactly as it prints the live one:
grep "transonicPCOption" run.log
#   transonicPCOption 2;      <- reads identically whether or not it does anything

# What the option DOES is drop fum::div(phid,p) from the PC pressure equation.
# On a case that fails, the discriminating evidence is the reason code itself:
grep "PetcConvergedReason" run.log
#   Total iterations: 400. PetcConvergedReason: -5      (tpc 2, both solves)
#   Total iterations: 368. PetcConvergedReason:  2      (tpc 1, CD)
#   Total iterations: 383. PetcConvergedReason:  2      (tpc 1, CL)

# CAUTION, two traps in this exact workflow (both measured, D4/D5 section):
# 1. A -5 run prints "Residual tolerance satisfied, solution finished!" -- the
#   denormal-collapsed residual passes the relative-ratio gate. Read the reason
#   code, never the sentence.
# 2. Re-running in the same case directory silently warm-starts: pyDAFoam wrote
#   the previous run's end state into time 0. Verify the start state in-log --
#   cold-from-uniform prints its own signature and warm prints ~55x lower:
grep -m1 "Time step continuity errors" run.log
#   ... sum local = 0.5969274433533561      <- cold, this case, six runs, 16 digits
#   ... sum local = 0.0107                  <- warm: a previous run's converged fields

```

A.7 Reproducing the IDWarp defect in isolation

No DAFoam, no OpenFOAM, no pyGeo, no CFD — IDWarp's own test mesh, options, displacement and verification routine:

```

# input_files/symm_block.cgns, md5 0d5a0411cbaa00f6afd88a9430a7f634,
# from upstream's own input_files/get-input-files.sh
mesh.verifyWarpDeriv(dXv_warp, solverVec=False, dofStart=0, dofEnd=5)

```

```
# Rotations ON : DOF 0 AD 2.87055669475 vs FD -115.875789153 -> 210.16%
# Rotations OFF: DOF 0 AD 2.87055669475 vs FD 2.87055639160 -> -1.06e-05%
# ^ the AD column is BIT-IDENTICAL between the two runs: the proof that
# the rotation term was never in the derivative.
# Adversarial check on the FD: stable to 6 significant figures over
# h = 1e-4, 1e-5, 1e-6; roundoff only at 1e-8. The FD is converged.
```

A.8 Rebuilding the sub-LU image

```
docker run -d --name dafoam-build dafoam/opt-packages:latest sleep infinity
# Apply the committed diff (one hunk at DALinearEqn.C:266, env-gated). Since
# 2026-08-07 the diff carries repo-relative a/ b/ headers and is mechanically
# applicable -- run this from the directory containing src/, in-container
# /home/dafoamuser/dafoam/repos/dafoam:
git apply --check demo-output/website/dafoam/subpclu_patch/DALinearEqn_subpclu.patch
git apply demo-output/website/dafoam/subpclu_patch/DALinearEqn_subpclu.patch
# then, INSIDE the container, for each of the three AD modes:
source /home/dafoamuser/dafoam/loadDAFoam.sh # original
cd repos/dafoam && wmakeLnInclude src/adjoint && (cd src/adjoint && wmake -j 8)
# ADR and ADF: sed WM_AD_MODE in OpenFOAM-AD/etc/bashrc, re-source, wmake again
docker commit dafoam-build dafoam-subpclu:v1
# Caution: 'set -e' before sourcing the OpenFOAM environment kills the shell.
# Source first. All three libDASolver*.so relink in seconds.
```

A.9 Artifact index

All paths are in the Certonomous repository unless marked as a run directory. Run directories live outside the repository, on the measurement host.

artifact	what it holds
Repository: demo-output/website/dafoam/	
VERIFICATION_A4_decomposition_supervisor_sweep.md (27D25762)	The verified phenomenon table, five verification axes, the hanging-node refutation
DISCRIMINATORS_A4_decomposition_mechanism.md (+ 2026-08-04 amendments)	The four discriminators, the cross-residual, the localization, the coloring-off finding
VERIFICATION_A4_mechanism_supervisor_sweep.md (8E0A08BC)	Adversarial sweep of the mechanism: independent map, independent scripts, fresh serial run, five defects found
UPSTREAM_BUG_REPORT_decomposition_adjoint.md	The prepared, unfilled report (D1). It now carries the papers-protocol correction of Section 4 in its own header, the reach addendum, a 2026-08-07 robustness addendum (“the trigger is a recorded BRANCH ... and one previously ‘clean’ configuration is wrong at operator level”), and a 2026-08-07 acquisition addendum carrying D1s. Still NOT FILED
UPSTREAM_BUG_REPORT_mesh_warpDeriv.md	The prepared, unfilled report (D2)
DEFECT_ROBUSTNESS_mesh_and_setup.md (pre-registration 25F52868, results 49631723, R7 85DFBE68)	The robustness and acquisition campaigns: the scheme ladder, the two levers at operator level, the conformal pair, the refinement arms, the seven-case survey, the serial variant, and every registered prediction scored as written

artifact	what it holds
VERIFICATION_defect_robustness_supervisor_sweep.md (8BD47B35)	Adversarial sweep of the robustness campaign: cross-residuals recomputed from raw vectors, map rebuilt bit-for-bit, the 590/719 erratum, three named protocol gaps
VERIFICATION_A1_serial_limiter_supervisor_sweep.md (0858E640)	Adversarial sweep of the serial variant: byte-level config audit, independent re-extraction, and the one-sided-slope reconstruction that closed the FD-kink hypothesis at zero cost
SUPERVISOR_FAMILY_REVIEW_2026-08-07.md	The instrument-level review the number sweeps cannot do: unit assumptions, silent fallbacks, reuse traps. Sources the sign-convention repair (A-1) and the applicable sub-PC diff (B-2), each with its OUTCOME line
DAFOAM_PAPERS_VERIFICATION_PROTOCOLS.md (BF6AC53B)	Three method papers read in full; the protocol table of Section 4
LIAISON_NOVELTY_SWEEP_decomposition_defect.md (3C74DC03)	63 searches, 10 venues, the query log, the near-misses, the PETSc precedent
ROOTCAUSE_getRotationMatrix3d.md, PATCH_getRotationMatrix3d.md	The line, the mechanism, the derivation-before-code, the acceptance results
VERIFICATION_rotation_patch_supervisor_sweep.md (6782D33A)	Adversarial sweep of D2: math re-derived, own driver, own reproduction
ladder-b/W4_ADJOINT_PC_UNBLOCK.md	The conditioning wall, the offline ladder, the sub-LU unblock, the FD gate
VERIFICATION_cbfs_unblock_supervisor_sweep.md (9C19CCC8)	Adversarial sweep of D3: image-vs-diff audit, recomputed FD table, new cell, cold regression
DEFECT_REACH_decomposition_cases.md (99F5D41D)	The pre-registered reach matrix, since scored and audited (sweep 192C970C)
subpclu_patch/DALinearEqn_subpclu.patch (1CD44C04, regenerated 9796F90F)	The one-hunk sub-PC source diff. Repo-relative headers since the regeneration; <code>git apply -check</code> passes from the DAFoam repo root
rotation_branch/idwarp_v2.6.2_degenerate_branch_fix.patch	The four-line derivative correction (2 files, +44 lines)
A3_SUBLU_RESULT.md	The M6 arc end to end, in the order it happened: the mesh-gate not-evaluable, the memory-bound sub-LU re-file, the converged one-token arm (11B90D25), the bit-for-bit negative control (551A7BA5), the FD table (E4280B79)
A3_SUBLU_PREREGISTRATION.md (12D3A7A3), A3_SUBLU_SWEEP_PREREGISTRATION.md (054D10B2), A3_TPC1_ARM_PREREGISTRATION.md (13C244CD), A3_TPC1_CONTROL_PREREGISTRATION.md (31EEDE16), A3_FD3_PREREGISTRATION.md (4AEF4A2F)	The five pre-registrations of Section 7, each committed before the launch it governs. §2 of the first is where <code>transonicPCOption: 2</code> was found dead

artifact	what it holds
A3_RUNG2_N28_RESULT.md (4E982B4A)	Rung 2: both solves converged, the three-component FD pass, the refused -3 proven a budget knob to ten digits, the measured iteration-scaling exponents, and the independent agent's three items
A3_TRIAGE_LEVERS_PREREGISTRATION.md (CFD6B5FF, results §8 at 3AC8257E)	The three levers on the working configuration, and the overturning of R5's collapse-on-strengthening prior
DEFECT_CANDIDATE_ksp_options_override.md (F29378D9; §8 added at 5E720034)	D6, filing-ready and unfiled: the twelve call sites, the commands a user would type with observed-vs-expected, the monitors-work trap, the five-minute reproducer, the three graded fixes, and — since §8 — the implemented fix (a) with both gate results, the restart-reset finding, and the revised recommendation of (a) plus an effective-value echo
A3_KSPOPTS_PATCH_PREREGISTRATION.md (results §6 at 0FCC98A0)	Stage 1: the patch stated exactly before it was written, the image discipline, and the two gates — Gate A's failure branch pre-declaring that this document's own recommended fix would be wrong and must be re-graded, Gate B's pass restoring <code>type: fgmbres</code> , and the stale- <code>printInfo</code> wart disclosed in §1 before it was measured in §6
kspopts_patch/DALinearEqn_kspopts.patch (3F631C4D)	The one-call relocation, 28 diff lines, archived beside the sub-PC patch
A3_STAGE2_UNREACHABLE_CLASS_PREREGISTRATION.md (A15B839D, results E1755100)	Stage 2: the three bindings written before any number — failure is the eleventh elimination, the Gate B restart lesson carried per arm, grading against the standing control — the two declined arms with their reasons, both readbacks quoted before grading, the rising-residual anomaly flagged rather than binned, and the verdict bounded to off-the-shelf members at documented defaults
A3_SAAD_DELIBERATE_CONDITIONING_PREREGISTRATION.md (A9BB202D, diagnosis DDA82819, outcome 368996C3)	The decision rule written before the diagnosis existed, the diagnosis that returned a signature the rule did not anticipate, the disclosed weakening of a post-diagnosis branch, the falsifier firing, and the elimination table
A3_NONNORMALITY_DIAGNOSTIC_PREREGISTRATION.md (7783F603, results 15B64E81)	The tenth elimination: the scale-invariant measurable chosen and its bar fixed before the number, the converging rung run as the control, both readings refuted, and the preconditioner-not-preconditioned-operator limitation stated up front and repeated at the verdict

artifact	what it holds
A3_RUNG3_RESTART_CHALLENGE_PREREGISTRATION.md, A3_RUNG3_FILL1_ENGINEERING_PREREGISTRATION.md (48B34CBF)	The two reachable engineering attempts on the ceiling, both graded against a $10\times$ bar fixed before the numbers: $1.647\times$ and $1.859\times$. The second carries the seven-decade condition-estimate surprise and the hypothesis it is not allowed to claim
A3_RUNG3_N52_PREREGISTRATION.md (3E585B07), A3_RUNG3_N52_RESULT.md	Rung 3 as filed — the certified member, the adopted lever with its mandatory transfer arm, the exponent's falsifiable prediction, the price up front — and as measured: the transfer test's collapse, the stagnation, the memory evidence that makes the verdict sayable, and the ceiling stated as a bracket
R5_ADJOINT_CONDITIONING.md, ADJOINT_MEMORY_ENVELOPE.md, PROOF.md §25.2	The M6 conditioning record as measured — each now carrying its dated dead-lever annotation (E9A651E9)
D3_VARIANT_COLD_RERUN_PREREGISTRATION.md §6 (8C9B06C7)	The 11 warm-flagged variant nulls, retired as superseded rather than rerun, with the reasoning stated at both sites
WARMSTART_AUDIT.md (9BB948B2)	D5: the nine-row verdict table, the invalidated signature, the $55\times$ discriminator
PROOF.md §25.5, §23–24, §25.2–25.3	The primary measurement record
LESSONS.md L-29 – L-40	The method-of-investigation thread
Run directories: /home/ubuntu/certonomous-runs/	
W4-a4-stepsweep/	The phenomenon table's per-cell logs
W4-a4-discriminators/	Per-arm dirs, dumped operators and vectors, <code>ledger.txt</code> , the analysis scripts
W4-a4-verify-audit/	The mechanism sweep's own staged case, own mapped vectors, <code>audit_crossres.log</code>
VERIFY-a4-np2-20260804/	The independent re-run of one phenomenon cell
W4-defect-reach/	The reach matrix's arms and <code>ledger.txt</code>
W4-defect-robustness/	The scheme ladder, the lever dumps and cross-residuals, the conformal and refined meshes, the A1 acquisition arms; <code>ledger.txt</code> (21 audited lines) and <code>ledger_r7.txt</code>
W5-idwarp-source/, W5-patch/, W5-rotation-branch/	IDWarp source clone, patched clone, and the A5 case copies
W4-adjoint-pc-unblock/	Offline petsc4py ladder, CBFS and hump runs, the 9-point FD sweep
W4-verify-sweep/	The conditioning sweep's isolated case copy and its own logs
A3-onera-m6-sweep-n15_21840/	The 21,840-cell M6 rung: the 2026-07-30 record (<code>run_opt5_onera_n15_21840.log</code>), the three sub-LU attempts, and the three arms of Section 7 — <code>tpc1_computetotals.log</code> , <code>ctrl1_computetotals.log</code> , <code>fd3_run{ _attempt1, }.log</code> — with their launchers, scripts and <code>.t0/.rc/.t1</code> ledgers

artifact	what it holds
A3-onera-m6-adjoint-vcoarse/	The born-broken rung: the mesh-gate abort, the standalone <code>checkMesh</code> reproduction, and the preserved pre-repair decomposition that carries the same 23 negative cells
A3-rung3-n52/, A3-stage0-n28-richardson/	Rung 3's stagnation arm (<code>rung3_stage1.log</code> , the flat 4,000-iteration residual, birth certificate and <code>log.checkMesh</code>) and the stage-0 transfer test whose -5 at iteration 200 voided the adopted lever
A3-gateA-regression/, A3-gateB-restoration/	The two gates on the option-channel patch: the bit-identical 368/383 regression log with its cold signature, and the <code>-ksp_view</code> restoration log where <code>type: fgmsres</code> sits eleven lines below the stale <code>Solver Type: gmsres</code> echo. Each carries its own <code>lever_echo.txt</code> recording the image tag and the exact <code>PETSC_OPTIONS</code> string
A3-stage2-lgmres/, A3-stage2-gang/	The eleventh and twelfth eliminations: both arms' full logs with the readbacks, the shared iteration-0 residual, the -3 at 400 and the -5 at 200, and their <code>lever_echo.txt</code> option strings
A3-rung2-n28-tpc1/	Rung 2's staged copy (the archived member left pristine): both attempt logs, the FD log, the launchers and ledgers, <code>log.checkMesh</code> and the <code>constant/birth_certificate.json</code> sidecar this rung was admitted on

A.10 What is still open

1. **D1/D1s, mechanism to a line.** The subsystem, the spatial signature and now the recorded construct are named — the limiter's `min/max` selection, switched by one word — but the defective source line in the `cellLimited/limitedGrad` reverse sweep is not. Reaching it still requires an instrumented rebuild of `libDASolver.so`; what has changed is that the reproducer to instrument can now be a single process (Section 3).
2. **D1, reproducer on a stock upstream tutorial.** The measured case is lab-built — its mesh recipe and case directories are archived and shippable, but it is not an official tutorial. A tutorial-based reproducer varying the `decomposeParDict daOption` on an official case should exist before filing.
3. **D1, the one-mesh-family confound.** Not repaired — *confirmed*, twice, by the two sharpest tests available: the BC alone transplanted onto a clean case from another mesh family does nothing, and both ingredients together do nothing either. Every decomposition-borne measurement in this lab still lives on one 2,777-cell Ahmed background family. A filed reproducer must scope itself accordingly.
4. **D1, the unseparated third correlate.** `primalMinResTol` is $1e-4$ on both defective cases and $1e-6/1e-8$ on all five clean ones, and it has no within-case control, because `DAFoam` treats an unreached tolerance as a primal failure rather than a depth setting. Whether a loose primal is a necessary co-ingredient is unmeasured, and the acquisition null raised rather than lowered its standing.

5. **D1, the unbought operator control.** No state-override control was run for either lever of Table 6; the linearization-state confound is bounded indirectly and the ~ 0.4 core-min direct control should be taken next time a container is warm.
6. **D1s, novelty.** The community sweep predates the limiter findings and has not been re-run for them. No priority is claimed, and the gap should close before any filing.
7. **D2, the primary reference.** The IDWarp AIAA paper is unread and unobtainable from this host; the “undocumented” characterisation rests on code, docs and release notes only, and is stated conditionally throughout.
8. **D2, the second regime.** Real, measured at $\sim 1.26\%$ on ONERA M6, untouched by the shipped patch by design. The reparameterisation that subsumes both regimes is derived and numerically validated but not implemented, and cannot be primal-bit-identical.
9. **D2, the workaround’s price.** The mesh-quality cost of `useRotations: False` under large deformation has not been measured.
10. **D3, the hump.** Krylov convergence rate against a memory envelope. A prepared configuration wrapping the exact sub-solve in Richardson with a capped restart is staged but unrun.
11. **D4, the ceiling is bracketed and not located.** Two rungs are verified (21,840 and 42,120 cells); 79,560 stagnates on ample memory. No mesh between those two sizes has been run, and one arm at an intermediate size would halve the bracket for an estimated 40–60 core-min. 99,840 cells and above are untouched. Nothing is claimed about whether a different configuration reaches 79,560 — the named and unrun candidate is the restart lever (`gmresRestart` 200 $\rightarrow$ 1000), whose mechanism, retaining search directions instead of discarding them twenty times, matches the observed failure mode and whose memory cost that rung can afford.
12. **D4, the coarse space that was not tested.** Stage 2 ran algebraic multigrid at its documented defaults and it failed (Section 7.13). What remains untested is the thing the elimination table actually points at: a coarse space appropriate to a convection-dominated nonsymmetric compressible adjoint, or AMG with smoothers chosen for that operator rather than for an elliptic one. It is not a knob, it is not cheap, and no arm here bears on it. The same is true one notch weaker of `dgmres`, declined in stage 2’s filing on the ground that it shares arm 1’s family and mechanism.
13. **D4, the gradient’s unverified components.** At rung 1 two probed components carry no verdict (`twist`[1], `shape`[5], both step-inconsistent at the tested steps); rung 2’s three are all evaluable, which locates that failure in the small signals rather than in the gradient but does not retroactively verify rung 1’s two. The remaining components of both vectors are unprobed, and no optimization has been run on either gradient. The primal reaches its plateau below the 10^{-4} hard gate without attaining the requested 10^{-8} , which is disclosed on the FD arms’ faces.
14. **D4, the mechanism.** *Why* dropping $\text{div}(\phi_d, p)$ from the preconditioner’s pressure equation converts breakdown into convergence on this case is not demonstrated here — only that it does, reproducibly, at two rungs and against a control. The account in Section 7.8 of why strengthening then helps is a plausibility argument consistent with the measurements, not a derivation. The 14-decade diagonal spread of the assembled PC matrix has not been re-measured with the option active, and that measurement is cheap.
15. **D4, the restart hypothesis.** The mechanism offered for why strengthening the preconditioner helps at one rung and collapses at the next — that it destabilises the GMRES restart

recomputation, and that whether this bites depends on whether convergence completes inside the first restart cycle — has two supporting points and is **unproven**. Its falsifiable prediction is that fill 1 also collapses at 42,120 cells. That arm is cheap and has not been run.

16. **D4, the mechanism behind the ceiling, is unknown and stays unknown.** Twelve candidate causes are eliminated by measurement (Section 7.10) — the last two only because the option channel was patched open (Section 7.13) — and no thirteenth is named. **Twelve eliminations are not a proof of the thirteenth**; they bound the ignorance rather than resolving it, and the surviving characterisation is negative: $\kappa \approx 10^{11}$ after preconditioning, insensitive to every parameter the build exposes. The non-normality measurement carries its own limit — it is taken on the assembled dRdWTPC, not on the preconditioned operator whose normality governs GMRES, which this build never assembles — so it refutes a discriminator without certifying the operator.
17. **D6, the fixes — one item CLOSED and the rest still open.** This entry previously read that the three graded fixes were drafted and untested, and that “reordering KSPSetFromOptions is behaviour-preserving” was an argument rather than a measurement. **That clause is now closed by measurement and is struck rather than deleted:** fix (a) is implemented, and Gate A returns rung 1’s exact 368/383 iteration counts, **reason 2** on both, cold signature identical to sixteen digits (Table 28). What replaces it is narrower and still open — the regression is one configuration on one host, so an upstream reviewer would want the same check on an incompressible case and a Richardson-wrapped one; the effective-value echo that the revised recommendation now leads with is drafted and *unimplemented*, so the diagnosability gap is closed on paper and not in code; and fixes (b) and (c) remain untested. Nothing about any of it has been sent upstream, and the patched image is local to one host.
18. **D4, the two negative signatures.** Whether the stagnation at rung 3 and the denormal breakdown at the archived rungs share a mechanism is not established; they are different signatures and no arm here identifies either.
19. **D4/D5, the unverifiable levers.** The dead-lever audit left 16 lever/conclusion pairs whose activity no surviving log can settle; per the charter they ship marked unverifiable-from-logs rather than assumed. Separately, the 11 warm-flagged M6 variant nulls were closed on 2026-08-10 by *retirement* rather than by rerun: each tested a lever against a preconditioner that was never on, so a cold rerun would certify nulls about a superseded configuration. The audit’s cold-rerun-before-citation requirement is satisfied by never citing them, which is the stricter option and is stated as such at both sites.
20. **D5, upstream’s intent.** Whether the time-0 rewrite is deliberate (a restart convenience) or incidental is not established from the source here; the finding as stated is about its silence, not its existence.

Filing status, restated plainly. Nothing in this document has been filed, sent, or posted — not the decomposition report, not its three addenda, not the comment on `mdolab/idwarp#57`, not the sub-PC pull request, not the option-channel defect candidate that is complete and filing-ready, and not the other two silent-configuration findings of Section 7, which have no prepared report at all. All of it is **PREPARED and HELD**. That the option-channel fix is now implemented, regression-controlled and archived as a 28-line diff changes what a filing would contain and not whether one has happened: the patched image is local to one measurement host, nothing has been pushed to any repository, and no maintainer has been contacted about it or anything else here. Submissions are parked at the owner’s discretion, and whether any of it goes upstream is the lab principal’s decision, not the authors’. This document is what the case file looks like while it waits.